Computer Networks

Mihir Rao

Contents

1 Computer Networks and the Internet **3**
1.1 What is the Internet? . . . 3
1.2 The network edge . . . 4
1.3 The network core . . . 6
1.4 Performance: loss, delay, and throughput . . . 9
1.5 Security . . . 11
1.6 Protocol layers and service models . . . 11
1.7 History . . . 13

2 The Application Layer **14**
2.1 Principles of network applications . . . 14
2.2 Web and HTTP . . . 16
2.3 E-mail, SMTP, IMAP . . . 23
2.4 The Domain Name System (DNS) . . . 24
2.5 Video streaming and content distribution networks . . . 28
2.6 Socket programming with UDP and TCP . . . 30

3 The Transport Layer **33**
3.1 Transport-layer services . . . 33
3.2 Multiplexing and demultiplexing . . . 33
3.3 Connectionless transport: UDP . . . 34
3.4 Principles of reliable data transfer . . . 35
3.5 Connection-oriented transport: TCP . . . 46
3.6 Principles of congestion control . . . 54
3.7 TCP congestion control . . . 55
3.8 Evolution of transport-layer functionality . . . 60

4 The Network Layer: The Data Plane **62**
4.1 Network layer overview . . . 62
4.2 What's inside a router . . . 63
4.3 IP: the Internet Protocol . . . 68
4.4 Generalized forwarding . . . 76
4.5 Middleboxes and Internet architecture . . . 77

5 The Network Layer: The Control Plane **79**
5.1 Introduction . . . 79
5.2 Routing algorithms . . . 79
5.3 Intra-AS routing in the Internet: OSPF . . . 84
5.4 Routing among the ISPs: BGP . . . 85
5.5 The SDN control plane . . . 87
5.6 ICMP: the Internet Control Message Protocol . . . 91
5.7 Network management and SNMP, NETCONF/YANG . . . 91

6 The Link Layer and LANs **94**
6.1 Introduction to the link layer . . . 94
6.2 Error-detection and -correction techniques . . . 95
6.3 Multiple access links and protocols . . . 96
6.4 Switched local area networks . . . 99
6.5 Link virtualization: a network as a link layer . . . 103
6.6 Data center networking . . . 104
6.7 Retrospective: a day in the life of a web page request . . . 105

7 Wireless and Mobile Networks **107**
7.1 Introduction . . . 107
7.2 Radio: the physical layer . . . 108
7.3 Sharing a wireless access channel . . . 111
7.4 WiFi wireless LANs . . . 113
7.5 The 5G radio access network . . . 114
7.6 Common challenges: network discovery, frame scheduling, energy . . . 116
7.7 The wireless core network . . . 118
7.8 Mobility . . . 120
7.9 Bluetooth, satellite, and IoT wireless networks . . . 121

8 Security in Computer Networks **124**
8.1 What is network security? . . . 124
8.2 Principles of cryptography . . . 124
8.3 Authentication, digital signatures, message integrity, shared key agreement . . . 126
8.4 Securing e-mail . . . 129
8.5 Securing TCP connections: TLS . . . 129
8.6 Network-layer security: IPsec . . . 131
8.7 Security in wireless and mobile networks . . . 133
8.8 Operational security: firewalls and IDS . . . 135

Lecture 1: Computer Networks and the Internet

1.1 What is the Internet?

In a *nuts and bolts* view the Internet consists of:

- billions of connected computing devices, called HOSTS or END SYSTEMS, running network applications at the Internet's edge (phones and laptops, but also cameras, appliances, cars, game consoles, and medical monitors);
- PACKET SWITCHES (routers and switches), which forward PACKETS, chunks of data;
- COMMUNICATION LINKS (fiber, copper, radio, satellite), each with a transmission rate, its BANDWIDTH;
- NETWORKS: collections of devices, routers, and links managed by one organization, such as a mobile network, a home network, an enterprise network, a local or regional ISP, a national or global ISP, a datacenter network, or a content provider network.

The Internet is a *network of networks*: interconnected Internet Service Providers (ISPs). PROTOCOLS, which control the sending and receiving of messages, are everywhere: HTTP (Web), streaming video, Zoom, TCP, IP, WiFi, 4G/5G, Ethernet. Internet standards are RFCs (Requests for Comments) produced by the IETF (Internet Engineering Task Force).

In a *services* view the Internet is infrastructure that provides services to applications (Web, streaming video, multimedia teleconferencing, email, games, e-commerce, social media, interconnected appliances). It provides a programming interface to distributed applications: hooks that let sending and receiving applications connect to and use the Internet's transport service, with service options analogous to those of the postal service.

Definition: A PROTOCOL defines the format and order of messages sent and received among network entities, and the actions taken on message transmission and receipt.

Human protocols (asking the time, asking a question, introductions) have the same ingredients: specific messages are sent, and specific actions are taken when a message is received or another event occurs. Network protocols are the same with computers (devices) in place of humans, and all communication activity in the Internet is governed by protocols (Figure 1).

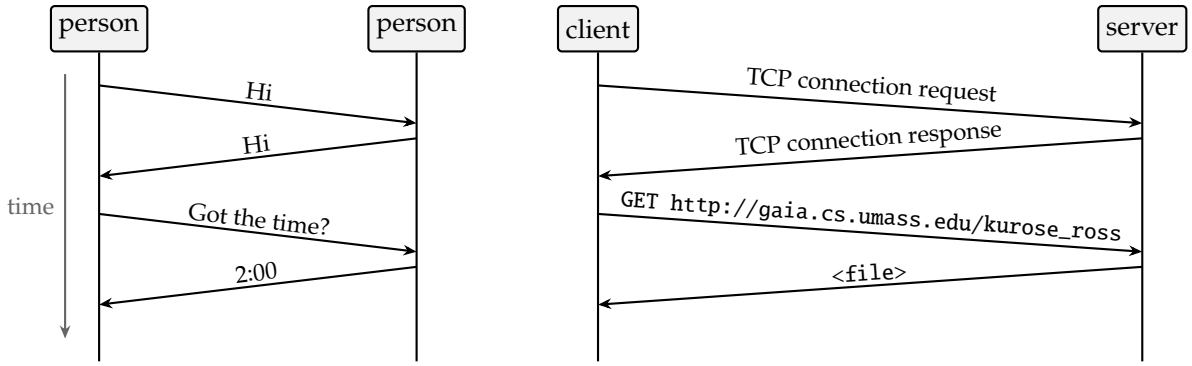


Figure 1: A human protocol and a computer network protocol. The client opens a TCP connection and then requests a Web page with an HTTP `GET`; the server replies with the file.

1.2 The network edge

The Internet has three parts. The NETWORK EDGE consists of the hosts, clients and servers (servers often sit in data centers). ACCESS NETWORKS and PHYSICAL MEDIA are the wired and wireless communication links that connect end systems to the first (*edge*) router. The NETWORK CORE consists of interconnected routers, a network of networks. End systems connect through residential access networks, institutional access networks (school, company), and mobile access networks (WiFi, 4G/5G).

Cable and DSL. Cable-based access uses FREQUENCY DIVISION MULTIPLEXING (FDM): different channels (video, data, control) are transmitted in different frequency bands over the same cable. Data and TV are carried at different frequencies over a *shared* cable distribution network: a network of cable and fiber (HFC, hybrid fiber coax) attaches homes to the ISP router, so homes share the access network up to the *cable headend*, where a cable modem termination system (CMTS) connects to the ISP. A digital subscriber line (DSL) instead uses the existing telephone line to the central office: voice and data travel at different frequencies over a *dedicated* line, and at the central office a DSL access multiplexer (DSLAM) sends data to the Internet and voice to the telephone network.

Access	Medium to ISP	Sharing	Downstream	Upstream
Cable (HFC)	cable and fiber	shared to headend	40 Mbps–1.2 Gbps	30–100 Mbps
DSL	telephone line	dedicated to central office	24–52 Mbps	3.5–16 Mbps

Both are asymmetric: the downstream rate is higher than the upstream rate (cable rates are "up to").

Home, wireless, enterprise, and data center networks. A home network connects wired and wireless devices to a cable or DSL modem through a router (with firewall and NAT), wired Ethernet (1 Gbps), and a WiFi wireless access point (54 or 450 Mbps); the router, access point, and modem

are often combined in a single box. A shared *wireless* access network connects end systems to a router via a base station, also called an ACCESS POINT. Wireless local area networks (WLANs) typically reach within or around a building (about 100 ft) at 11, 54, or 450 Mbps (802.11b/g/n, WiFi); wide-area cellular access networks (4G/5G), provided by a mobile network operator, reach tens of kilometers at tens of Mbps. Enterprise networks (companies, universities) mix wired and wireless link technologies connecting a mix of switches and routers: Ethernet at 100 Mbps, 1 Gbps, or 10 Gbps and WiFi access points at 11, 54, or 450 Mbps, with an institutional router linking to the ISP. In data center networks, high-bandwidth links (10s to 100s of Gbps) connect hundreds to thousands of servers to each other and to the Internet.

Sending packets. A host takes an application message, breaks it into smaller chunks, PACKETS, of length L bits, and transmits each packet into the access network at TRANSMISSION RATE R (also called link capacity or link bandwidth). The PACKET TRANSMISSION DELAY, the time needed to transmit an L-bit packet into the link, is

$$d_{\text{trans}} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}.$$

Physical media. A bit propagates between transmitter/receiver pairs; the PHYSICAL LINK is what lies between transmitter and receiver. In GUIDED MEDIA signals propagate in solid media (copper, fiber, coax); in UNGUIDED MEDIA they propagate freely, as radio does.

- *Twisted pair*: two insulated copper wires; Category 5 carries 100 Mbps and 1 Gbps Ethernet, Category 6 carries 10 Gbps Ethernet.
- *Coaxial cable*: two concentric copper conductors, bidirectional; *broadband*, with multiple frequency channels of 100s of Mbps each.
- *Fiber optic cable*: glass fiber carrying light pulses, each pulse a bit; high-speed point-to-point transmission (10s–100s of Gbps) and a low error rate, since repeaters are spaced far apart and the fiber is immune to electromagnetic noise.
- *Wireless radio*: the signal is carried in bands of the electromagnetic spectrum, with no physical wire; it is broadcast and half-duplex (sender to receiver), and suffers from the propagation environment: reflection, obstruction by objects, interference and noise.

Radio link type	Characteristics
Wireless LAN (WiFi)	10–100s of Mbps; 10s of meters
Wide-area (4G/5G cellular)	100s of Mbps over about 10 km
Bluetooth	cable replacement: short distances, limited rates
Terrestrial microwave	point-to-point; 45 Mbps channels
Satellite	downlink up to under 100 Mbps (Starlink); 270 ms end-end delay (geostationary)

1.3 The network core

The network core is a mesh of interconnected routers. In PACKET SWITCHING, hosts break application-layer messages into packets, and the network forwards packets from one router to the next, across links on the path from source to destination. The core has two key functions (Figure 2).

Definition: FORWARDING (also called *switching*) is a *local* action: moving a packet arriving on a router's input link to the appropriate output link, by looking up the destination address in the packet's header in a local FORWARDING TABLE. ROUTING is a *global* action: determining the source-to-destination paths taken by packets, using ROUTING ALGORITHMS, which compute the forwarding tables.

On a road trip from San Jose to Northampton, routing is planning the whole route across the country, and forwarding is choosing the exit at each intersection along it.

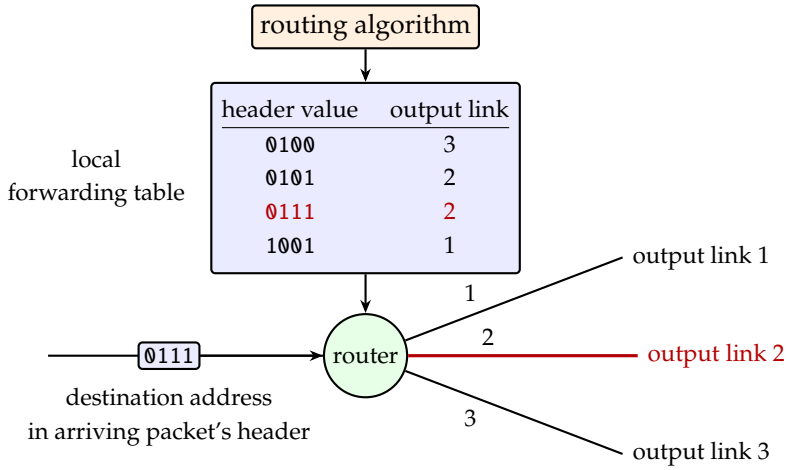


Figure 2: Forwarding and routing. The routing algorithm computes the local forwarding table; a packet whose header carries 0111 is forwarded to output link 2.

Store and forward. Transmitting (pushing out) an L-bit packet into a link at R bps takes L/R seconds. Routers STORE AND FORWARD: the entire packet must arrive at the router before it can be transmitted on the next link. For example, with $L = 10$ Kbits and $R = 100$ Mbps the one-hop transmission delay is $10^4/10^8$ s $= 0.1$ ms.

Queueing and loss. Queueing occurs when work arrives faster than it can be serviced. In Figure 3, hosts A and B send over 100 Mb/s links to a router whose output link runs at only 1.5 Mb/s. If the arrival rate (in bps) to a link exceeds its transmission rate (bps) for some period of time, packets queue, waiting to be transmitted on the output link, and packets can be dropped (lost) if the memory (buffer) in the router fills up.

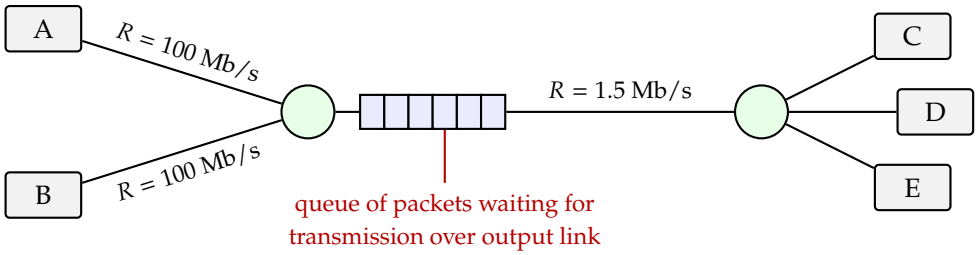
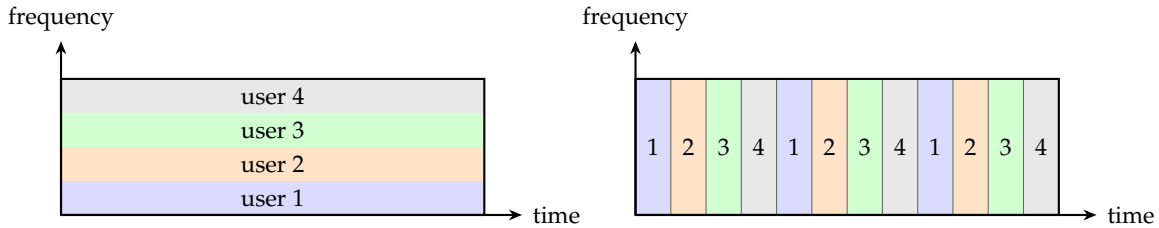


Figure 3: Packets from A and B queue at the router for the slower output link.

Circuit switching. The alternative to packet switching is CIRCUIT SWITCHING, in which end-to-end resources are allocated to, and reserved for, a "call" between source and destination. If each link has four circuits, a call might get the second circuit of one link and the first circuit of the next. Resources are dedicated, with no sharing, so the call gets circuit-like (guaranteed) performance, but a circuit segment sits idle when its call does not use it. Circuit switching is commonly used in traditional telephone networks. A link is divided into circuits in one of two ways (Figure 4):

- FREQUENCY DIVISION MULTIPLEXING (FDM): optical or electromagnetic frequencies are divided into narrow frequency bands; each call is allocated its own band and can transmit at the maximum rate of that narrow band.
- TIME DIVISION MULTIPLEXING (TDM): time is divided into slots; each call is allocated periodic slots and can transmit at the maximum rate of the (wider) frequency band, but only during its slots.



(a) FDM (b) TDM (slot numbers are users)

Figure 4: Frequency division and time division multiplexing of a link among four users.

Example: *Packet switching versus circuit switching.* Users share a 1 Gb/s link; each user sends at 100 Mb/s when active and is active 10% of the time. Circuit switching can support 1000/100 = 10 users. With packet switching and 35 users, if users are active independently the number active at once is binomial with $n = 35$ and $p = 0.1$, and

$$\mathbb{P}(\text{more than 10 active}) = \sum_{k=11}^{35} \binom{35}{k} (0.1)^k (0.9)^{35-k} \approx 0.0004,$$

so 35 users can almost always be served at full rate.

Packet switching is great for *bursty* data, which sometimes has data to send and sometimes not: it shares resources and is simpler, with no call setup. But excessive congestion is possible, causing packet delay and loss from buffer overflow, so protocols are needed for reliable data transfer and congestion control. Providing circuit-like behavior with packet switching is complicated; various techniques try to make packet switching as circuit-like as possible.

Internet structure. Hosts connect to the Internet via ACCESS ISPs, and access ISPs must in turn be interconnected so that any two hosts, anywhere, can send packets to each other. The resulting network of networks is very complex, and its evolution is driven by economics and national policies. Built up step by step:

1. Connecting each of the millions of access ISPs directly to every other does not scale: it takes $O(N^2)$ connections.
2. Each access ISP could connect to one global *transit* ISP; *customer* and *provider* ISPs have an economic agreement.
3. If one global ISP is a viable business there will be competitors (ISPs A, B, C), and they will want to be connected to each other, through PEERING LINKS and INTERNET EXCHANGE POINTS (IXPs).
4. Regional networks may arise to connect access networks to ISPs.
5. Content provider networks (such as Google, Microsoft, Akamai) may run their own networks, to bring services and content close to end users.

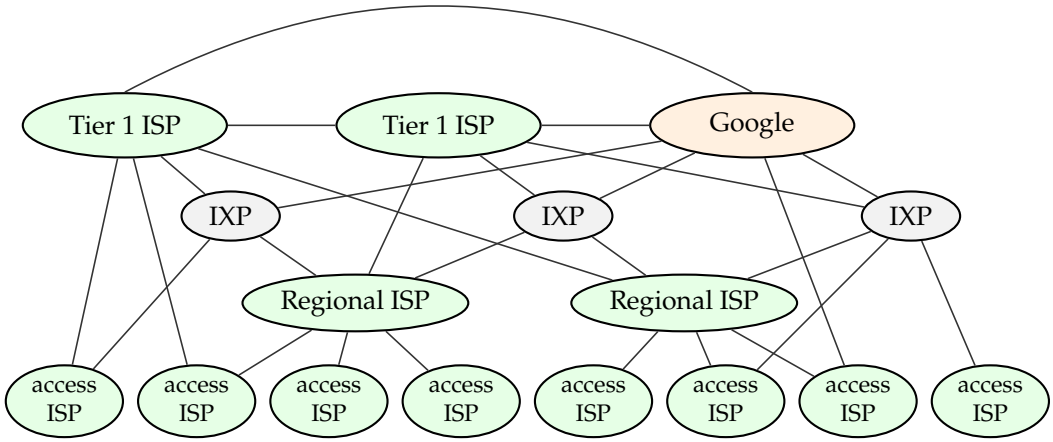


Figure 5: The Internet as a network of networks: tier-1 ISPs and a content provider network at the center, connected to each other, to IXPs, to regional ISPs, and to access ISPs.

At the "center" is a small number of well-connected large networks (Figure 5): TIER-1 commercial ISPs (such as Level 3, Sprint, AT&T, NTT) with national and international coverage, and content provider networks (such as Google, Facebook), private networks that connect their data centers to the Internet, often bypassing tier-1 and regional ISPs.

1.4 Performance: loss, delay, and throughput

Packets queue in router buffers, waiting their turn for transmission; the queue length grows when the arrival rate to a link (temporarily) exceeds the output link capacity, and packets are lost when the memory holding queued packets fills up.

Definition: The NODAL DELAY of a packet at a router has four sources (Figure 6):

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}.$$

- d_{proc}, NODAL PROCESSING: checking bit errors and determining the output link; typically under a microsecond.
- d_{queue}, QUEUEING DELAY: time waiting at the output link for transmission; it depends on the congestion level of the router.
- $d_{\text{trans}} = L/R$, TRANSMISSION DELAY, for a packet of L bits on a link of rate R bps.
- $d_{\text{prop}} = d/s$, PROPAGATION DELAY, for a physical link of length d and propagation speed $s \approx 2 \times 10^8$ m/s.

The transmission and propagation delays are very different quantities: d_{trans} depends on the packet length and the link rate, and d_{prop} on the length of the link.

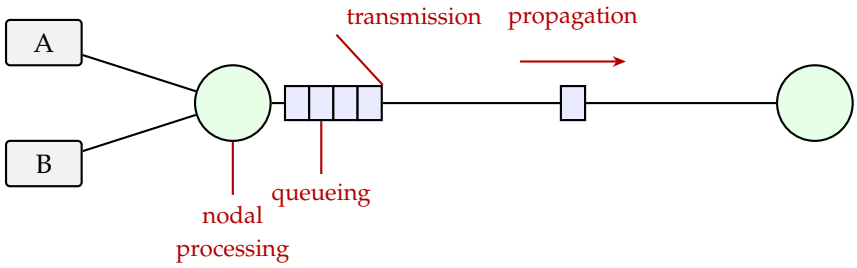


Figure 6: Where the four components of nodal delay arise.

Example: *Caravan analogy.* Cars are bits, a ten-car caravan is a 10-bit packet, and a toll booth is a link's transmitter; toll booths are 100 km apart. If a booth takes 12 s to service a car (the bit transmission time) and cars "propagate" at 100 km/hr, pushing the whole caravan through the first booth onto the highway takes $12 \cdot 10 = 120$ s, and the last car then needs 100 km/(100 km/hr) = 1 hr to reach the second booth: the caravan is lined up before the second booth after 62 minutes. If instead cars propagate at 1000 km/hr and a booth takes 1 min per car, the first car reaches the second booth after $1 + 6 = 7$ min, when three cars are still at the first booth: bits can arrive at the next router before the whole packet has been transmitted by the previous one.

Queueing delay. Let a be the average packet arrival rate, L the packet length (bits), and R the link bandwidth (bit transmission rate). The TRAFFIC INTENSITY

$$\frac{L \cdot a}{R} = \frac{\text{arrival rate of bits}}{\text{service rate of bits}}$$

governs the average queueing delay (Figure 7): it is small when $La/R \approx 0$ and large as $La/R \to 1$; if $La/R > 1$, more work arrives than can be serviced and the average delay is infinite.

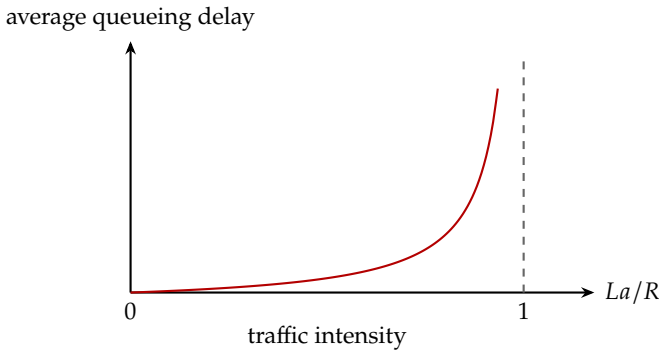


Figure 7: Average queueing delay as a function of traffic intensity (schematic).

Traceroute. The `traceroute` program measures the delay from a source to each router along the end-to-end path to a destination. For every i, it sends three packets that will reach router i on the path (their time-to-live field has value i); router i returns them to the sender, which measures the interval between transmission and reply. In a trace from `gaia.cs.umass.edu` to `www.eurecom.fr`, the three measurements to the first campus router `cs-gw` are 1, 1, and 2 ms; hop 7 (`nycm-wash.abilene.ucaid.edu`) answers in 22 ms and hop 8 (`62.40.103.253`) in 104–109 ms, a jump caused by a trans-oceanic link; hops 17 and 18 print `* * *`, meaning no response (probe lost, or router not replying); and the destination `fantasia.eurecom.fr` at hop 19 answers in 128–136 ms. Delays can even appear to decrease along the path (hop 10 took 113–121 ms, hop 11 112–114 ms), since each probe is a separate measurement that sees its own queueing delays.

Packet loss. The queue (buffer) preceding a link has finite capacity, so a packet arriving to a full queue is dropped (lost). A lost packet may be retransmitted by the previous node, by the source end system, or not at all.

Throughput. THROUGHPUT is the rate (bits per unit time) at which bits are sent from sender to receiver: *instantaneous* throughput is the rate at a given point in time, *average* throughput the rate over a longer period. Picture a server sending a file of F bits as fluid through a pipe of rate R_s (bits/sec) followed by a pipe of rate R_c to the client. If $R_s < R_c$ the average end-to-end throughput is R_s; if $R_s > R_c$ it is R_c. In general it is set by the BOTTLENECK LINK, the link on the end-to-end path that constrains end-to-end throughput. If 10 connections, each with server link R_s and client link

R_c, (fairly) share a backbone link of R bits/sec, the per-connection end-to-end throughput is

$$\min\big(R_c,\, R_s,\, R/10\big);$$

in practice R_c or R_s is often the bottleneck.

1.5 Security

The Internet was not originally designed with (much) security in mind. Its original vision was "a group of mutually trusting users attached to a transparent network," and protocol designers have been playing catch-up, with security considerations in all layers. We need to think about how attackers can attack computer networks, how to defend networks against attacks, and how to design architectures that are immune to attacks.

- PACKET SNIFFING (interception): on broadcast media (shared Ethernet, wireless), a promiscuous network interface reads and records all packets passing by, including passwords. Wireshark is a free packet sniffer: its packet-capture component (pcap) receives a copy of every Ethernet frame the computer sends or receives and hands it to a packet analyzer.
- IP SPOOFING (fake identity): injecting a packet with a false source address, as when host C sends a packet to A with source address B.
- DENIAL OF SERVICE (DoS): attackers make resources (a server, bandwidth) unavailable to legitimate traffic by overwhelming them with bogus traffic. The attacker selects a target, breaks into hosts around the network (a BOTNET), and sends packets to the target from the compromised hosts.

The lines of defense are: *authentication*, proving you are who you say you are (cellular networks provide a hardware identity via the SIM card; the traditional Internet has no such hardware assist); *confidentiality* via encryption; *integrity checks*, such as digital signatures that prevent or detect tampering; *access restrictions*, such as password-protected VPNs; and *firewalls*, specialized MIDDLEBOXES in access and core networks that are off by default, filter incoming packets to restrict senders, receivers, and applications, and detect and react to DoS attacks.

1.6 Protocol layers and service models

Networks are complex, with many pieces: hosts, routers, links of various media, applications, protocols, hardware, and software. Layering organizes both the structure of a network and the discussion of it. Air travel is an analogy: a trip is a series of steps (ticket purchase, baggage check, gate loading, runway takeoff, airplane routing, then runway landing, gate unloading, baggage claim, ticket complaint), and these group into a ticketing service, a baggage service, a gate service, a runway service, and a routing service.

Definition: In a LAYERED architecture, each layer implements a service via its own internal-layer actions, relying on services provided by the layer below.

Layering gives an explicit structure that identifies a complex system's pieces and their relationships (a layered REFERENCE MODEL for discussion), and its modularity eases maintenance and updating: a change in the *implementation* of a layer's service is transparent to the rest of the system (a change in gate procedure does not affect the rest of air travel).

Layer	Function	Examples
application	supporting network applications	HTTP, IMAP, SMTP, DNS
transport	process-to-process data transfer	TCP, UDP
network	routing of datagrams from source to destination	IP, routing protocols
link	data transfer between neighboring network elements	Ethernet, 802.11 (WiFi), PPP
physical	bits "on the wire"	

Encapsulation. Each layer uses the services of the layer below and adds its own header (Figure 8). Applications exchange MESSAGES M to implement some application service using the services of the transport layer. The transport-layer protocol transfers M (e.g., reliably) from one *process* to another using the services of the network layer: it ENCAPSULATES M with a transport-layer header H_t, used by the transport protocol to implement its service, to create a transport-layer SEGMENT $[H_t \mid M]$. The network-layer protocol transfers the segment from one *host* to another using link-layer services, adding a header H_n to create a network-layer DATAGRAM $[H_n \mid [H_t \mid M]]$. The link-layer protocol transfers the datagram from a host to a *neighboring* host, adding H_l to create a link-layer FRAME. Like nested stacking (Matryoshka) dolls, each unit contains the one above. Along the path, a link-layer switch processes the frame only up to the link layer, and a router up to the network layer, where it reads the datagram and encapsulates it in a new frame for the next link; the destination removes the headers in reverse order on the way up.

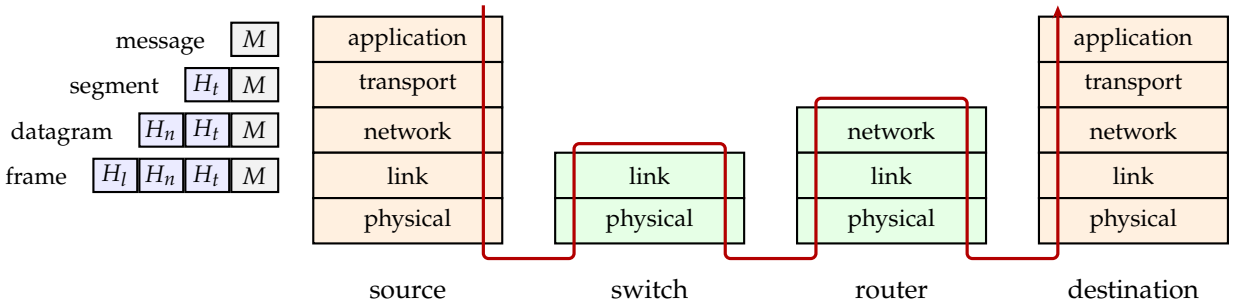


Figure 8: Encapsulation from end to end: the units of data created at the source (left) and the layers each device implements.

The seven-layer ISO/OSI REFERENCE MODEL has two layers not found in the Internet protocol stack, between application and transport: a PRESENTATION layer that lets applications interpret the meaning of data (encryption, compression, machine-specific conventions) and a SESSION layer for

synchronization, checkpointing, and recovery of data exchange. If these services are needed in the Internet, they must be implemented in the application.

1.7 History

1961–1972: early packet-switching principles. 1961, Kleinrock: queueing theory shows the effectiveness of packet switching. 1964, Baran: packet switching in military networks. 1967: ARPAnet conceived by the Advanced Research Projects Agency. 1969: first ARPAnet node operational. 1972: public ARPAnet demo, NCP (Network Control Protocol) as the first host-host protocol, the first e-mail program, and 15 ARPAnet nodes.

1972–1980: internetworking, new and proprietary networks. 1970: ALOHAnet satellite network in Hawaii. 1974: Cerf and Kahn's architecture for interconnecting networks. 1976: Ethernet at Xerox PARC. Late 1970s: proprietary architectures (DECnet, SNA, XNA). 1979: ARPAnet has 200 nodes. Cerf and Kahn's internetworking principles define today's Internet architecture: minimalism and autonomy (no internal changes required to interconnect networks), a best-effort service model, stateless routing, and decentralized control.

1980–1990: new protocols, a proliferation of networks. 1982: SMTP e-mail protocol defined. 1983: deployment of TCP/IP; DNS defined for name-to-IP-address translation. 1985: FTP defined. 1988: TCP congestion control. New national networks (CSnet, BITnet, NSFnet, Minitel); 100,000 hosts connected to the confederation of networks.

1990s and 2000s: commercialization, the Web, new applications. Early 1990s: ARPAnet decommissioned. 1991: NSF lifts restrictions on commercial use of NSFnet (decommissioned in 1995). Early 1990s: the Web, built on hypertext (Bush 1945, Nelson 1960s) with HTML and HTTP by Berners-Lee; 1994: Mosaic, later Netscape; late 1990s: commercialization of the Web. Late 1990s to 2000s: more killer applications (instant messaging, P2P file sharing), network security to the forefront, an estimated 50 million hosts and 100 million or more users, and backbone links running at Gbps.

2005 to the present: scale, SDN, mobility, cloud. Aggressive deployment of broadband home access (10–100s of Mbps); 2008: software-defined networking (SDN); increasingly ubiquitous high-speed wireless access (4G/5G, WiFi). Service providers (Google, Facebook, Microsoft) create their own networks that bypass the commercial Internet to connect close to end users, providing near-instantaneous access to social media, search, and video content. Enterprises run their services in the cloud (Amazon Web Services, Microsoft Azure). With the rise of smartphones, more mobile than fixed devices were on the Internet by 2017, and about 15 billion devices were attached in 2023.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 1 slides (9th edition).

Lecture 2: The Application Layer

2.1 Principles of network applications

Network applications include social media, the Web, text messaging, e-mail, multi-user games, streaming stored video (YouTube, Hulu, Netflix), P2P file sharing, voice over IP, real-time video conferencing (Zoom), Internet search, and remote login. Creating a network application means writing programs that run on (different) end systems and communicate over the network, such as web server software that communicates with browser software. No software is written for network-core devices: they do not run user applications, and keeping applications on end systems allows rapid application development and propagation.

Definition: In the client-server paradigm a server is an always-on host with a permanent IP address, often in a data center for scaling. Clients contact and communicate with the server; they may be intermittently connected, may have dynamic IP addresses, and do not communicate directly with each other (examples: HTTP, IMAP, FTP). In a peer-to-peer (P2P) architecture there is no always-on server: arbitrary end systems (peers) communicate directly, requesting service from other peers and providing service in return (example: P2P file sharing with BitTorrent).

P2P has self scalability: new peers bring new service capacity as well as new service demands. Peers are intermittently connected and change IP addresses, which makes management complex.

A process is a program running within a host. Two processes on the same host communicate by inter-process communication defined by the operating system; processes on different hosts communicate by exchanging messages. The client process initiates communication and the server process waits to be contacted; applications with P2P architectures have both client and server processes.

Definition: A process sends and receives messages through its socket, which is analogous to a door: the sending process shoves the message out the door and relies on the transport infrastructure on the other side to deliver it to the socket of the receiving process. Two sockets are involved, one on each side. The application process above the socket is controlled by the application developer; the transport, network, link, and physical layers below it are controlled by the operating system.

To receive messages a process needs an identifier. A host has a unique 32-bit IP address, but the IP address does not suffice, since many processes can run on the same host: the identifier consists of the IP address and a port number associated with the process. Example port numbers are 80 for an HTTP server and 25 for a mail server; to send an HTTP message to the gaia.cs.umass.edu web server, use IP address 128.119.245.12 and port 80.

An application-layer protocol defines the types of messages exchanged (request, response), the

message syntax (which fields and how they are delineated), the message semantics (the meaning of the information in fields), and the rules for when and how processes send and respond to messages. Open protocols such as HTTP and SMTP are defined in RFCs, so everyone has access to the definition, which allows interoperability; proprietary protocols (e.g., Zoom) are not.

An application may need four kinds of transport service. *Data integrity*: some applications (file transfer, web transactions) require 100% reliable data transfer, while others (audio) tolerate some loss. *Timing*: some (Internet telephony, interactive games) require low delay to be "effective". *Throughput*: some (multimedia) require a minimum throughput, while elastic applications use whatever throughput they get. *Security*: encryption, data integrity, and so on.

Application	Data loss	Throughput	Time sensitive?
File transfer/download	no loss	elastic	no
E-mail	no loss	elastic	no
Web documents	no loss	elastic	no
Real-time audio/video	loss-tolerant	audio: 5 Kbps–1 Mbps video: 10 Kbps–5 Mbps	yes, 10s of msec
Streaming audio/video	loss-tolerant	same as above	yes, a few secs
Interactive games	loss-tolerant	Kbps+	yes, 10s of msec
Text messaging	no loss	elastic	yes and no

The Internet offers two transport services. TCP provides reliable transport between the sending and receiving processes, flow control (the sender will not overwhelm the receiver), congestion control (the sender is throttled when the network is overloaded), and is connection-oriented (setup is required between the client and server processes); it does not provide timing, a minimum throughput guarantee, or security. UDP provides unreliable data transfer between the sending and receiving processes, and none of reliability, flow control, congestion control, timing, a throughput guarantee, security, or connection setup.

Application	Application-layer protocol	Transport protocol
File transfer/download	FTP [RFC 959]	TCP
E-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
Streaming audio/video	HTTP [RFC 7230], DASH	TCP
Interactive games	WOW, FPS (proprietary)	UDP or TCP

Vanilla TCP and UDP sockets provide no encryption: cleartext passwords sent into a socket traverse the Internet in cleartext. Transport Layer Security (TLS) provides encrypted TCP connections, data integrity, and end-point authentication. TLS is implemented in the application layer: applications use TLS libraries, which in turn use TCP, and cleartext sent into the TLS "socket" traverses the Internet encrypted.

2.2 Web and HTTP

A web page consists of OBJECTS (an HTML file, a JPEG image, a Java applet, an audio file, . . .), each of which can be stored on a different Web server. A page consists of a BASE HTML FILE that includes several referenced objects, each addressable by a URL such as `www.someschool.edu/someDept/pic.gif`, which consists of a host name (`www.someschool.edu`) and a path name (`/someDept/pic.gif`).

Definition: HTTP (hypertext transfer protocol) is the Web's application-layer protocol. It follows the client/server model: the client is a browser that requests, receives (using HTTP), and displays Web objects; the server is a Web server that sends objects (using HTTP) in response to requests.

HTTP uses TCP. The client initiates a TCP connection (creates a socket) to the server, port 80; the server accepts the connection; browser and server exchange HTTP messages; and the TCP connection is closed. HTTP is STATELESS: the server maintains no information about past client requests. Protocols that maintain state are complex: past history must be maintained, and if the server or client crashes, their views of the state may be inconsistent and must be reconciled.

Definition: In NON-PERSISTENT HTTP a TCP connection is opened, at most one object is sent over it, and the connection is closed, so downloading multiple objects requires multiple connections. In PERSISTENT HTTP multiple objects can be sent over a single TCP connection between the client and that server before the connection is closed.

Example: *Non-persistent HTTP.* Suppose that the user enters the URL `www.someSchool.edu/someDepartment/home.index`, a page containing text and references to 10 JPEG images. (1a) The HTTP client initiates a TCP connection to the HTTP server (process) at `www.someSchool.edu` on port 80. (1b) The server, waiting for TCP connections at port 80, accepts the connection and notifies the client. (2) The client sends an HTTP REQUEST MESSAGE containing the URL into the TCP connection socket, indicating that it wants `someDepartment/home.index`. (3) The server receives the request, forms a RESPONSE MESSAGE containing the requested object, and sends it into its socket. (4) The server closes the TCP connection. (5) The client receives the response containing the HTML file and displays it; parsing the file, it finds 10 referenced JPEG objects. (6) Steps 1–5 are repeated for each of the 10 JPEG objects, so the page takes 11 TCP connections.

Definition: The ROUND-TRIP TIME (RTT) is the time for a small packet to travel from client to server and back.

The non-persistent HTTP response time for one object (Figure 9) is one RTT to initiate the TCP connection, plus one RTT for the HTTP request and the first few bytes of the response to return, plus the object's transmission time:

$$\text{non-persistent HTTP response time} = 2\,\text{RTT} + \text{file transmission time}.$$

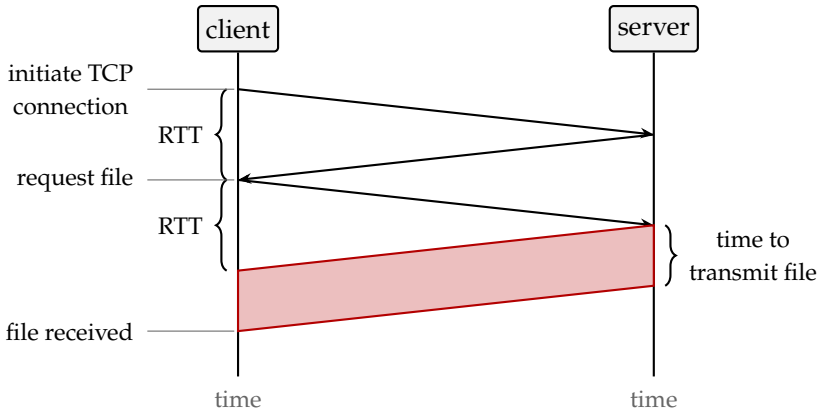


Figure 9: Non-persistent HTTP response time for one object: one RTT to set up the TCP connection, one RTT until the request is answered, and the file's transmission time.

Non-persistent HTTP requires 2 RTTs per object and incurs operating-system overhead for each TCP connection; browsers often open multiple parallel TCP connections to fetch referenced objects in parallel. In persistent HTTP (HTTP 1.1) the server leaves the connection open after sending a response, subsequent HTTP messages between the same client and server are sent over the open connection, and the client sends requests as soon as it encounters a referenced object, so all the referenced objects can take as little as one RTT (cutting the response time in half).

HTTP messages. There are two types of HTTP messages, request and response, both in ASCII (human-readable) format. A request message consists of a REQUEST LINE (method, URL, version), HEADER LINES (each a field name and value), a carriage return and line feed at the start of a line to end the header lines, and an optional ENTITY BODY; every line ends with a carriage return `\r` and line feed `\n`:

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101
↪ Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

With `POST`, user input (a web page often includes form input) is sent from client to server in the entity body of the request. `GET` can also send data to the server, by including user data in the URL field after a '?', as in `www.somesite.com/animalsearch?monkeys&banana`. `HEAD` requests only the headers that would be returned if the URL were requested with a GET. `PUT` uploads a new file (object) to the server, completely replacing any file at the specified URL with the content in the entity body of the PUT request.

A response message consists of a STATUS LINE (protocol, status code, status phrase), header lines, and the data, e.g., the requested HTML file:

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.9 mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

Sample status codes: `200 OK` (request succeeded, requested object later in this message); `301 Moved Permanently` (requested object moved, new location given in the `Location:` field); `400 Bad Request` (request message not understood by the server); `404 Not Found` (requested document not found on this server); `505 HTTP Version Not Supported`. To try HTTP by hand, `nc -c -v gaia.cs.umass.edu 80` (on a Mac; `ncat -C` on Windows) opens a TCP connection to port 80, and typing `GET /kurose_ross/interactive/index.php HTTP/1.1` and `Host: gaia.cs.umass.edu` followed by an empty line sends a minimal but complete GET request.

Cookies. Because HTTP GET/response interaction is stateless, there is no notion of a multi-step exchange of HTTP messages to complete a Web "transaction": client and server need not track the state of a multi-step exchange, all requests are independent of each other, and nothing must be recovered from a transaction that was started but never completed. Contrast a stateful protocol in which a client makes either two changes to a record X or none at all: it locks X, updates X to X', updates X' to X'', and unlocks X, each step acknowledged. If the network connection or the client crashes between the two updates, the server is left holding a lock and a partially updated record.

Definition: Web sites and browsers use COOKIES to maintain some state between transactions. There are four components: (1) a cookie header line in an HTTP response message, (2) a cookie header line in the next HTTP request message, (3) a cookie file kept on the user's host and managed by the browser, and (4) a back-end database at the Web site.

Example: Susan uses a browser on her laptop and visits a specific e-commerce site for the first time. When her initial HTTP request arrives, the site creates a unique ID (the "cookie") and an entry for that ID in its back-end database. Subsequent HTTP requests from Susan to this site contain the cookie ID value, allowing the site to "identify" Susan (Figure 10).

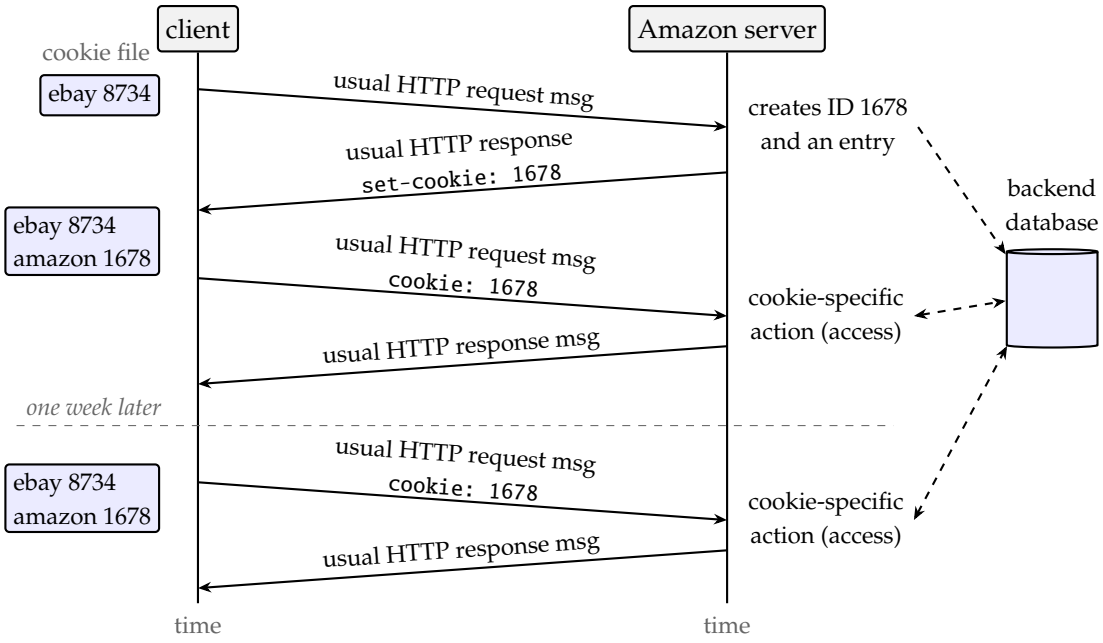


Figure 10: A cookie exchange. The server creates ID 1678 and a database entry on the first visit; the browser stores the ID and returns it in later requests, even a week later, triggering cookie-specific actions.

Cookies are used for authorization, shopping carts, recommendations, and user session state (Web e-mail). There are two ways to keep state: at the protocol endpoints (sender and receiver maintain state over multiple transactions) or in messages (cookies in HTTP messages carry the state). Cookies also raise privacy issues: they permit sites to learn a lot about you on their site, and third-party persistent cookies (TRACKING COOKIES) allow a common identity (the cookie value) to be tracked across multiple web sites.

Example: *Tracking a user's browsing.* A New York Times page embeds an ad served by AdX.com: the browser GETs the base HTML file from nytimes.com, fetches the ad from AdX.com, and displays the composed page. Visiting the sports section, the browser receives a FIRST-PARTY COOKIE (from the site the user chose to visit) `1634` from nytimes.com, which records "1634: sports, 2/15/22". The GET for the ad carries `Referrer: NY Times Sports`, and AdX.com sets a THIRD-PARTY COOKIE (from a site the user did not choose to visit) `7493`, recording "7493: NY Times sports, 2/15/22". The next day the user visits socks.com, which also carries AdX ads; the GET to AdX.com carries `Referrer: socks.com, cookie: 7493`, and AdX records "7493: socks.com, 2/16/22". A day later, reading the arts section of nytimes.com (recorded there as "1634: arts, 2/17/22"), the ad request again carries cookie `7493` with the nytimes.com referrer; AdX records the visit and returns an ad for socks. AdX thus tracks the user's browsing over all sites with AdX ads and can return targeted ads based on the browsing history.

First-party cookies track user behavior on a given website; third-party cookies track behavior across multiple websites without the user ever choosing to visit the tracker site. Tracking may be invisible to the user: rather than a displayed ad, an invisible link can trigger the HTTP GET to the tracker. Third-party tracking via cookies is disabled by default in the Firefox and Safari browsers, and was

to be disabled in Chrome in 2023. Under the EU General Data Protection Regulation (GDPR, recital 30, May 2018), "natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers", whose traces may be used to create profiles of them and identify them. When cookies can identify an individual, they are personal data, subject to GDPR personal data regulations, and the user has explicit control over whether cookies are allowed.

Web caches. The goal of a WEB CACHE (also called a PROXY SERVER) is to satisfy client requests without involving the origin server. The user configures the browser to point to a (local) Web cache, and the browser sends all HTTP requests to the cache. If the object is in the cache, the cache returns it to the client; otherwise the cache requests the object from the origin server, caches the received object, and returns it to the client. The cache is thus both a server (for the original requesting client) and a client (to the origin server). The server tells the cache about an object's allowable caching in the response header, e.g., `Cache-Control: max-age=<seconds>` or `Cache-Control: no-cache`. Caching reduces the response time for client requests (the cache is closer to the client) and reduces traffic on an institution's access link; the Internet is dense with caches, which enables "poor" content providers to deliver content more effectively.

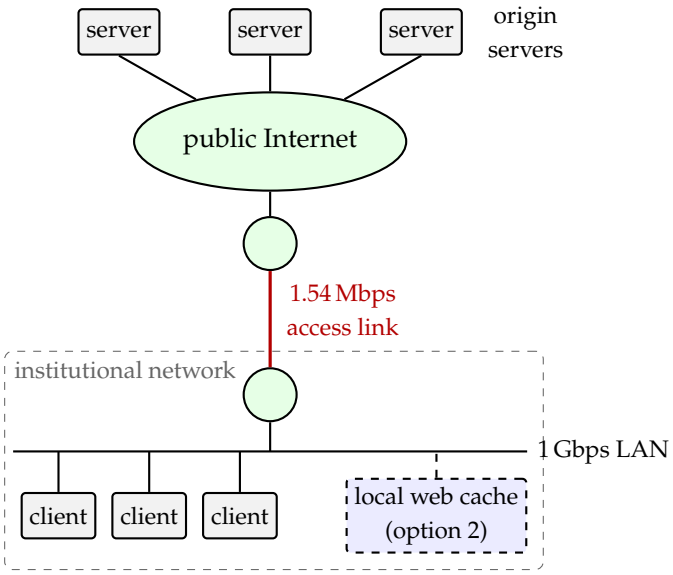


Figure 11: The caching example: an institutional network reaches the origin servers over a 1.54 Mbps access link; option 2 adds a local web cache on the LAN.

Example: *Caching.* An institutional network with a 1 Gbps LAN connects to the public Internet over a 1.54 Mbps access link (Figure 11). The RTT from the institutional router to any origin server is 2 s, web objects are 100 Kbits, and browsers request 15 objects per second from origin servers, an

average data rate to the browsers of 15×100 Kbits/s = 1.50 Mbps. Then

$$\text{access link utilization} = \frac{1.50}{1.54} \approx 0.97, \qquad \text{LAN utilization} = \frac{1.50 \text{ Mbps}}{1 \text{ Gbps}} = 0.0015.$$

The end-to-end delay is the Internet delay plus the access link delay plus the LAN delay, 2 s + minutes + microseconds: at utilization near 1 the access link has large queueing delays.

- *Option 1: buy a* 154 *Mbps access link.* The access link utilization drops to $1.50/154 \approx 0.0097$, and the access link delay to milliseconds; but a faster access link is expensive.
- *Option 2: install a local web cache* (cheap). Suppose the cache hit rate is 0.4. Then 40% of requests are served by the cache with low (millisecond) delay, and 60% are satisfied at the origin, so the rate to the browsers over the access link is $0.6 \times 1.50 = 0.9$ Mbps. The access link utilization is $0.9/1.54 \approx 0.58$, which means low (millisecond) queueing delay at the access link. The average end-to-end delay is

  $$\begin{aligned} &0.6 \times (\text{delay from origin servers}) + 0.4 \times (\text{delay when satisfied at cache}) \\ &\quad = 0.6\,(2.01) + 0.4\,(\text{msecs}) \approx 1.2 \text{ s}, \end{aligned}$$

  lower than with the 154 Mbps link, and cheaper.

Definition: With a CONDITIONAL GET the server does not send an object if the browser has an up-to-date cached version, avoiding the object transmission delay and the use of network resources. The client specifies the date of its browser-cached copy in the request, `If-modified-since: <date>`. If the object has not been modified since that date, the response contains no object: `HTTP/1.0 304 Not Modified`. If it has been modified, the response is `HTTP/1.0 200 OK` followed by the data.

HTTP/2 and HTTP/3. The key goal of HTTP/2 is decreased delay in multi-object HTTP requests. HTTP 1.1 introduced multiple pipelined GETs over a single TCP connection, but the server responds to the GET requests in order (first-come-first-served, FCFS), so a small object may have to wait for transmission behind large objects (HEAD-OF-LINE (HOL) BLOCKING), and loss recovery (retransmitting lost TCP segments) stalls object transmission. HTTP/2 [RFC 7540, 2015] increases the server's flexibility in sending objects to the client. Methods, status codes, and most header fields are unchanged from HTTP 1.1, but the transmission order of requested objects is based on a client-specified object priority (not necessarily FCFS), the server can push unrequested objects to the client, and objects are divided into FRAMES whose transmission is scheduled to mitigate HOL blocking (Figure 12).

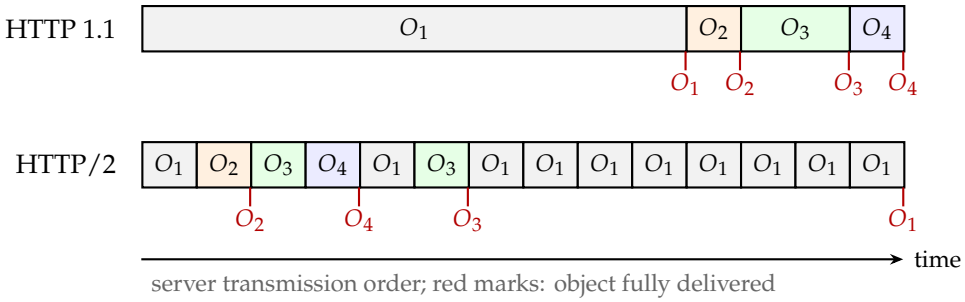


Figure 12: A client requests one large object O_1 and three smaller objects O_2, O_3, O_4 (schematic; time runs to the right, one cell per frame). In HTTP 1.1 the objects are delivered in the order requested, so O_2, O_3, O_4 wait behind O_1. In HTTP/2 frames are interleaved: O_2, O_3, O_4 are delivered quickly and O_1 is slightly delayed.

HTTP/2 over a single TCP connection still has problems. Recovery from packet loss still stalls all object transmissions, so, as in HTTP 1.1, browsers have an incentive to open multiple parallel TCP connections to reduce stalling and increase overall throughput; and there is no security over a vanilla TCP connection. HTTP/3 adds security and per-object error and congestion control (more pipelining) over UDP.

Definition: QUIC (Quick UDP Internet Connections) is an application-layer protocol on top of UDP that increases the performance of HTTP; it is deployed on many Google servers and apps (Chrome, the mobile YouTube app). HTTP/2 over QUIC over UDP replaces the stack HTTP/2 over TLS over TCP:

Layer	HTTP/2 over TCP	HTTP/3
Application	HTTP/2, TLS	HTTP/2 (slimmed), QUIC
Transport	TCP	UDP
Network	IP	IP

QUIC adopts the transport-layer approaches to connection establishment, error control, and congestion control: its specification notes that "readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones". In connection establishment, the reliability, congestion-control, authentication, and encryption state is established in one RTT. TCP (reliability and congestion-control state) followed by TLS (authentication and crypto state) needs two serial handshakes; QUIC needs one. Moreover, a TLS session ticket sent by the server at the end of a QUIC connection is cached at the client, which uses the last session ticket for encryption and authentication of a new connection and sends data immediately, with zero RTT of handshake delay (Figure 13).

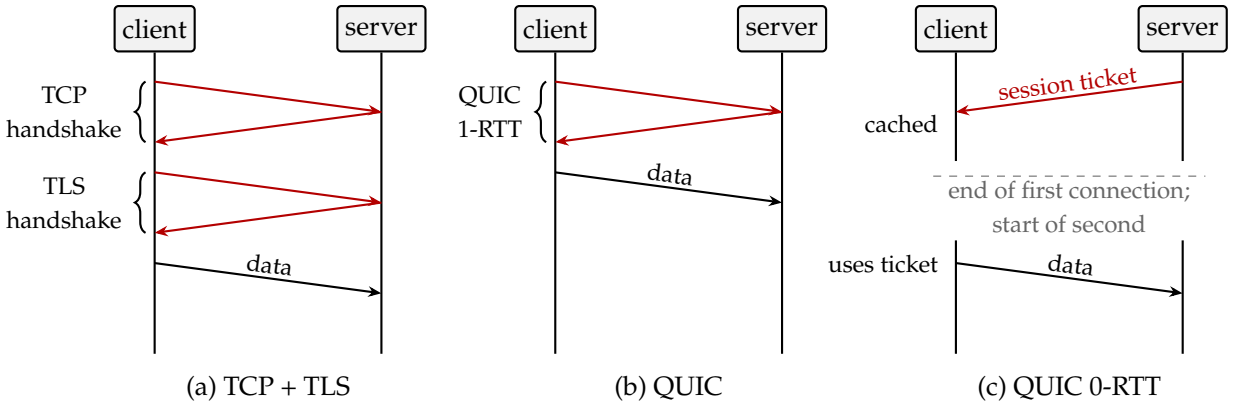


Figure 13: Connection establishment: (a) TCP then TLS, two serial handshakes; (b) QUIC, one handshake; (c) QUIC 0-RTT, where the client reuses a cached session ticket from an earlier connection and sends data at once.

2.3 E-mail, SMTP, IMAP

E-mail has three major components: user agents, mail servers, and the simple mail transfer protocol, SMTP. A USER AGENT ("mail reader", e.g., Outlook or the iPhone mail client) composes, edits, and reads mail messages; outgoing and incoming messages are stored on the server. A MAIL SERVER has a MAILBOX containing incoming messages for each user and a MESSAGE QUEUE of outgoing (to be sent) mail messages. Mail servers use the SMTP protocol between them to send e-mail messages: the client is the sending mail server and the "server" is the receiving mail server.

Definition: SMTP [RFC 5321] uses TCP to reliably transfer e-mail messages from the client (the mail server initiating the connection) to the server, port 25. Transfer is direct, from the sending server (acting like a client) to the receiving server, in three phases: SMTP handshaking (greeting), SMTP transfer of messages, and SMTP closure. Like HTTP it is a command/response interaction: commands are ASCII text, and responses are a status code and phrase.

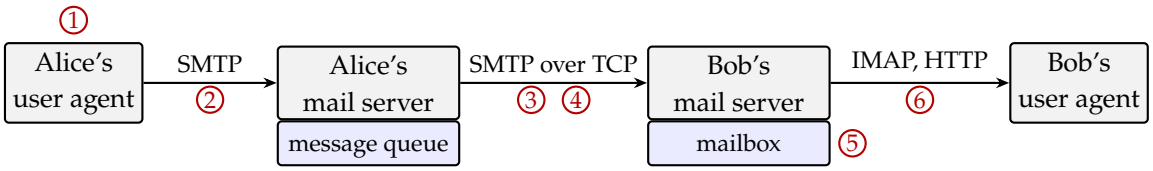


Figure 14: Alice sends e-mail to Bob: SMTP carries the message to Alice's mail server and then, over a TCP connection, to Bob's mail server; Bob retrieves it with a mail access protocol such as IMAP or HTTP.

Example: *Alice sends e-mail to Bob* (Figure 14). (1) Alice uses her user agent to compose a message "to" `bob@someschool.edu`. (2) Her user agent sends the message to her mail server using SMTP, where it is placed in the message queue. (3) The client side of SMTP at her mail server opens a TCP connection with Bob's mail server. (4) The SMTP client sends Alice's message over the TCP connection. (5) Bob's mail server places the message in Bob's mailbox. (6) Bob invokes his user

agent to read the message.

After the TCP connection is set up (one RTT), a sample SMTP interaction between a client `crepes.fr` and a server `hamburger.edu` is

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP uses persistent connections, requires the message (header and body) to be in 7-bit ASCII, and the server uses `CRLF.CRLF` to determine the end of the message. Compared with HTTP: HTTP is client pull while SMTP is client push; both have ASCII command/response interaction and status codes; HTTP encapsulates each object in its own response message, while SMTP sends multiple objects in a multipart message.

SMTP [RFC 5321] is the protocol for exchanging e-mail messages; RFC 2822 defines the syntax of the e-mail message itself (as HTML defines the syntax of web documents). A message consists of header lines, e.g., `To:`, `From:`, `Subject:`, then a blank line, then the body (the "message", ASCII characters only). These header lines are within the body of the SMTP transfer and are different from the SMTP `MAIL FROM:` and `RCPT TO:` commands.

SMTP handles delivery and storage of e-mail messages to the receiver's server; a MAIL ACCESS PROTOCOL handles retrieval from the server. IMAP (Internet Mail Access Protocol) [RFC 3501] keeps messages stored on the server and provides retrieval, deletion, and folders of stored messages on the server. Web mail (Gmail, Hotmail, Yahoo! Mail, etc.) uses HTTP to provide a web-based interface on top of SMTP (to send) and IMAP (or POP) (to retrieve e-mail messages).

2.4 The Domain Name System (DNS)

People have many identifiers (SSN, name, passport number). Internet hosts and routers have an IP address (32 bits), used for addressing datagrams, and a "name", e.g., `cs.umass.edu`, used by humans. The question is how to map between IP addresses and names.

Definition: The DOMAIN NAME SYSTEM (DNS) is a distributed database implemented in a hierarchy of many NAME SERVERS, together with an application-layer protocol by which hosts and DNS servers communicate to resolve names (address/name translation). It is a core Internet function implemented as an application-layer protocol, an instance of keeping complexity at the network's "edge".

DNS provides hostname-to-IP-address translation, HOST ALIASING (canonical and alias names), mail server aliasing, and load distribution (for replicated Web servers, many IP addresses correspond to one name). A centralized DNS would not scale: it would be a single point of failure, would carry an enormous traffic volume (Comcast's DNS servers alone handle 600 billion DNS queries per day, Akamai's 2.2 trillion), would be a distant centralized database, and would be hard to maintain. DNS is a humongous distributed database (about a billion records, each simple) handling many trillions of queries a day, with many more reads than writes; performance matters, since almost every Internet transaction interacts with DNS and milliseconds count. It is organizationally and physically decentralized (millions of different organizations are responsible for their records) and must be "bulletproof" in reliability and security.

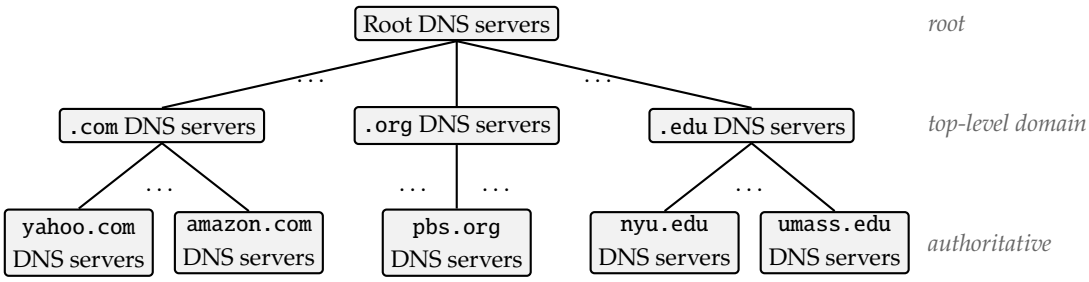


Figure 15: DNS as a distributed, hierarchical database: root servers, top-level domain (TLD) servers, and authoritative servers.

The hierarchy has three levels (Figure 15). As a first approximation, a client wanting the IP address of `www.amazon.com` queries a root server to find a `.com` DNS server, queries the `.com` DNS server to get the `amazon.com` DNS server, and queries the `amazon.com` DNS server to get the IP address of `www.amazon.com`.

- ROOT NAME SERVERS are the official contact of last resort for name servers that cannot resolve a name. They are an incredibly important Internet function (the Internet could not function without them); DNSSEC provides security (authentication, message integrity), and ICANN (Internet Corporation for Assigned Names and Numbers) manages the root DNS domain. There are 13 logical root name "servers" worldwide, each replicated many times.
- TOP-LEVEL DOMAIN (TLD) SERVERS are responsible for `.com`, `.org`, `.net`, `.edu`, `.aero`, `.jobs`, `.museums`, and all top-level country domains (`.cn`, `.uk`, `.fr`, `.ca`, `.jp`). Network Solutions is the authoritative registry for the `.com` and `.net` TLDs, and Educause for `.edu`.
- AUTHORITATIVE DNS SERVERS are an organization's own DNS servers, providing authoritative

hostname-to-IP mappings for the organization's named hosts; they can be maintained by the organization or by a service provider.

When a host makes a DNS query, it is sent to its LOCAL DNS SERVER, which replies either from its local cache of recent name-to-address translation pairs (possibly out of date) or by forwarding the request into the DNS hierarchy for resolution. Each ISP has a local DNS name server (on macOS `scutil --dns`, on Windows `ipconfig /all` shows yours); it does not strictly belong to the hierarchy.

Definition: In an ITERATED QUERY the contacted server replies with the name of the server to contact ("I don't know this name, but ask this server"). In a RECURSIVE QUERY the contacted name server bears the burden of name resolution, which puts a heavy load on the upper levels of the hierarchy.

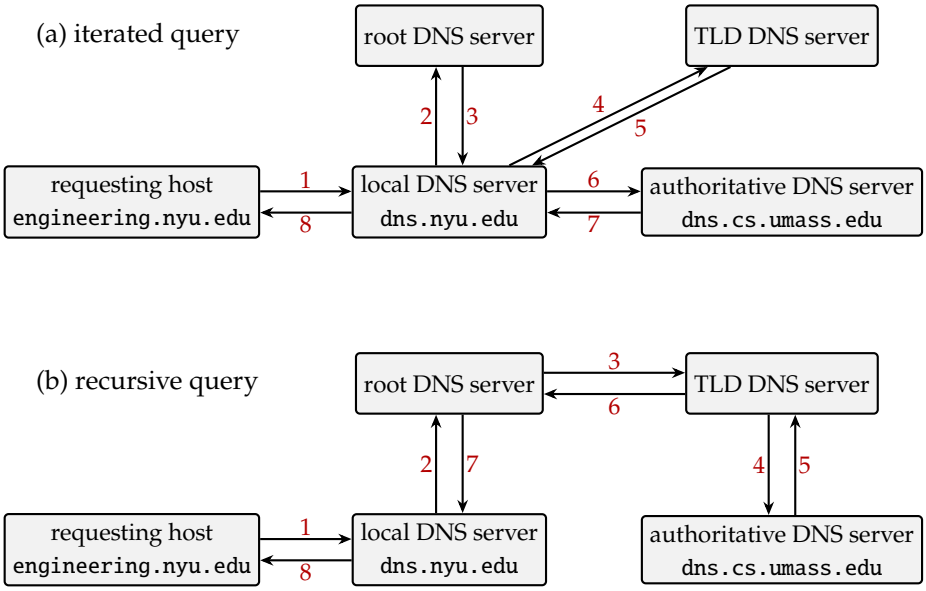


Figure 16: A host at `engineering.nyu.edu` resolves `gaia.cs.umass.edu`, with messages numbered in order. (a) Iterated: the local DNS server queries the root, TLD, and authoritative servers in turn. (b) Recursive: each server queries the next level on the requester's behalf.

Once any name server learns a mapping, it CACHES the mapping and immediately returns it in response to a query. Caching improves response time; cache entries time out (disappear) after some time, the TIME TO LIVE (TTL); and TLD servers are typically cached in local name servers, so root servers are rarely visited. Cached entries may be out of date: if a named host changes its IP address, the change may not be known Internet-wide until all TTLs expire, so DNS provides best-effort name-to-address translation.

Definition: DNS is a distributed database storing RESOURCE RECORDS (RRs) of the form `(name, value, type, ttl)`:

Type	name	value
A	hostname	IP address
NS	domain (e.g., foo.com)	hostname of the authoritative name server for the domain
CNAME	alias name	canonical (real) name
MX	name	name of the SMTP mail server associated with name

For example, www.ibm.com is really servereast.backup2.ibm.com.

DNS query and reply messages have the same format (Figure 17). The message header contains a 16-bit IDENTIFICATION number for the query (the reply to a query uses the same number) and FLAGS: query or reply, recursion desired, recursion available, and reply is authoritative.

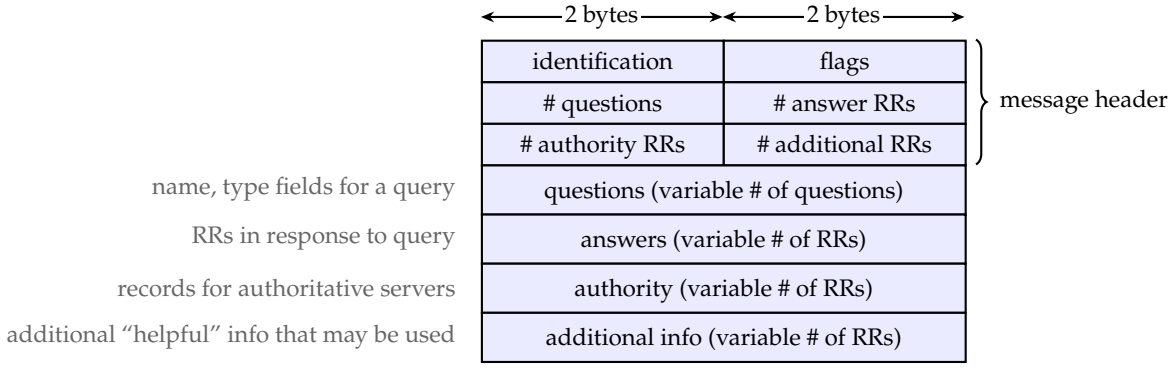


Figure 17: DNS query and reply message format.

Example: *Getting your information into the DNS.* A new startup, "Network Utopia", registers the name networkutopia.com at a DNS REGISTRAR (e.g., Network Solutions), providing the names and IP addresses of its authoritative name servers (primary and secondary). The registrar inserts NS and A RRs into the .com TLD server:

```
(networkutopia.com, dns1.networkutopia.com, NS)
  (dns1.networkutopia.com, 212.212.212.1, A)
```

The startup creates its authoritative server locally with IP address 212.212.212.1, holding a type A record for www.networkutopia.com and a type MX record for networkutopia.com.

DNS faces two kinds of attack. *DDoS attacks* can bombard the root servers with traffic, which has not been successful to date thanks to traffic filtering and because local DNS servers cache the IP addresses of TLD servers, allowing them to bypass the root; bombarding TLD servers is potentially more dangerous. *Spoofing attacks* intercept DNS queries and return bogus replies (DNS CACHE POISONING); DNSSEC [RFC 4033] provides authentication services against them.

2.5 Video streaming and content distribution networks

Streaming video is a major consumer of Internet bandwidth: Netflix, YouTube, and Amazon Prime accounted for 80% of residential ISP traffic in 2020. The challenges are scale (how to reach about a billion users) and heterogeneity (different users have different capabilities, e.g., wired versus mobile, bandwidth rich versus bandwidth poor). The solution is a distributed, application-level infrastructure.

A VIDEO is a sequence of images displayed at a constant rate, e.g., 24 images per second. A digital image is an array of pixels, each represented by bits. CODING uses redundancy within and between images to decrease the number of bits used to encode an image. SPATIAL coding works within an image: instead of sending N values of the same color (all purple), send only two values, the color value (purple) and the number of repeated values N. TEMPORAL coding works from one image to the next: instead of sending the complete frame $i+1$, send only the differences from frame i. In CONSTANT BIT RATE (CBR) video the encoding rate is fixed; in VARIABLE BIT RATE (VBR) video it changes as the amount of spatial and temporal coding changes. Examples are MPEG 1 (CD-ROM) at 1.5 Mbps, MPEG2 (DVD) at 3–6 Mbps, and MPEG4 (often used in the Internet) at 64 Kbps–12 Mbps.

Streaming stored video. In the simple scenario a video server holding stored video sends it over the Internet to a client. The main challenges are that server-to-client bandwidth varies over time with changing congestion levels (in the house, the access network, the network core, and the video server), and that packet loss and delay due to congestion delay playout or result in poor video quality. The video is recorded (e.g., at 30 frames per second), sent, and after a network delay received and played out at the client at 30 frames per second; in STREAMING, the client plays out the early part of the video while the server is still sending the later part.

Definition: The CONTINUOUS PLAYOUT CONSTRAINT requires that during client video playout the playout timing match the original timing. Network delays are variable (JITTER), so a client-side buffer is needed to meet the constraint: CLIENT-SIDE BUFFERING and a PLAYOUT DELAY compensate for network-added delay and delay jitter (Figure 18).

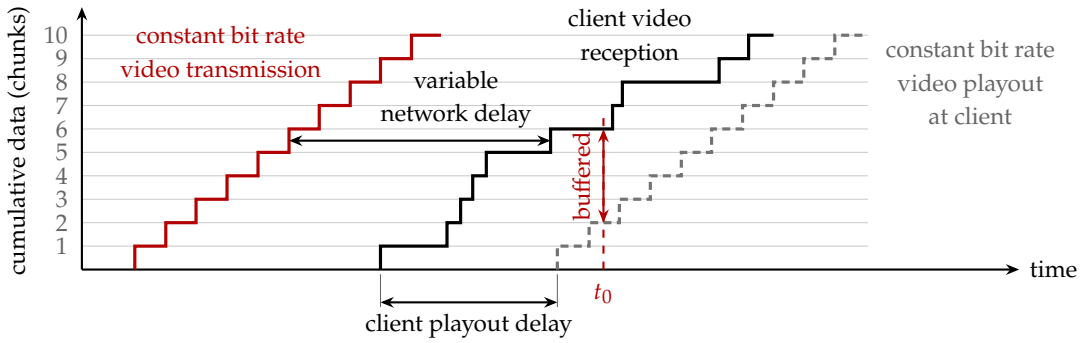


Figure 18: Playout buffering (schematic). At t_0, 6 chunks have arrived and 2 have been played out, so the buffer holds 4 chunks.

Other challenges are client interactivity (pause, fast-forward, rewind, jumping through the video) and video packets that may be lost and retransmitted.

Definition: In DASH (Dynamic, Adaptive Streaming over HTTP) the server divides the video file into multiple CHUNKS, encodes each chunk at multiple different rates, stores the different rate encodings in different files, and replicates the files in various CDN nodes; a MANIFEST FILE provides the URLs for the different chunks. The client periodically estimates the server-to-client bandwidth and, consulting the manifest, requests one chunk at a time, choosing the maximum coding rate sustainable given the current bandwidth; it can choose different coding rates at different points in time (depending on the bandwidth available then) and different servers.

The "intelligence" is at the client, which determines *when* to request a chunk (so that buffer starvation or overflow does not occur), *what encoding rate* to request (higher quality when more bandwidth is available), and *where* to request it (from a URL server that is "close" to the client or has high available bandwidth). Streaming video is encoding plus DASH plus playout buffering.

Content distribution networks. The challenge is to stream content selected from millions of videos to hundreds of thousands of simultaneous users. Option 1, a single large "mega-server", is a single point of failure and a point of network congestion, and has a long (and possibly congested) path to distant clients: it does not scale. Option 2 is to store and serve multiple copies of videos at multiple geographically distributed sites, a CONTENT DISTRIBUTION NETWORK (CDN), in one of two ways:

- ENTER DEEP: push CDN servers deep into many access networks, close to users (Akamai: 240,000 servers deployed in more than 120 countries in 2015);
- BRING HOME: a smaller number (tens) of larger clusters in points of presence (POPs) near access networks (used by Limelight).

Today Akamai's edge has 360K servers in 4,200+ locations, 1,350+ networks, 840+ cities, and 135 countries, serving 100+ million hits per second, 7+ trillion deliveries per day, and 175+ terabits per second (250+ peak).

Netflix stores copies of content (e.g., MADMEN) at its worldwide OpenConnect CDN nodes. A subscriber requests content and the service provider returns a manifest; using the manifest, the client retrieves the content at the highest supportable rate, and may choose a different rate or copy if the network path is congested. Such services are OVER THE TOP (OTT): they use Internet host-to-host communication as a service and must cope with a congested Internet from the "edge", deciding what content to place in which CDN node, and from which CDN node and at which rate to retrieve content.

Example: *CDN content access via DNS.* Bob (a client) requests the video `http://netcinema.com/6Y7B23V`, which is stored in a CDN at `http://KingCDN.com/NetC6y&B23V` (Figure 19). (1) Bob gets the URL of the video from the netcinema.com web page. (2) He resolves it via his local DNS server.

(3) netcinema's authoritative DNS returns a CNAME for the KingCDN URL. (4) The local DNS server resolves that name with KingCDN's authoritative DNS, which returns the address of a KingCDN content server, and (5) passes the address to Bob. (6) Bob requests the video from the KingCDN server, and it is streamed via HTTP.

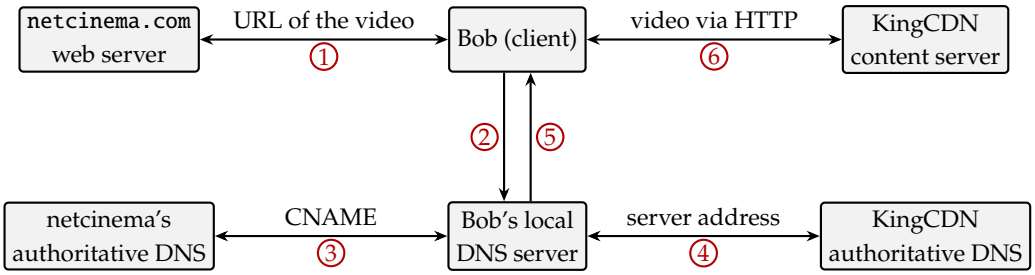


Figure 19: CDN content access: DNS redirects Bob's request for a netcinema.com video to a KingCDN server through a CNAME record.

Example: *Netflix.* Netflix uploads copies of multiple versions of each video from the Amazon cloud to CDN servers. (1) Bob manages his Netflix account through Netflix's registration and accounting servers. (2) Bob browses Netflix videos. (3) A manifest file is returned for the requested video. (4) A DASH server is selected and contacted, and streaming begins.

2.6 Socket programming with UDP and TCP

The goal is to build client/server applications that communicate using sockets, the door between the application process and the end-to-end transport protocol. There are two socket types for the two transport services: UDP (unreliable datagram) and TCP (reliable, byte stream-oriented). The running example: (1) the client reads a line of characters from its keyboard and sends it to the server; (2) the server converts the characters to uppercase; (3) the server sends the modified data to the client; (4) the client displays the line on its screen.

UDP. With UDP there is no "connection" between client and server: there is no handshaking before sending data, the sender explicitly attaches the destination IP address and port number to each packet, and the receiver extracts the sender's IP address and port number from the received packet. Transmitted data may be lost or received out of order. From the application's viewpoint, UDP provides unreliable transfer of groups of bytes (DATAGRAMS) between client and server processes. The server creates a socket on port x and reads datagrams from it; the client creates a socket, creates a datagram with the server's IP address and port x, and sends it; the server writes its reply to its socket, specifying the client's address and port number; and the client reads the reply and closes its socket. In Python:

```python
# UDPClient
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)  # create UDP socket
message = input('Input lowercase sentence:')
# attach server name and port to the message; send into the socket
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

```python
# UDPServer
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))  # bind to local port 12000
print('The server is ready to receive')
while True:
    # read a datagram, getting the client's address (IP and port)
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

TCP. With TCP the client must contact the server, so the server process must first be running and must have created a socket (door) that welcomes the client's contact. The client contacts the server by creating a TCP socket, specifying the IP address and port number of the server process; when the client creates the socket, client TCP establishes a connection to server TCP. When contacted by a client, server TCP creates a new socket for the server process to communicate with that particular client. This allows the server to talk with multiple clients, and the client source port number and IP address are used to distinguish the clients. From the application's viewpoint, TCP provides reliable, in-order byte-stream transfer (a "pipe") between client and server processes.

```python
# TCPClient
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)  # TCP socket, remote port 12000
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())  # no need to attach server name, port
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

```python
# TCPServer
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)  # welcoming socket
serverSocket.bind(('', serverPort))
serverSocket.listen(1)  # listen for incoming TCP requests
print('The server is ready to receive')
while True:
    # wait on accept(); a new socket is created on return
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()  # bytes, not address
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()  # close this client's socket, not the welcoming one
```

Timeouts. Sometimes a program must wait for one of several events: a reply from the other end of a socket or a timeout (a timer), or replies from several open sockets (`select()`, multithreading); timeouts are used extensively in networking. In Python, `s.settimeout(t)` sets a timeout on all future operations of socket `s`: if no packet arrives within t seconds of calling `s.recv()`, the call is interrupted and raises a timeout exception, which a `try/except` block catches (execution jumps from the `try` block directly into the `except` block).

Example: *Socket timeouts.* A shepherd boy can send the villagers a wolf's location over a TCP socket, but he likes to connect without sending anything; after three connections without a location within 10 seconds, the villagers' server stops listening to him forever:

```python
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
counter = 0
while counter < 3:
    connectionSocket, addr = serverSocket.accept()
    connectionSocket.settimeout(10)  # 10-second timeout on future operations
    try:
        wolf_location = connectionSocket.recv(1024).decode()
        send_hunter(wolf_location)  # a villager function
        connectionSocket.send('hunter sent'.encode())
    except timeout:  # no message within 10 seconds
        counter += 1
    connectionSocket.close()
```

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 2 slides (9th edition).

Lecture 3: The Transport Layer

3.1 Transport-layer services

Definition: A transport protocol provides logical communication between application processes running on different hosts. It runs in the end systems: the sender breaks application messages into segments and passes them to the network layer; the receiver reassembles segments into messages and passes them to the application layer.

The network layer provides logical communication between *hosts*; the transport layer provides logical communication between *processes*, and it relies on and enhances the network-layer service. In a household analogy, twelve kids in Ann's house send letters to twelve kids in Bill's house: hosts are houses, processes are kids, application messages are letters in envelopes, the transport protocol is Ann and Bill, who demultiplex the mail to their in-house siblings, and the network-layer protocol is the postal service.

A sending transport layer is passed an application-layer message, determines the segment's header field values, creates the segment, and passes it to IP. The receiving transport layer receives the segment from IP, checks the header values, extracts the application-layer message, and demultiplexes it up to the application via a socket.

Internet applications have two transport protocols. TCP (Transmission Control Protocol) offers reliable, in-order delivery, congestion control, flow control, and connection setup. UDP (User Datagram Protocol) offers unreliable, unordered delivery, a no-frills extension of "best-effort" IP. Neither provides delay guarantees or bandwidth guarantees.

3.2 Multiplexing and demultiplexing

Definition: Multiplexing at the sender handles data from multiple sockets and adds a transport header (later used for demultiplexing). Demultiplexing at the receiver uses header information to deliver each received segment to the correct socket.

For example, a client host running Skype, Netflix, and Firefox receives a segment carrying an HTTP response; demultiplexing is what delivers it to the Firefox process rather than the others. A host receives IP datagrams; each datagram has a source and a destination IP address and carries one transport-layer segment, and each segment has a source and a destination port number (the first two 16-bit fields of both the TCP and the UDP segment format, followed by other header fields and the application data). The host uses IP addresses and port numbers to direct the segment to the appropriate socket. Multiplexing and demultiplexing, based on segment and datagram header field values, happen at all layers.

Connectionless demultiplexing. A socket is created with a host-local port number (in Java, `new DatagramSocket(12534)`), and a datagram sent into a UDP socket must specify the destination IP address and destination port number. A receiving host checks the destination port number of a UDP segment and directs the segment to the socket with that port. UDP demultiplexes on the destination port number only: datagrams with the same destination port but different source IP addresses or source ports are directed to the *same* socket. For example, a server process binds a UDP socket to port 6428 (`mySocket.bind(myaddr,6428)`) and two clients bind ports 9157 and 5775. The first client's segments carry source port 9157 and destination port 6428, and the server's replies carry source port 6428 and destination port 9157; likewise the second client sends from 5775 to 6428 and receives from 6428 at 5775. Both clients' segments reach the same server socket.

Connection-oriented demultiplexing. A TCP socket is identified by a 4-TUPLE: source IP address, source port number, destination IP address, destination port number. The receiver uses all four values to direct a segment to the appropriate socket. A server may support many simultaneous TCP sockets, each identified by its own 4-tuple and associated with a different connecting client. For example, three segments all destined to IP address B, port 80, with sources (A, 9157), (C, 5775), and (C, 9157), are demultiplexed to three different sockets; the first and third share a source port but differ in source IP address.

3.3 Connectionless transport: UDP

UDP [RFC 768] is a "no frills," "bare bones" Internet transport protocol with best-effort service: segments may be lost or delivered out of order to the application. It is connectionless: there is no handshaking between UDP sender and receiver, and each segment is handled independently of the others. UDP exists because it needs no connection establishment (which can add an RTT of delay), keeps no connection state at sender or receiver, has a small header, and has no congestion control, so it can send as fast as desired and can function in the face of congestion or when the network service is compromised. It is used by streaming multimedia applications (loss tolerant, rate sensitive), DNS, SNMP, and HTTP/3. If an application over UDP needs reliable transfer (as HTTP/3 does), it adds the needed reliability and congestion control at the application layer.

A UDP sender (for example an SNMP client) is passed an application message, determines the UDP header field values, creates the UDP segment, and passes it to IP. The UDP receiver receives the segment from IP, checks the UDP checksum header value, extracts the message, and demultiplexes it up to the application via a socket. The UDP header has four 16-bit fields (Figure 20); the length field gives the length in bytes of the UDP segment, including the header.

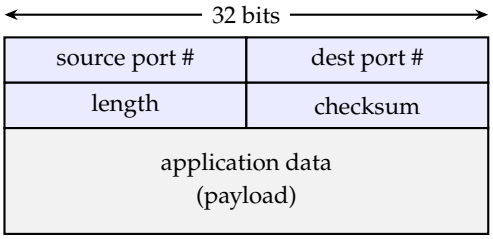


Figure 20: UDP segment format: four 16-bit header fields followed by the application data (payload).

The checksum detects errors (flipped bits) in a transmitted segment. As a toy illustration, a sender transmits the numbers 5 and 6 with their sum 11; if the receiver gets 4, 6, 11, its own sum 10 differs from the received sum 11, and the error is detected.

Definition: The INTERNET CHECKSUM. The sender treats the contents of the UDP segment (including the UDP header fields and the IP addresses) as a sequence of 16-bit integers, computes their one's complement sum, and puts the one's complement of that sum into the checksum field. The receiver computes the checksum of the received segment and compares it with the checksum field: if they differ, an error is detected; if they are equal, no error is detected (though errors may still be present).

Example: Adding two 16-bit integers, a carry out of the most significant bit is wrapped around and added to the result:

	1110011001100110
	1101010101010101
wraparound	11011101110111011
sum	1011101110111100
checksum	0100010001000011

The 17-bit sum 11011101110111011 has a carry out; adding it back to the low 16 bits gives the sum 1011101110111100, and the checksum is its bitwise complement. The protection is weak: if the last two bits of the first number flip from 10 to 01 and those of the second flip from 01 to 10, the numbers change but the sum, and hence the checksum, does not.

3.4 Principles of reliable data transfer

A reliable channel is a service abstraction: the sending process passes data into the channel and the receiving process gets it intact and in order. A RELIABLE DATA TRANSFER (rdt) protocol implements this abstraction with a sender side and a receiver side that communicate over an unreliable channel. Its complexity depends strongly on the characteristics of that channel (does it lose, corrupt, or reorder data?), and the sender and receiver do not know each other's state (was a message received?) unless it is communicated in a message. Figure 21 shows the four interfaces.

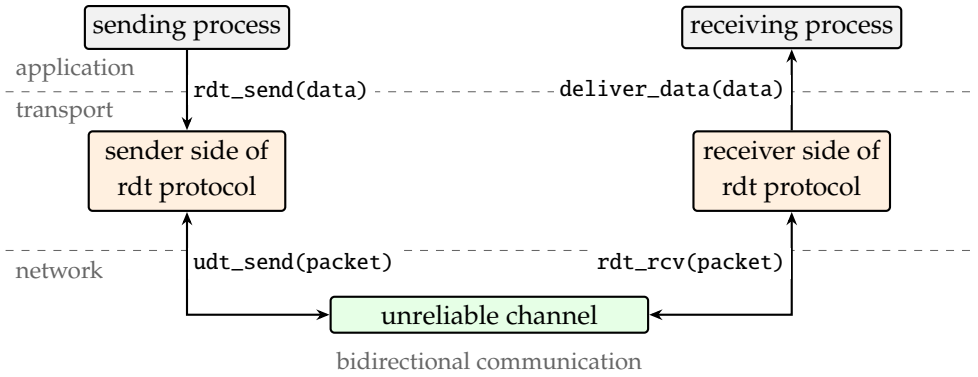


Figure 21: Interfaces of an rdt protocol. `rdt_send()` is called from above (for example by the application) with data to deliver to the receiver's upper layer; `udt_send()` is called by rdt to transfer a packet over the unreliable channel; `rdt_rcv()` is called when a packet arrives on the receiver side; `deliver_data()` is called by rdt to deliver data to the upper layer. Communication over the channel is bidirectional.

We develop the sender and receiver sides of an rdt protocol incrementally, for unidirectional data transfer (control information still flows in both directions), and specify each side by a FINITE STATE MACHINE (FSM). In each state, the next state is uniquely determined by the next event. A transition is labeled with the event that causes it above a rule and the actions taken below it; Λ denotes no event or no action, and a dashed arrow marks the initial state.

rdt1.0: a reliable channel. If the underlying channel is perfectly reliable (no bit errors, no loss), separate one-state FSMs suffice: the sender sends data into the channel and the receiver reads data from it (Figure 22).

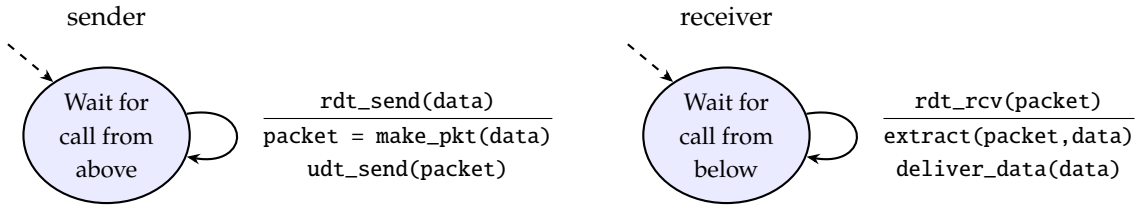


Figure 22: rdt1.0 sender and receiver.

rdt2.0: a channel with bit errors. If the channel may flip bits in a packet, a checksum (such as the Internet checksum) detects bit errors. To recover, the receiver sends ACKNOWLEDGEMENTS (ACKs), telling the sender explicitly that a packet was received OK, and NEGATIVE ACKNOWLEDGEMENTS (NAKs), telling it that the packet had errors; the sender retransmits a packet on receipt of a NAK. The protocol is STOP-AND-WAIT: the sender sends one packet and then waits for the receiver's response (Figure 23). With no errors, the receiver delivers the data and returns an ACK; with a corrupted packet, it returns a NAK and the sender retransmits. The receiver's state (did it get the message correctly?) is unknown to the sender unless it is communicated from receiver to sender, which is why a protocol is needed.

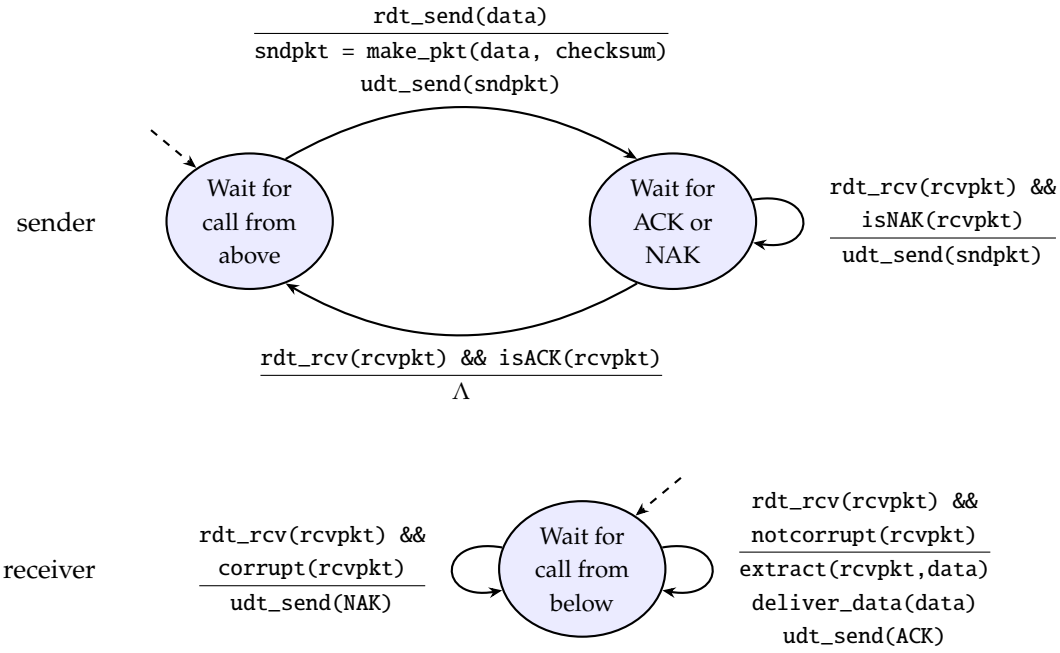


Figure 23: rdt2.0 sender (top) and receiver (bottom).

rdt2.0 has a fatal flaw: if an ACK or NAK is corrupted, the sender does not know what happened at the receiver, and it cannot simply retransmit, since the retransmission may be a duplicate. To handle duplicates, the sender retransmits the current packet if the ACK/NAK is corrupted, the sender adds a SEQUENCE NUMBER to each packet, and the receiver discards (does not deliver up) a duplicate packet.

rdt2.1: handling garbled ACK/NAKs. The sender adds sequence number 0 or 1 to each packet and checks whether a received ACK/NAK is corrupted (Figure 24). Two sequence numbers suffice because only one packet is outstanding at a time, so the receiver need only tell a new packet from a retransmission of the previous one. The sender has twice as many states, since a state must remember whether the expected packet has sequence number 0 or 1. The receiver must check whether a received packet is a duplicate; its state indicates whether 0 or 1 is the expected sequence number (Figure 25). The receiver cannot know whether its last ACK/NAK was received OK at the sender.

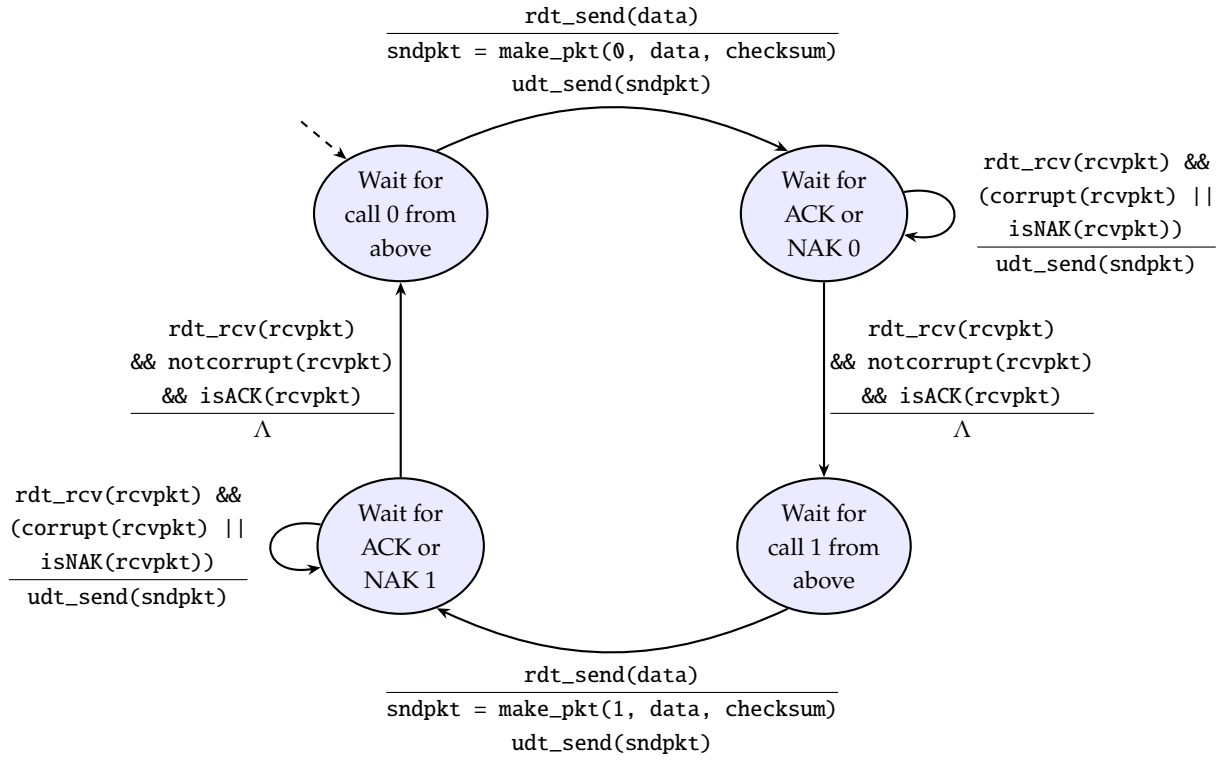


Figure 24: rdt2.1 sender.

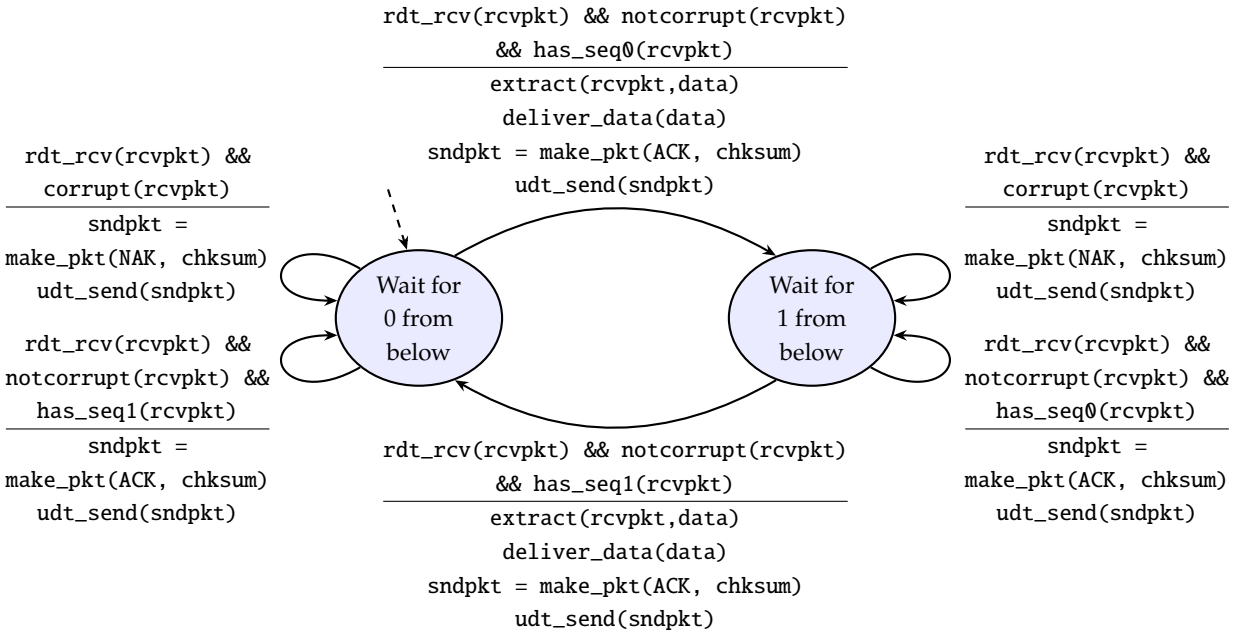


Figure 25: rdt2.1 receiver. A duplicate (the packet with the other sequence number) is not delivered but is ACKed again.

rdt2.2: a NAK-free protocol. rdt2.2 has the same functionality as rdt2.1 using ACKs only. Instead of a NAK, the receiver sends an ACK for the last packet received OK, and it must explicitly include the sequence number of the packet being ACKed. A duplicate ACK at the sender results in the same action as a NAK: retransmit the current packet. TCP uses this approach to be NAK-free. Table 1 lists the transitions that change from rdt2.1.

State	Event	Actions	Next state
Sender			
Wait for call 0	`rdt_send(data)`	`sndpkt = make_pkt(0, data, checksum); udt_send(sndpkt)`	Wait for ACK 0
Wait for ACK 0	`rdt_rcv(rcvpkt) && (corrupt(rcvpkt) || isACK(rcvpkt,1))`	`udt_send(sndpkt)`	Wait for ACK 0
Wait for ACK 0	`rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && isACK(rcvpkt,0)`	Λ	Wait for call 1
Receiver			
Wait for 0	`rdt_rcv(rcvpkt) && (corrupt(rcvpkt) || has_seq1(rcvpkt))`	`udt_send(sndpkt)` (resend the last ACK)	Wait for 0
Wait for 1	`rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && has_seq1(rcvpkt)`	`extract(rcvpkt,data); deliver_data(data); sndpkt = make_pkt(ACK1, chksum); udt_send(sndpkt)`	Wait for 0

Table 1: rdt2.2 sender and receiver fragments ("Wait for call *i*" is "Wait for call *i* from above"; "Wait for *i*" is "Wait for *i* from below"). The ACK carries the sequence number it acknowledges.

rdt3.0: a channel with errors and loss. Now the channel can also lose packets (data or ACKs). Checksums, sequence numbers, ACKs, and retransmissions help but are not quite enough. The sender waits a "reasonable" amount of time for an ACK and retransmits if none arrives in that time; a countdown timer interrupts after that time. If a packet or ACK was only delayed (not lost), the retransmission is a duplicate, which sequence numbers already handle; the receiver must specify the sequence number of the packet being ACKed. Figure 26 gives the sender, and Figures 27 and 28 four executions. The receiver detects a duplicate `pkt1` by its sequence number and re-ACKs it, and the sender ignores a duplicate `ack1`.

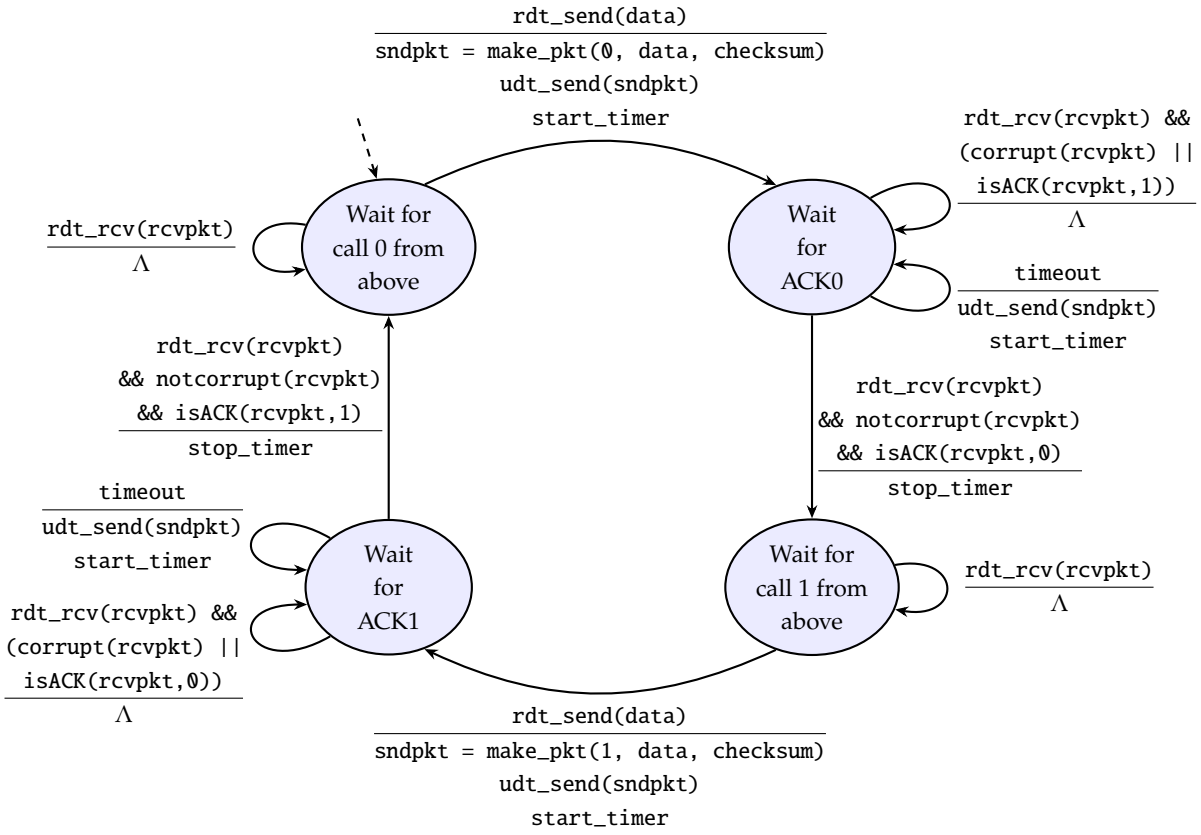


Figure 26: rdt3.0 sender. Unexpected or corrupted ACKs are ignored (Λ); the timer, not a duplicate ACK, triggers retransmission.

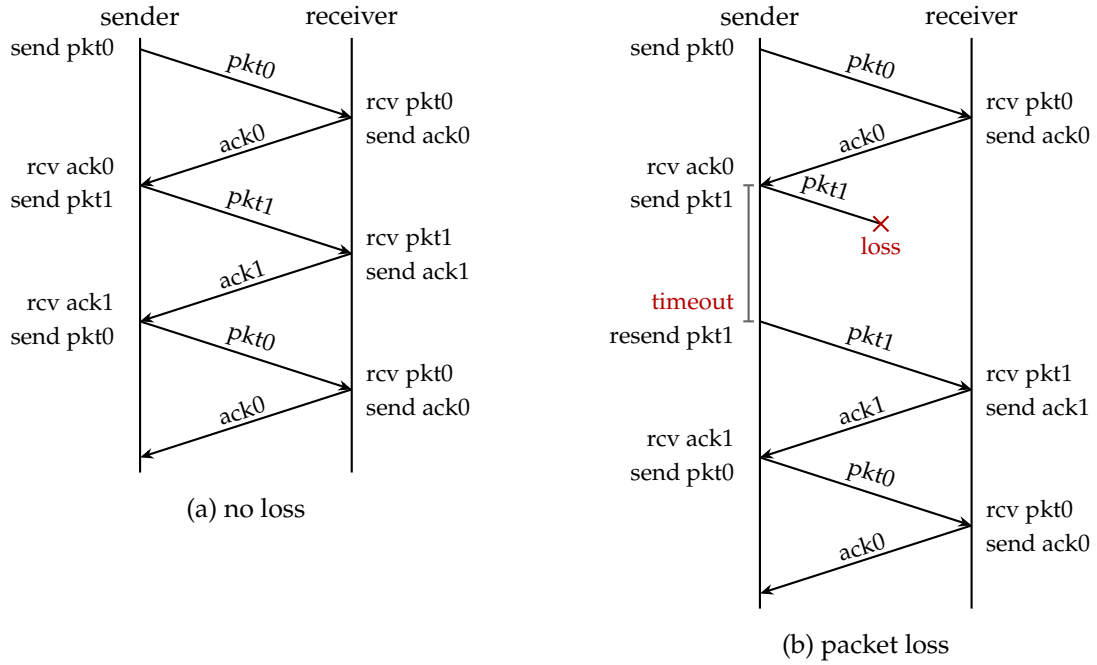


Figure 27: rdt3.0 in action: (a) no loss; (b) packet loss, recovered by the timeout.

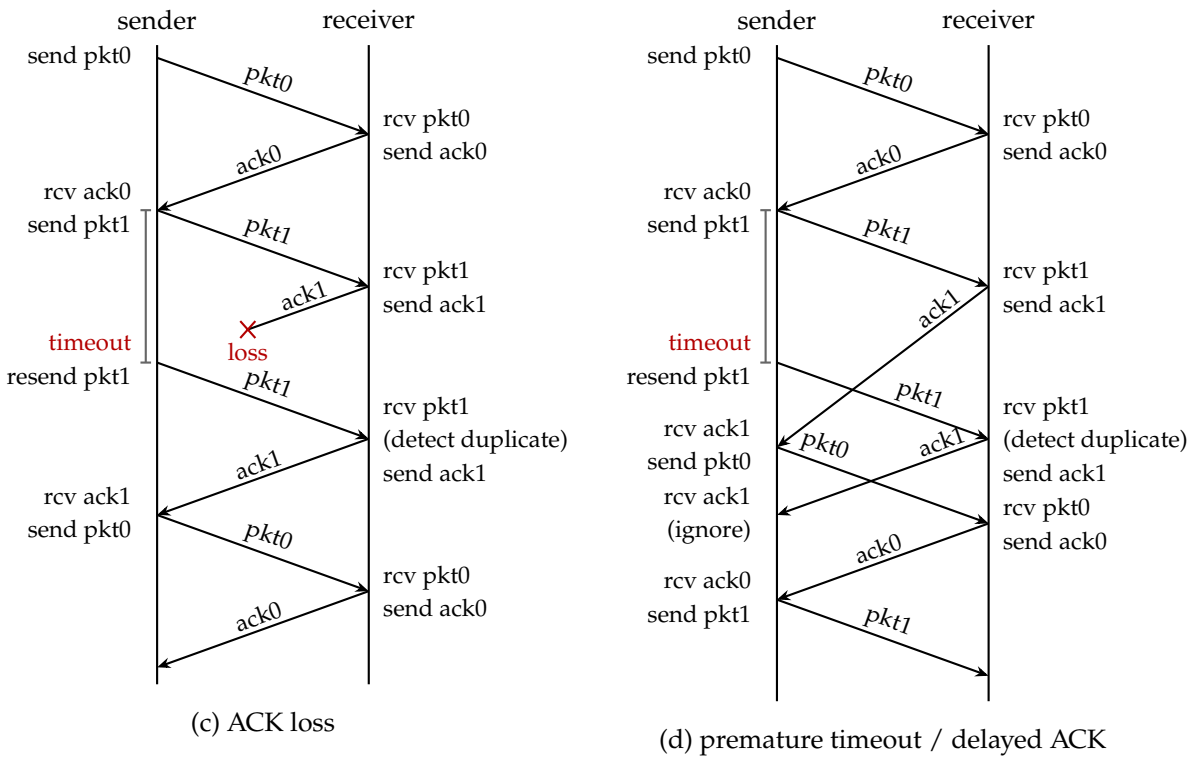


Figure 28: rdt3.0 in action: (c) ACK loss; (d) premature timeout or delayed ACK.

Performance of stop-and-wait. The sender UTILIZATION U_{sender} is the fraction of time the sender is busy sending. Take a 1 Gbps link, a 15 ms propagation delay (so RTT = 30 ms), and 8000-bit packets. The time to transmit a packet into the channel is

$$D_{\text{trans}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \ \mu\text{s}.$$

The first bit is transmitted at $t = 0$; the last bit arrives at the receiver, which sends an ACK; the ACK arrives and the next packet is sent at $t = \text{RTT} + L/R$ (Figure 29a). Hence

$$U_{\text{sender}} = \frac{L/R}{\text{RTT} + L/R} = \frac{0.008}{30.008} \approx 0.00027.$$

The protocol, not the channel, limits performance.

Definition: In a PIPELINED protocol the sender allows multiple "in-flight," yet-to-be-acknowledged packets. The range of sequence numbers must be increased, and packets must be buffered at the sender and/or receiver.

Sending three packets per round trip (Figure 29b) triples the utilization:

$$U_{\text{sender}} = \frac{3L/R}{\text{RTT} + L/R} = \frac{0.024}{30.008} \approx 0.0008.$$

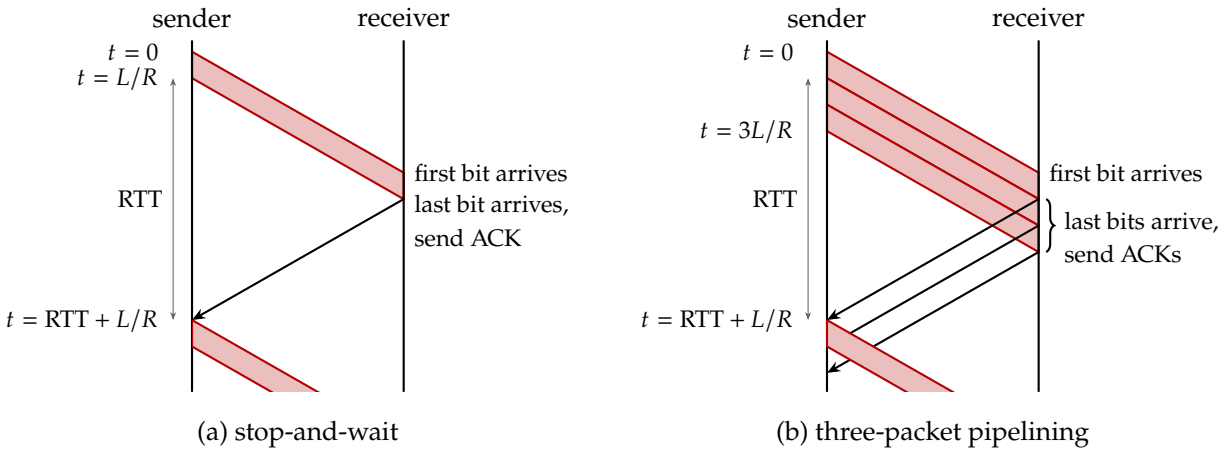


Figure 29: (a) Stop-and-wait: one packet per RTT + L/R. (b) Three-packet pipelining: the receiver ACKs as the last bit of each packet arrives, and the sender is busy for $3L/R$ of each RTT + L/R.

Go-Back-N. The sender keeps a WINDOW of up to N consecutive transmitted but unACKed packets, with a k-bit sequence number in the packet header (Figure 30a). ACKs are CUMULATIVE: ACK(n) acknowledges all packets up to and including sequence number n, and on receiving it the sender moves the window forward to begin at $n + 1$. A single timer runs for the oldest in-flight packet; on timeout(n) the sender retransmits packet n and all higher-numbered packets in the window. The receiver is ACK-only: it always sends an ACK for the correctly received packet with the highest *in-order* sequence number, which may generate duplicate ACKs, and it need only remember `rcv_base`, the next expected sequence number (Figure 30b). An out-of-order packet can be discarded or buffered (an implementation decision), and the receiver re-ACKs the packet with the highest in-order sequence number.

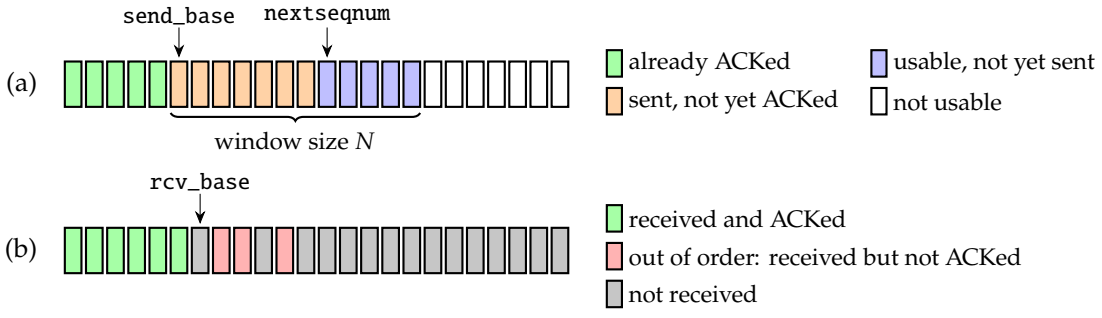


Figure 30: Go-Back-N views of the sequence-number space: (a) sender; (b) receiver.

In Figure 31, with $N = 4$, packet 2 is lost. The receiver discards packets 3, 4, and 5 and re-sends ACK 1 for each; the sender ignores the duplicate ACKs. When packet 2 times out, the sender resends packets 2–5, which the receiver now delivers and ACKs.

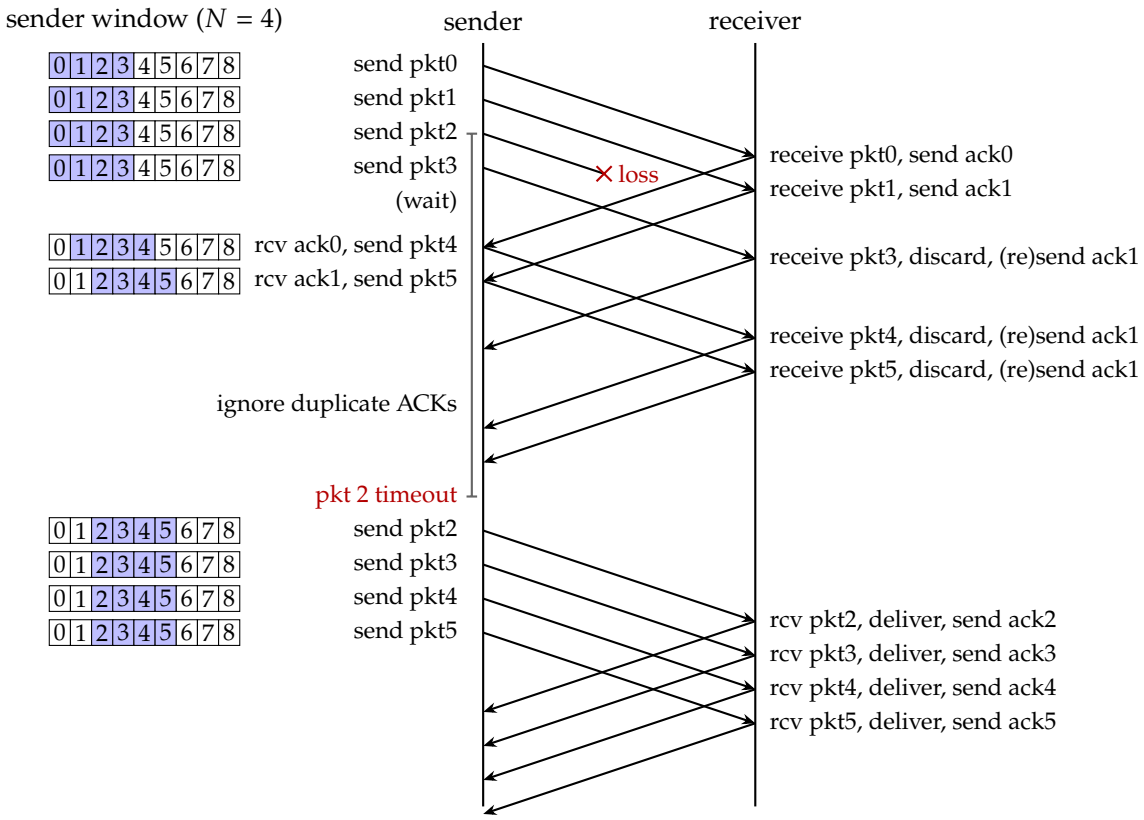


Figure 31: Go-Back-N in action with $N = 4$; the highlighted cells are the sender's window.

Algorithms 3.1 and 3.2 give the extended FSMs (each has a single state, Wait); this receiver discards out-of-order packets, so it needs no receiver buffering.

Algorithm 3.1: Go-Back-N sender

Initially: $base \leftarrow 1$; $nextseqnum \leftarrow 1$
1 **on** `rdt_send(data)`
2 **if** $nextseqnum < base + N$ **then**
3 $sndpkt[nextseqnum] \leftarrow$ `make_pkt`($nextseqnum$, data, chksum)
4 `udt_send`($sndpkt[nextseqnum]$)
5 **if** $base = nextseqnum$ **then** `start_timer`
6 $nextseqnum \leftarrow nextseqnum + 1$
7 **else**
8 `refuse_data(data)`
9 **on** `timeout`
10 `start_timer`
11 `udt_send`($sndpkt[base]$), `udt_send`($sndpkt[base+1]$), ..., `udt_send`($sndpkt[nextseqnum-1]$)
12 **on** `rdt_rcv(rcvpkt)` **and** `notcorrupt(rcvpkt)`
13 $base \leftarrow$ `getacknum(rcvpkt)` $+ 1$
14 **if** $base = nextseqnum$ **then** `stop_timer` **else** `start_timer`
15 **on** `rdt_rcv(rcvpkt)` **and** `corrupt(rcvpkt)`
16 do nothing (Λ)

Algorithm 3.2: Go-Back-N receiver

```
Initially: expectedseqnum ← 1; sndpkt ← make_pkt(expectedseqnum, ACK, chksum)
on rdt_rcv(rcvpkt) and notcorrupt(rcvpkt) and hasseqnum(rcvpkt, expectedseqnum)
    extract(rcvpkt,data); deliver_data(data)
    sndpkt ← make_pkt(expectedseqnum, ACK, chksum)
    udt_send(sndpkt)
    expectedseqnum ← expectedseqnum + 1
on any other event
    udt_send(sndpkt)                          // re-ACK the highest in-order packet
```

Selective repeat. The receiver individually ACKs every correctly received packet and buffers packets as needed for in-order delivery to the upper layer. The sender maintains (conceptually) a timer for each unACKed packet, and a timeout retransmits only the unACKed packet associated with it; it also maintains (conceptually) a window over N consecutive sequence numbers, which limits the pipelined, in-flight packets (Figure 32).

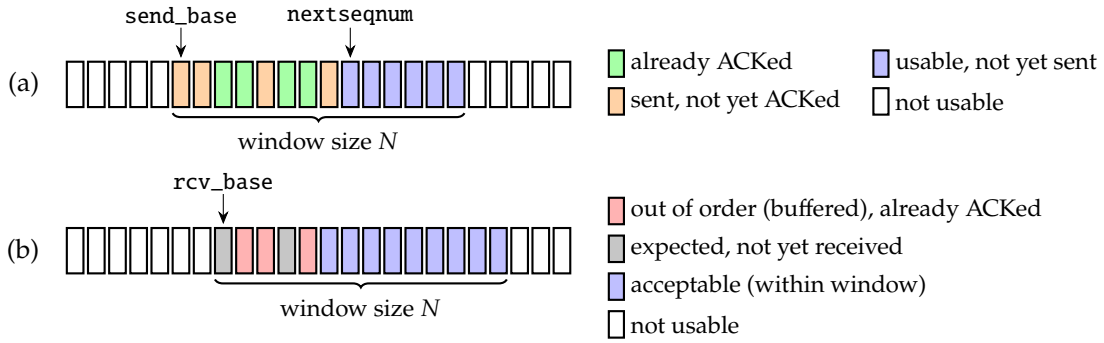


Figure 32: Selective-repeat views of the sequence-number space: (a) sender, whose window may hold packets ACKed out of order; (b) receiver, whose window of size N begins at `rcv_base`, the smallest expected packet not yet received.

The sender reacts to three events:

- *data from above:* if the next available sequence number is in the window, send the packet;
- *timeout*(n)*:* resend packet n and restart its timer;
- *ACK*(n) *with* n *in* [*sendbase*, *sendbase* + N − 1]*:* mark packet n as received; if n is the smallest unACKed packet, advance the window base to the next unACKed sequence number.

The receiver acts on the sequence number n of an arriving packet:

- n *in* [*rcvbase*, *rcvbase* + N − 1]*:* send ACK(n); if the packet is out of order, buffer it; if it is in order, deliver it (and any buffered in-order packets) and advance the window to the next not-yet-received packet;
- n *in* [*rcvbase* − N, *rcvbase* − 1]*:* send ACK(n) again;

- *otherwise:* ignore the packet.

In Figure 33, packet 2 is again lost, but the receiver buffers and ACKs packets 3, 4, and 5, and the sender records those ACKs. On the timeout it resends only packet 2; the receiver then delivers packets 2–5 and sends ACK 2. When ACK 2 arrives, packets 2–5 are all ACKed, so the sender's window advances to begin at 6.

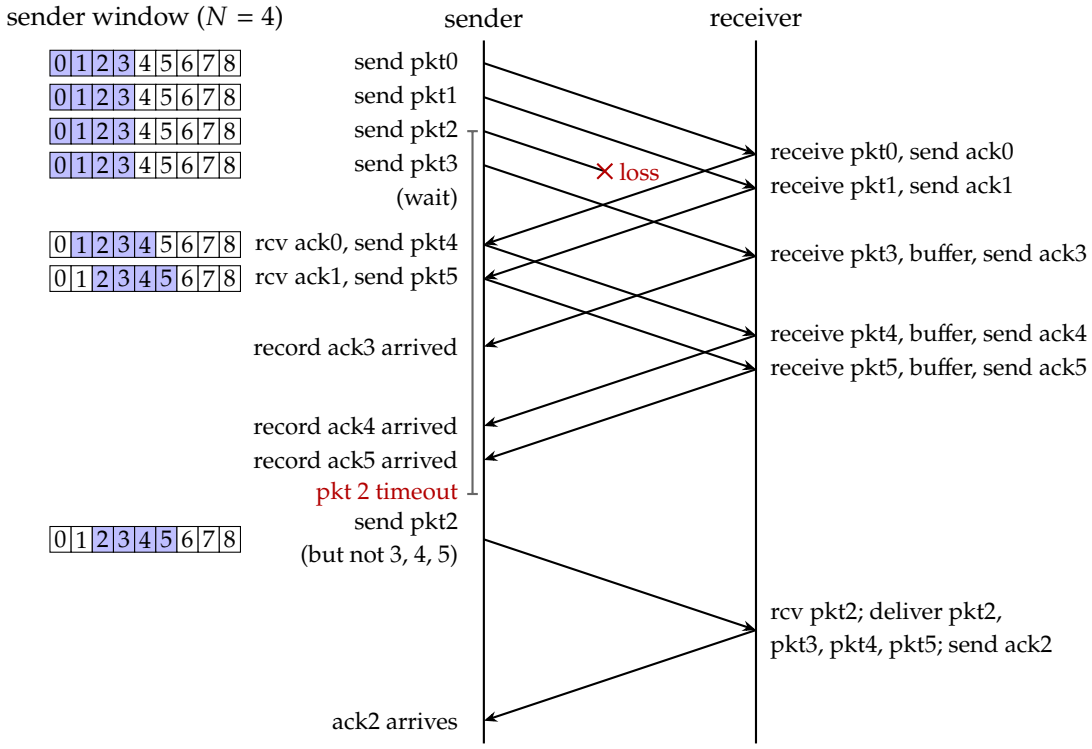


Figure 33: Selective repeat in action with $N = 4$.

Selective repeat has a dilemma when the sequence-number space is small (Figure 34). With sequence numbers $0, 1, 2, 3$ and window size 3, in (a) all ACKs arrive, the sender sends a new packet 0, and the receiver correctly accepts it. In (b) all three ACKs are lost and the sender retransmits the old packet 0, which the receiver accepts as new data. The receiver cannot see the sender side: its behavior is identical in both cases. The problem is avoided when the receiver's window can never contain both an old and a new packet with the same sequence number, which requires the window size to be at most half the size of the sequence-number space.

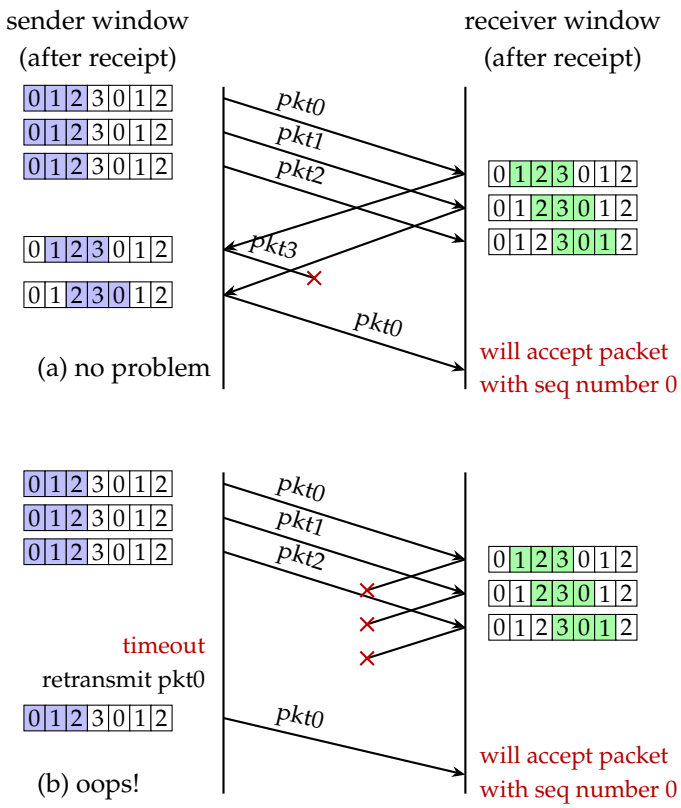


Figure 34: The selective-repeat dilemma with sequence numbers 0–3 and window size 3; windows are shown after each receipt. (a) No problem. (b) Oops: the retransmitted old packet 0 is accepted as new.

3.5 Connection-oriented transport: TCP

TCP (RFCs 793, 1122, 2018, 5681, 7323) is point-to-point (one sender, one receiver) and provides a reliable, in-order *byte stream* with no "message boundaries." It carries full-duplex data, a bidirectional data flow in the same connection, in segments of at most the MAXIMUM SEGMENT SIZE (MSS). It uses cumulative ACKs and pipelining, with TCP congestion and flow control setting the window size. It is connection-oriented: handshaking (an exchange of control messages) initializes sender and receiver state before data exchange. It is flow controlled: the sender will not overwhelm the receiver.

Segment structure. Figure 35 shows the TCP segment. The SEQUENCE NUMBER counts bytes of data in the byte stream, not segments: it is the byte-stream number of the first byte in the segment's data. The ACKNOWLEDGEMENT NUMBER is the sequence number of the next byte expected from the other side, and the ACK is cumulative; the A bit says that the segment carries an ACK. The header-length field gives the length of the TCP header, the C and E bits carry congestion notification, the RST, SYN, and FIN bits are used for connection management, the receive window (`rwnd`) carries flow control (the number of bytes the receiver is willing to accept), and the checksum is the Internet

checksum. Options and the application data (the data sent by the application into the TCP socket) have variable length. How a receiver handles out-of-order segments is not specified by TCP; it is up to the implementor.

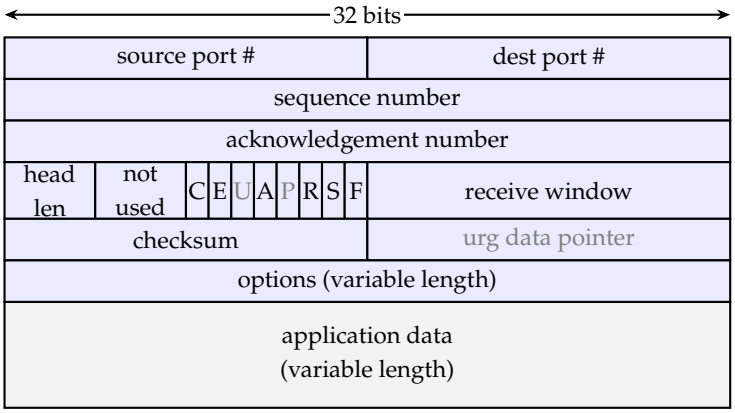


Figure 35: TCP segment structure (32 bits per row). U (urgent) and P (push) and the urgent data pointer are shown in gray.

In the simple Telnet scenario of Figure 36, a user on host A types 'C', which host B ACKs and echoes back; A then ACKs the echoed 'C'. Each side's ACK number is the next byte it expects, and an ACK is piggybacked on data.

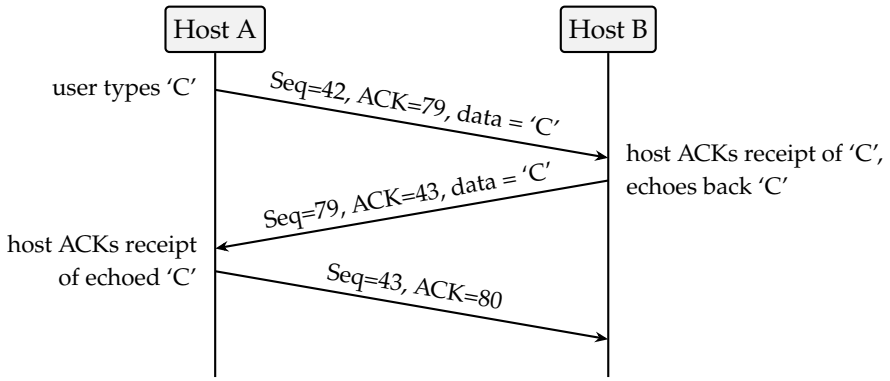


Figure 36: TCP sequence and acknowledgement numbers in a simple Telnet scenario.

Round-trip time and timeout. The timeout must be longer than the RTT, but the RTT varies: too short a timeout causes premature timeouts and unnecessary retransmissions, and too long a timeout reacts slowly to segment loss. TCP measures `SampleRTT`, the time from a segment's transmission until its ACK is received, ignoring retransmissions. `SampleRTT` varies, so TCP averages several recent measurements into a smoother estimate, an EXPONENTIAL WEIGHTED MOVING AVERAGE (EWMA) in which the influence of a past sample decreases exponentially fast:

$$\texttt{EstimatedRTT} = (1-\alpha)\cdot \texttt{EstimatedRTT} + \alpha \cdot \texttt{SampleRTT}, \qquad \text{typically } \alpha = 0.125.$$

On measured RTTs from `gaia.cs.umass.edu` to `fantasia.eurecom.fr`, the samples fluctuate widely while `EstimatedRTT` follows their trend smoothly. The timeout interval is `EstimatedRTT` plus a "safety margin" that grows with the variation:

$$\texttt{TimeoutInterval} = \texttt{EstimatedRTT} + 4 \cdot \texttt{DevRTT},$$
$$\texttt{DevRTT} = (1-\beta)\cdot\texttt{DevRTT} + \beta\cdot|\texttt{SampleRTT} - \texttt{EstimatedRTT}|, \qquad \text{typically } \beta = 0.25,$$

where `DevRTT` is an EWMA of the deviation of `SampleRTT` from `EstimatedRTT`.

Reliable data transfer. The simplified TCP sender reacts to three events (Algorithm 3.3). When data arrives from the application, it creates a segment whose sequence number is the byte-stream number of the first data byte and starts the timer if it is not already running; think of the timer as belonging to the oldest unACKed segment, with expiration interval `TimeoutInterval`. On a timeout, it retransmits the segment that caused the timeout and restarts the timer. When an ACK acknowledges previously unACKed segments, it updates what is known to be ACKed and starts the timer if unACKed segments remain.

Algorithm 3.3: TCP sender (simplified)

```
Initially: NextSeqNum ← InitialSeqNum; SendBase ← InitialSeqNum
on data received from application above
    create segment with sequence number NextSeqNum
    pass segment to IP (i.e., "send")
    NextSeqNum ← NextSeqNum + length(data)
    if timer currently not running then start timer
on timeout
    retransmit not-yet-ACKed segment with smallest sequence number
    start timer
on ACK received with ACK field value y
    if y > SendBase then
        SendBase ← y                          // SendBase − 1: last cumulatively ACKed byte
        if there are currently not-yet-ACKed segments then start timer else stop timer
```

The receiver generates ACKs as follows [RFC 5681]:

- *In-order segment with the expected sequence number, all data up to it already ACKed:* delayed ACK; wait up to 500 ms for the next segment, and if none arrives, send an ACK.
- *In-order segment with the expected sequence number while one other segment has an ACK pending:* immediately send a single cumulative ACK, ACKing both in-order segments.
- *Out-of-order segment with a higher-than-expected sequence number (a gap is detected):* immediately send a duplicate ACK, indicating the sequence number of the next expected byte (ACKing the last correctly received in-order byte).
- *Segment that partially or completely fills the gap:* immediately send an ACK, provided that the segment starts at the lower end of the gap.

Figure 37 shows three retransmission scenarios, and Figure 38 shows fast retransmit.

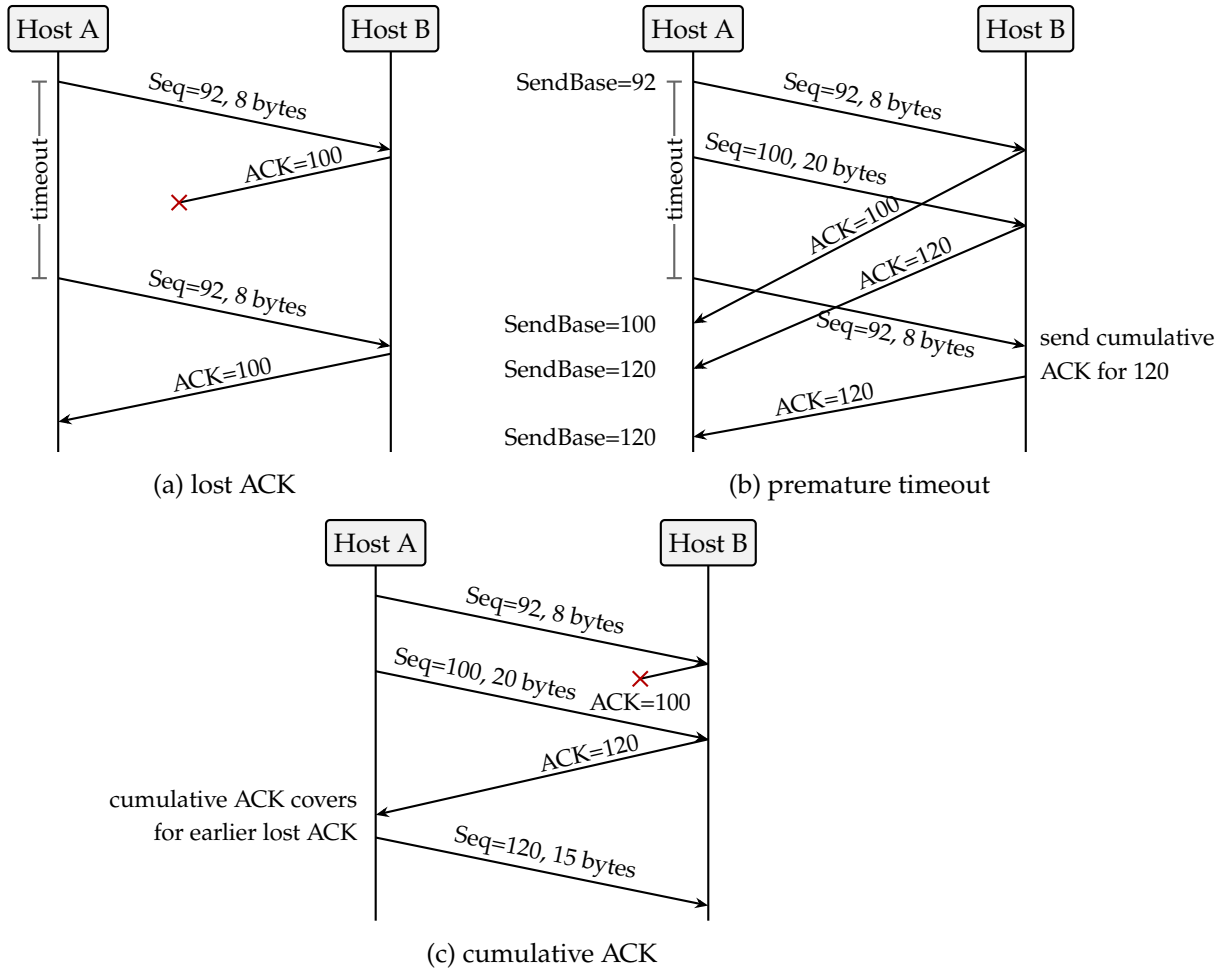


Figure 37: TCP retransmission scenarios. (a) Lost ACK: the timeout resends the segment, and B re-ACKs it. (b) Premature timeout: B sends a cumulative ACK for 120 in response to the duplicate. (c) A cumulative ACK covers for an earlier lost ACK, so nothing is retransmitted.

Definition: TCP FAST RETRANSMIT: if the sender receives three additional ACKs for the same data ("triple duplicate ACKs"), it resends the unACKed segment with the smallest sequence number without waiting for the timeout. Three duplicate ACKs indicate that three segments were received after a missing segment, so the segment was likely lost.

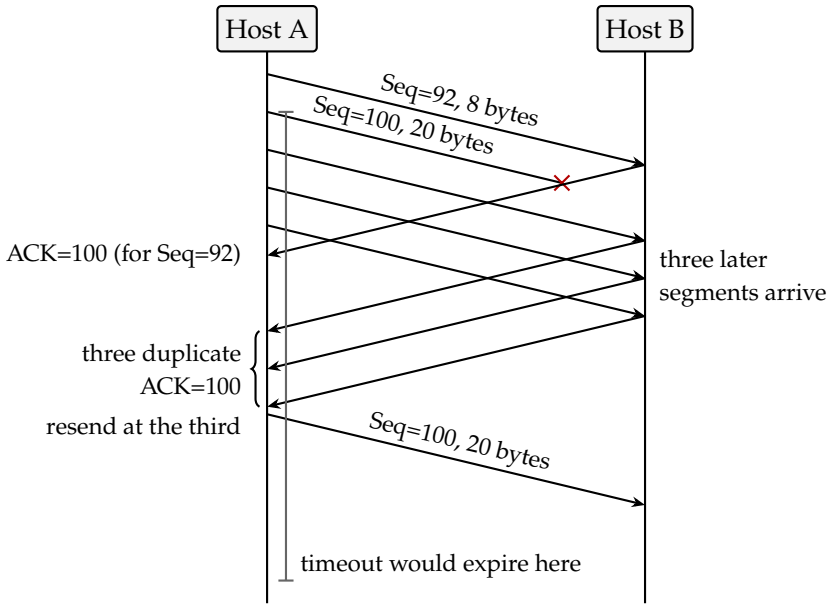


Figure 38: Fast retransmit: the segment with sequence number 100 is lost, the three later segments each produce a duplicate ACK 100, and the third duplicate triggers the retransmission well before the timeout expires.

Flow control. The network layer may deliver data (IP datagram payloads) into the TCP socket receiver buffers faster than the application process removes data from them.

Definition: FLOW CONTROL: the receiver controls the sender, so that the sender will not overflow the receiver's buffer by transmitting too much, too fast.

The TCP receiver "advertises" its free buffer space in the `rwnd` field of the TCP header (Figure 39). The `RcvBuffer` size is set via socket options (a typical default is 4096 bytes), and many operating systems auto-adjust it. The sender limits the amount of unACKed ("in-flight") data to the received `rwnd`, which guarantees that the receive buffer will not overflow.

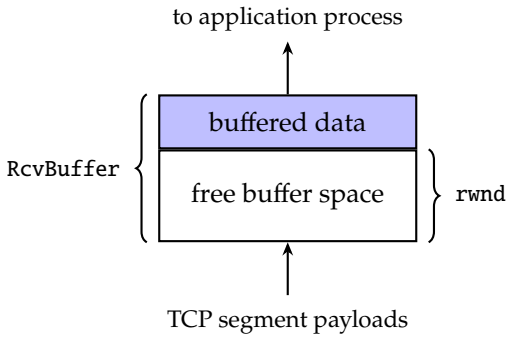
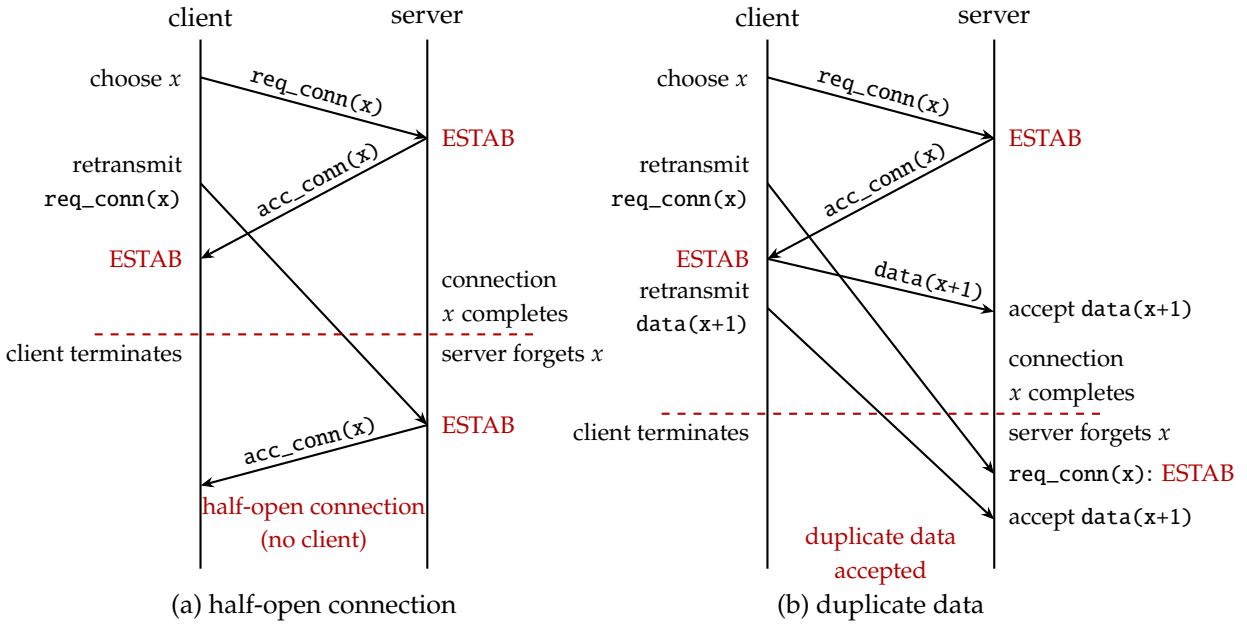


Figure 39: TCP receiver-side buffering: `rwnd` is the free part of `RcvBuffer`.

Connection management. Before exchanging data, sender and receiver "handshake": they agree to establish the connection (each knowing the other is willing) and agree on connection parameters, such as the starting sequence numbers. Each side then holds the connection state (ESTAB) and connection variables: the client-to-server and server-to-client sequence numbers and the `rcvBuffer` size at server and client. In Java, the client and the server create these sockets with

```
Socket clientSocket = new Socket("hostname", "port number");  // client
Socket connectionSocket = welcomeSocket.accept();             // server
```

In a 2-way handshake, the client sends `req_conn(x)`; the server enters ESTAB and answers `acc_conn(x)`; and the client enters ESTAB. This works when nothing goes wrong. In a network, however, there are variable delays, retransmitted messages (such as `req_conn(x)` after a loss), and message reordering, and neither side can "see" the other. Figure 40 shows two failures: a half-open connection with no client, and duplicate data accepted.



(a) half-open connection (b) duplicate data

Figure 40: Failures of a 2-way handshake. In both, the client retransmits `req_conn(x)`, connection x completes, the client terminates, and the server forgets x. (a) The late `req_conn(x)` opens a half-open connection with no client. (b) The late `req_conn(x)` and `data(x+1)` cause the server to accept duplicate data.

TCP uses a 3-way handshake (Figure 41). In Python, the server and the client set up the connection with

```
# server: waits in LISTEN
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
# client: starts the handshake
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
```

The client chooses an initial sequence number x and sends a SYN; the server chooses its initial sequence number y and sends a SYNACK acknowledging the SYN. The SYNACK shows the client that the server is live, and the client's ACK of it (which may carry client-to-server data) shows the server that the client is live. Figure 42 gives the handshake as an FSM.

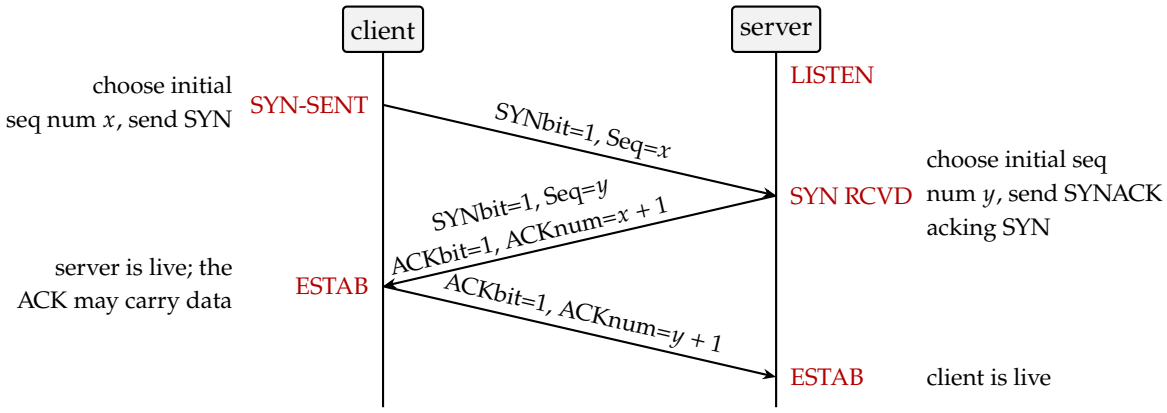


Figure 41: TCP 3-way handshake with client and server states.

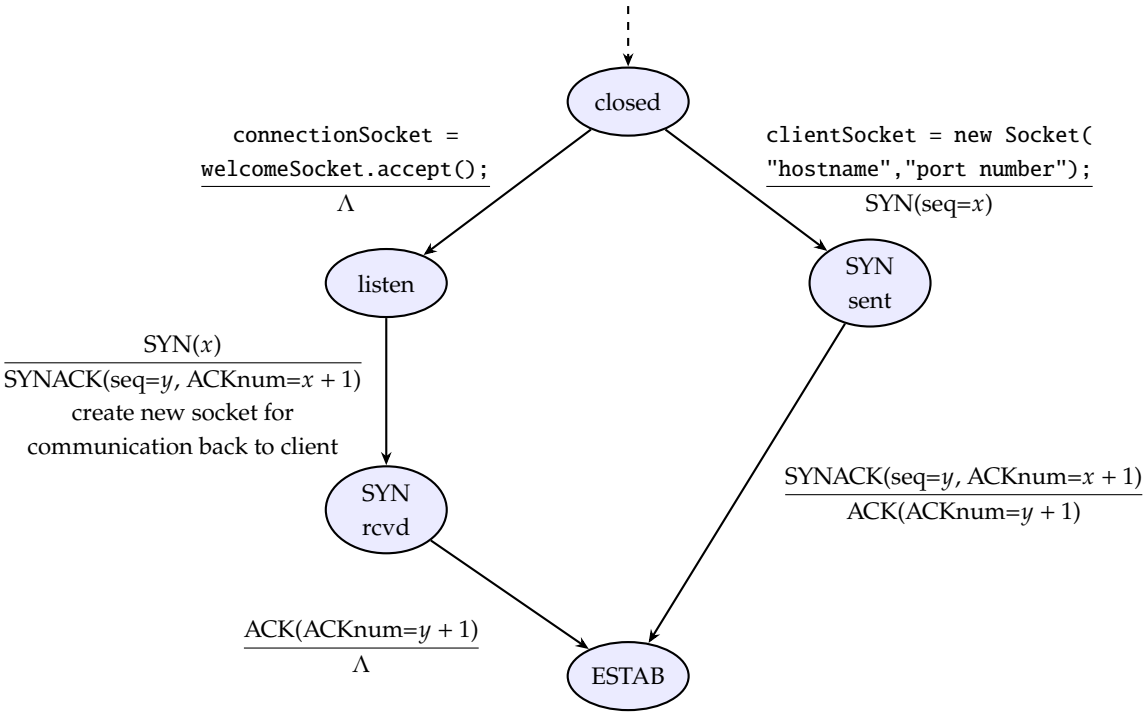


Figure 42: TCP 3-way handshake FSM. The server path is closed → listen → SYN rcvd → ESTAB; the client path is closed → SYN sent → ESTAB.

To close a connection, the client and the server each close their side by sending a TCP segment with FIN bit = 1 and respond to a received FIN with an ACK; on receiving a FIN, the ACK can be combined with the receiver's own FIN. Simultaneous FIN exchanges can be handled. Figure 43 shows a close initiated by the client.

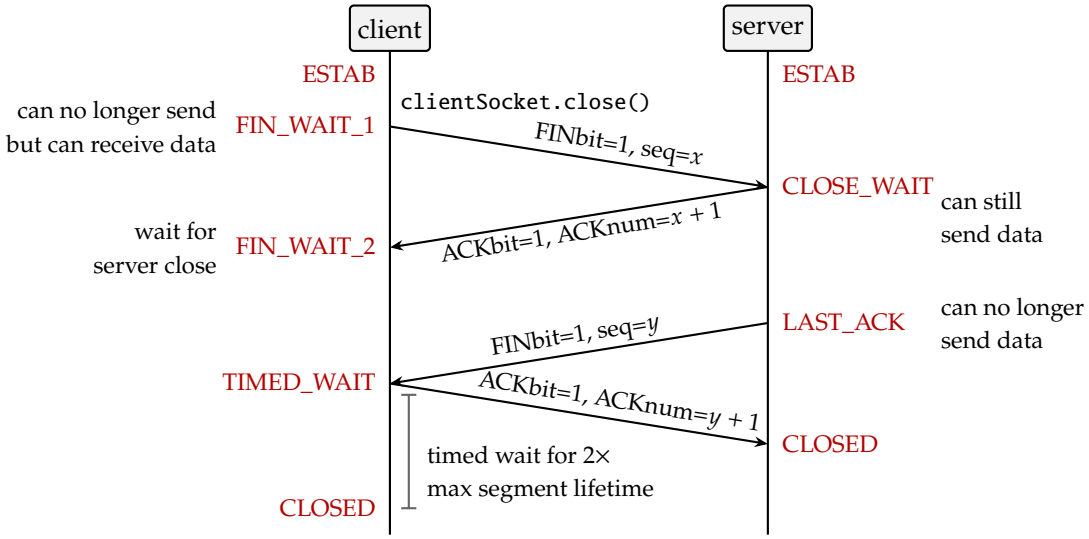


Figure 43: Closing a TCP connection. After its FIN the client can no longer send but can still receive data; the server can still send data until it sends its own FIN. The client waits 2× the maximum segment lifetime in TIMED_WAIT before closing.

3.6 Principles of congestion control

Definition: CONGESTION, informally, is "too many sources sending too much data too fast for the *network* to handle." It manifests as long delays (queueing in router buffers) and packet loss (buffer overflow at routers).

Congestion control is different from flow control: congestion means too many senders sending too fast, while flow control handles one sender too fast for one receiver. Three scenarios show the causes and costs of congestion (Figure 44). Let λ_{in} be the rate of original data, λ'_{in} the transport-layer sending rate including retransmissions, and λ_{out} the throughput.

Scenario 1. One router with infinite buffers, input and output link capacity R, two flows, and no retransmissions. The maximum per-connection throughput is $R/2$, and the delay becomes large as the arrival rate λ_{in} approaches $R/2$.

Scenario 2. One router with finite buffers, and a sender that retransmits a lost, timed-out packet. The application-layer input equals the application-layer output ($\lambda_{\text{in}} = \lambda_{\text{out}}$), but the transport-layer input includes retransmissions, $\lambda'_{\text{in}} \geq \lambda_{\text{in}}$.

- *Idealization, perfect knowledge:* the sender sends only when router buffers are available, so $\lambda_{\text{out}} = \lambda_{\text{in}}$ up to $R/2$.
- *Idealization, some perfect knowledge:* packets can be dropped at the router due to full buffers, but the sender knows when a packet has been dropped and resends only packets known to be lost. When sending at $R/2$, some packets are needed retransmissions, so throughput falls below $R/2$: capacity is "wasted" on retransmissions.
- *Realistic, un-needed duplicates:* the sender can also time out prematurely and send two copies, both of which are delivered. Throughput falls further, since retransmissions now include needed and un-needed duplicates.

The costs of congestion are more work (retransmission) for a given receiver throughput, and un-needed retransmissions: the link carries multiple copies of a packet, decreasing the maximum achievable throughput.

Scenario 3. Four senders, multi-hop paths, and timeout and retransmission. As the red rate λ'_{in} increases, all arriving blue packets at the upper queue are dropped, and blue throughput goes to 0. Another cost of congestion: when a packet is dropped, any upstream transmission capacity and buffering used for that packet was wasted.

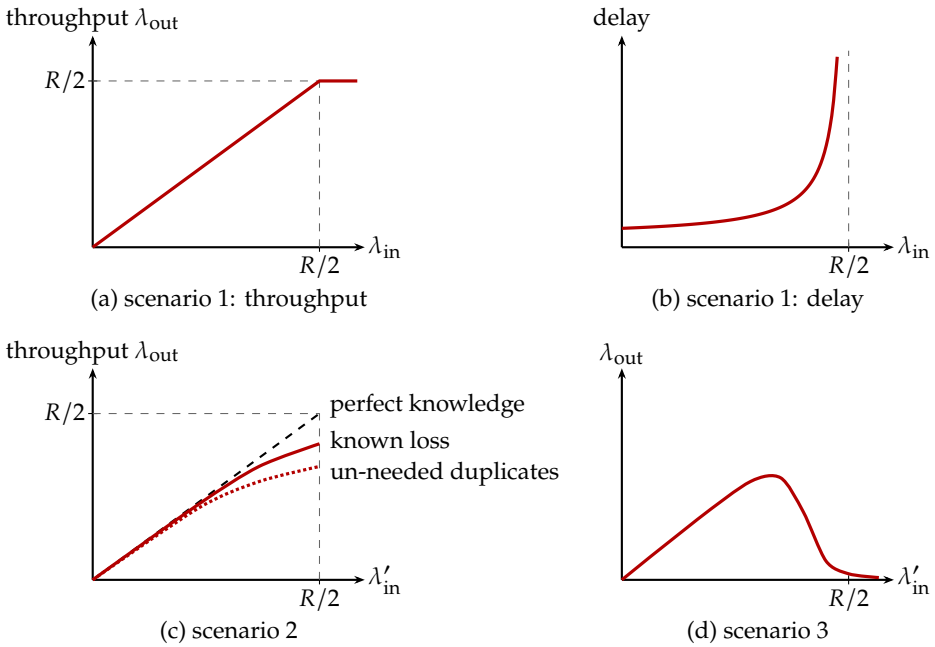


Figure 44: Schematic throughput and delay curves. (a), (b) Scenario 1: throughput never exceeds capacity, and delay grows as capacity is approached. (c) Scenario 2: loss and retransmission reduce effective throughput, and un-needed duplicates reduce it further. (d) Scenario 3: upstream capacity is wasted on packets lost downstream, and throughput collapses.

There are two approaches to congestion control. In END-END CONGESTION CONTROL there is no explicit feedback from the network: congestion is inferred from observed loss and delay; this is the approach taken by TCP. In NETWORK-ASSISTED CONGESTION CONTROL, routers provide direct feedback to sending and receiving hosts whose flows pass through a congested router; the feedback may indicate a congestion level or explicitly set a sending rate (TCP ECN, ATM, and DECbit protocols).

3.7 TCP congestion control

Definition: AIMD (additive increase, multiplicative decrease): senders increase their sending rate until packet loss (congestion) occurs, then decrease it on a loss event. *Additive increase*: increase the sending rate by 1 maximum segment size every RTT until loss is detected. *Multiplicative decrease*: cut the sending rate in half at each loss event.

The result is a sawtooth: the TCP sender probes for bandwidth (Figure 46a). In detail, the rate is cut in half on a loss detected by a triple duplicate ACK (TCP Reno) and cut to 1 MSS when loss is detected by a timeout (TCP Tahoe). AIMD, a distributed, asynchronous algorithm, has been shown to optimize congested flow rates network-wide [Kelly] and to have desirable stability properties.

The sender keeps a CONGESTION WINDOW `cwnd` and limits transmission so that

$$\texttt{LastByteSent} - \texttt{LastByteAcked} \leq \texttt{cwnd},$$

so the bytes sent but not yet ACKed ("in flight") fit in the window. Roughly, it sends `cwnd` bytes, waits an RTT for ACKs, and then sends more, so

$$\text{TCP rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec.}$$

`cwnd` is adjusted dynamically in response to observed network congestion; this implements TCP congestion control.

Slow start. When a connection begins, the rate increases exponentially until the first loss event: initially `cwnd` = 1 MSS, and `cwnd` doubles every RTT (one segment, then two, then four), which is done by incrementing `cwnd` for every ACK received. The initial rate is slow but ramps up exponentially fast. The exponential increase switches to linear when `cwnd` reaches half its value before the timeout. This is implemented with a variable `ssthresh`: on a loss event, `ssthresh` is set to half of `cwnd` just before the loss. Figure 45 shows a trace, and Table 2 gives the full congestion-control FSM, with states slow start, congestion avoidance, and fast recovery.

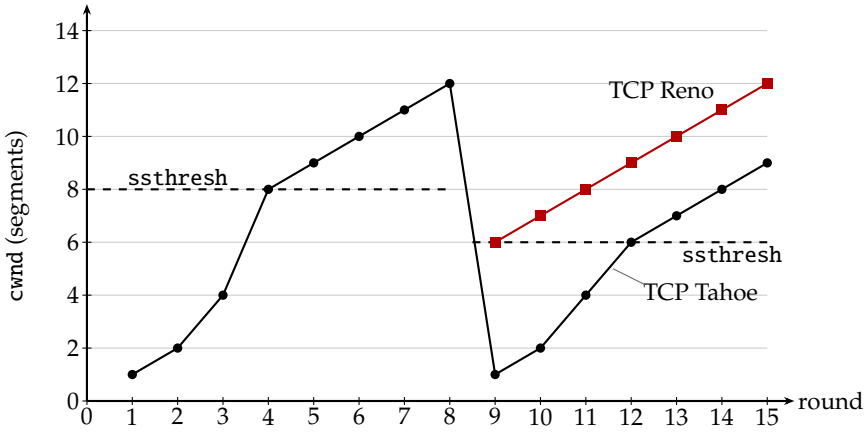
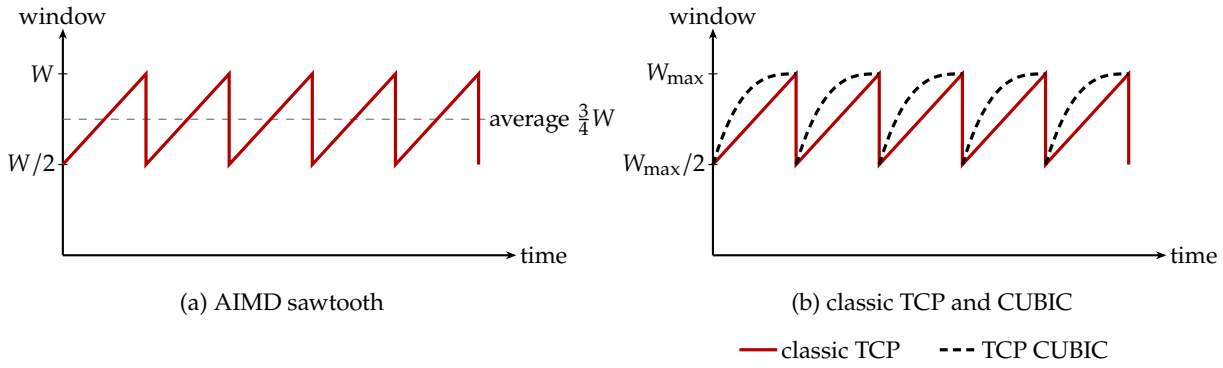


Figure 45: Evolution of `cwnd` (in segments) per transmission round. With `ssthresh` = 8, slow start grows `cwnd` as $1, 2, 4, 8$ and congestion avoidance as $9, 10, 11, 12$. A loss after round 8 sets `ssthresh` = $12/2 = 6$. TCP Tahoe restarts from 1 MSS in slow start and grows linearly once it reaches 6; TCP Reno continues from `ssthresh` = 6 and grows linearly.

TCP CUBIC. CUBIC probes for usable bandwidth better than AIMD. Let $W_{\max}$ be the sending rate at which congestion loss was detected. The congestion state of the bottleneck link has probably not changed much, so after cutting the rate (window) in half on loss, CUBIC initially ramps to $W_{\max}$ faster but then approaches $W_{\max}$ more slowly (Figure 46b). Let K be the point in time when the window will reach $W_{\max}$ (K itself is tunable). CUBIC increases W as a function of the *cube* of the distance between the current time and K: larger increases when further from K, smaller (cautious) increases nearer K. CUBIC is the default in Linux and was the most popular TCP for popular Web servers until about 2024.

State	Event	Actions	Next state
(start)	Λ	`cwnd` = 1 MSS, `ssthresh` = 64 KB, `dupACKcount` = 0	slow start
slow start	new ACK	`cwnd` = `cwnd` + MSS, `dupACKcount` = 0, transmit new segment(s) as allowed	slow start
	duplicate ACK	`dupACKcount`++	slow start
	`cwnd` $\geq$ `ssthresh`	Λ	cong. avoidance
	timeout	`ssthresh` = `cwnd`/2, `cwnd` = 1 MSS, `dupACKcount` = 0, retransmit missing segment	slow start
	`dupACKcount` = 3	`ssthresh` = `cwnd`/2, `cwnd` = `ssthresh` + 3, retransmit missing segment	fast recovery
cong. avoidance	new ACK	`cwnd` = `cwnd` + MSS $\cdot$ (MSS/`cwnd`), `dupACKcount` = 0, transmit new segment(s) as allowed	cong. avoidance
	duplicate ACK	`dupACKcount`++	cong. avoidance
	timeout	`ssthresh` = `cwnd`/2, `cwnd` = 1 MSS, `dupACKcount` = 0, retransmit missing segment	slow start
	`dupACKcount` = 3	`ssthresh` = `cwnd`/2, `cwnd` = `ssthresh` + 3, retransmit missing segment	fast recovery
fast recovery	duplicate ACK	`cwnd` = `cwnd` + MSS, transmit new segment(s) as allowed	fast recovery
	new ACK	`cwnd` = `ssthresh`, `dupACKcount` = 0	cong. avoidance
	timeout	`ssthresh` = `cwnd`/2, `cwnd` = 1, `dupACKcount` = 0, retransmit missing segment	slow start

Table 2: TCP congestion control as an FSM. In congestion avoidance `cwnd` grows by about one MSS per RTT; fast recovery inflates `cwnd` by one MSS per duplicate ACK and deflates it to `ssthresh` on a new ACK.



(a) AIMD sawtooth

(b) classic TCP and CUBIC

Figure 46: (a) The AIMD sawtooth of classic TCP, with window between $W/2$ and W. (b) Classic TCP and TCP CUBIC after the same losses: CUBIC ramps toward $W_{\max}$ faster and then flattens near it, which gives higher throughput in this example.

The congested bottleneck link. Classic TCP and CUBIC increase the sending rate until packet loss occurs at some router's output, the BOTTLENECK LINK, which is almost always busy and whose packet queue is almost never empty and sometimes overflows. Two insights follow. Increasing the TCP sending rate will *not* increase end-end throughput at a congested bottleneck, which is already transmitting at its full rate R; and it *will* increase the measured RTT, through increased queueing delay at the bottleneck. The goal is to "keep the end-end pipe just full, but not fuller." When N flows collectively congest a bottleneck link, each running its own TCP congestion control, each should ideally receive throughput R/N.

Keeping the pipe just full enough means keeping the bottleneck link busy transmitting while avoiding high delays and buffering. The measured throughput is

$$\text{measured throughput} = \frac{\text{\# bytes sent (in flight) in the RTT interval}}{\text{RTT}_{\text{measured}}},$$

and the ideal amount of in-flight data for a connection is its BANDWIDTH-DELAY PRODUCT:

$$n_{\text{inflight}} = \text{connection's share of bottleneck bandwidth} \times \text{min_RTT}.$$

Below this amount, increasing n_{inflight} does not increase the RTT, because packet data arrives at the bottleneck queue more slowly than the link transmission rate; at it, the arrival rate equals the link transmission rate; beyond it, increasing n_{inflight} increases the RTT, up to the bandwidth-delay product plus the queueing delay of a full bottleneck buffer (Figure 47). Loss-based TCP operates where the buffer is full; BBR operates at the bandwidth-delay product, keeping the pipe just full.

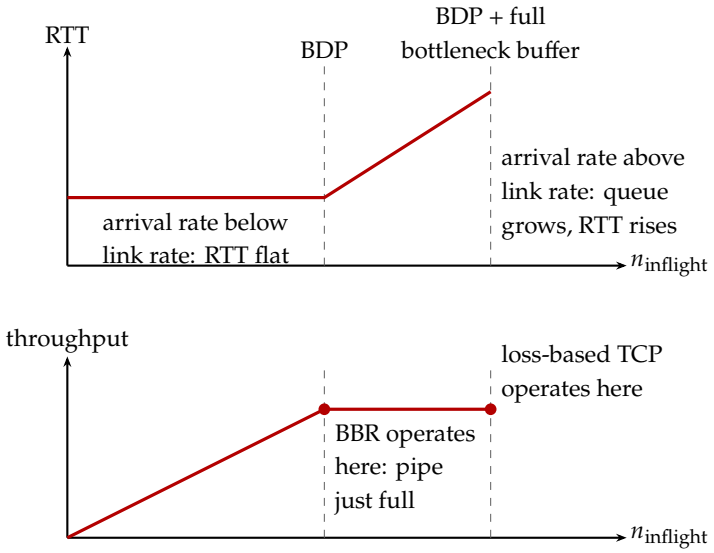


Figure 47: RTT and throughput as functions of the amount of in-flight data (schematic).

Definition: BBR (Bottleneck Bandwidth and RTT) regulates n_{inflight}. At longer time intervals it reduces n_{inflight} to drain the pipe and measure a new min_RTT. At shorter time intervals it cycles

through *acceleration* (increase the sending rate and n_{inflight} until throughput reaches a plateau, saturating the available per-flow link capacity), *cruising* (send at the rate the network is delivering data, as evidenced by received ACKs), and *deceleration* (purposefully reduce n_{inflight}, decreasing queue pressure, looking for a lower min_RTT).

Explicit congestion notification (ECN). TCP deployments often implement network-assisted congestion control. Two bits in the IP header (ToS field) are marked by a network router to indicate congestion, with the marking policy chosen by the network operator. The congestion indication is carried to the destination, which sets the ECE bit on an ACK segment to notify the sender. ECN thus involves both IP (ECN bit marking in the IP header) and TCP (C and E bit marking in the TCP header). For example, the source sends a datagram with ECN = 10, a congested router marks it ECN = 11, and the destination returns an ACK with ECE = 1.

Fairness. The fairness goal: if K TCP sessions share the same bottleneck link of bandwidth R, each should have an average rate of R/K. For two competing sessions (Figure 48), additive increase moves the throughput pair along a line of slope 1, and multiplicative decrease halves both throughputs, moving the pair proportionally toward the origin. Repeated, this converges to the equal-bandwidth-share line, so TCP is fair under idealized assumptions: the same RTT, and a fixed number of sessions, all in congestion avoidance.

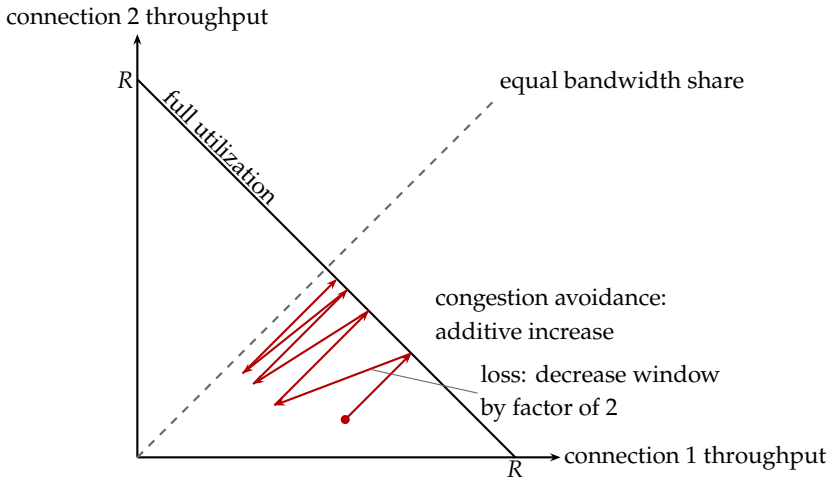


Figure 48: Two competing TCP sessions (schematic). Each loss halves both windows; congestion avoidance then increases both additively. The trajectory approaches the equal bandwidth share on the full-utilization line.

Not all network applications are "fair." Multimedia applications often do not use TCP, because they do not want their rate throttled by congestion control; they use UDP instead, sending audio or video at a constant rate and tolerating packet loss, and there is no "Internet police" policing the use of congestion control. An application can also open multiple parallel TCP connections between two hosts, as web browsers do. On a link of rate R with 9 existing connections, a new application that

asks for 1 TCP connection gets rate $R/10$, while one that asks for 11 gets $11R/20$, about $R/2$.

TCP throughput. Ignoring slow start and assuming there is always data to send, let W be the window size (in bytes) at which loss occurs. The window oscillates between $W/2$ and W, so the average window (the number of in-flight bytes) is $\frac{3}{4}W$ and

$$\text{average TCP throughput} = \frac{3}{4} \cdot \frac{W}{\text{RTT}} \text{ bytes/sec.}$$

For TCP over "long, fat pipes," take 1500-byte segments, a 100 ms RTT, and a desired throughput of 10 Gbps. This requires $W = 10^{10} \cdot 0.1/(1500 \cdot 8) \approx 83{,}333$ in-flight segments. In terms of the segment loss probability L [Mathis 1997],

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT}\sqrt{L}},$$

so 10 Gbps requires $\sqrt{L} = 1.22 \cdot 12{,}000/(0.1 \cdot 10^{10})$, a loss rate of $L \approx 2 \cdot 10^{-10}$, which is very small. This motivates versions of TCP for long, high-speed scenarios.

3.8 Evolution of transport-layer functionality

TCP and UDP have been the principal transport protocols for 40 years, and different "flavors" of TCP have been developed for specific scenarios:

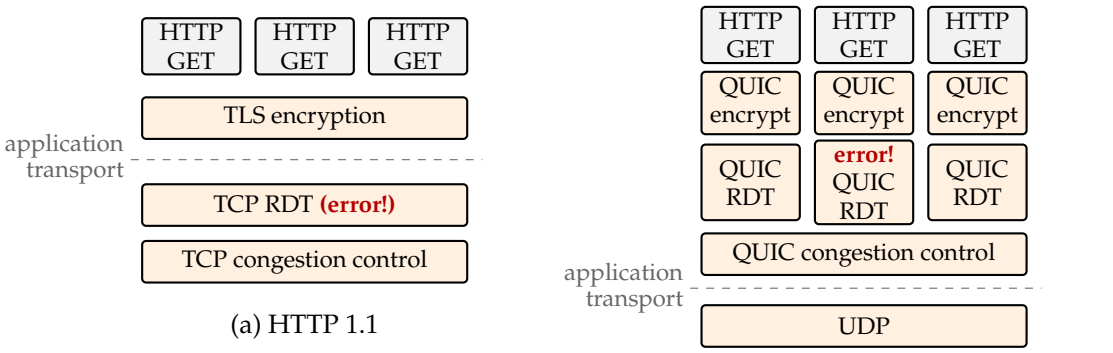
Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets "in flight"; loss shuts down the pipeline
Wireless networks	Loss due to noisy wireless links and mobility; TCP treats this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, "background" TCP flows

Transport-layer functions are also moving to the application layer, on top of UDP, as in HTTP/3 over QUIC.

Definition: QUIC (Quick UDP Internet Connections) is an application-layer protocol on top of UDP that increases the performance of HTTP. It is deployed on many Google servers and apps (Chrome, the mobile YouTube app).

Layer	HTTP/2 over TCP	HTTP/3 over UDP
Application	HTTP/2, TLS	HTTP/2 (slimmed) and QUIC, together HTTP/3
Transport	TCP	UDP
Network	IP	IP

QUIC adopts the approaches of this chapter for connection establishment, error control, and congestion control. For error and congestion control, in the words of the QUIC specification, "readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones." For connection establishment, reliability, congestion control, authentication, encryption, and state are established in one (or even zero) RTT. Multiple application-level STREAMS are multiplexed over a single QUIC connection, with separate reliable data transfer and security per stream and common congestion control. This gives parallelism without head-of-line (HOL) blocking (Figure 49).



(a) HTTP 1.1

(b) HTTP/2 with QUIC: no HOL blocking

Figure 49: (a) HTTP 1.1: several HTTP GETs share one TLS-encrypted TCP connection, so an error in TCP's reliable data transfer blocks them all. (b) HTTP/2 with QUIC: each stream has its own QUIC encryption and reliable data transfer over UDP, under common QUIC congestion control, so an error in one stream does not block the others.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 3 slides (9th edition).

Lecture 4: The Network Layer: The Data Plane

4.1 Network layer overview

The network layer transports a segment from the sending host to the receiving host. The sender encapsulates segments into datagrams and passes them to the link layer; the receiver delivers the segments to its transport-layer protocol. Network-layer protocols run in every Internet device, hosts and routers alike. A router examines header fields in all IP datagrams passing through it and moves datagrams from input ports to output ports to transfer them along the end-end path.

Definition: The network layer has two key functions. Forwarding moves packets from a router's input link to the appropriate router output link. Routing determines the route taken by packets from source to destination, using routing algorithms. On a trip, forwarding is the process of getting through a single interchange, and routing is the process of planning the trip from source to destination.

The data plane is a local, per-router function: it determines how a datagram arriving on a router input port is forwarded to a router output port, by looking up values in the arriving packet's header in a forwarding table. In the local table $\{0100 \mapsto 3,\ 0110 \mapsto 2,\ 0111 \mapsto 2,\ 1001 \mapsto 1\}$, for example, a packet whose header value is 0111 leaves on output port 2. The control plane is network-wide logic: it determines how a datagram is routed among routers along the end-end path from source host to destination host. There are two control-plane approaches.

- *Traditional routing algorithms*, implemented in routers (a per-router control plane): the routing-algorithm components in each and every router interact with one another, and each computes its own local forwarding table.
- Software-defined networking (SDN), implemented in (remote) servers: a remote controller computes forwarding tables and installs them in the routers, through a control agent (CA) in each router.

Service model. Which service should the "channel" transporting datagrams from sender to receiver provide? Example services for individual datagrams are guaranteed delivery, and guaranteed delivery with less than 40 msec delay. Example services for a flow of datagrams are in-order delivery, a guaranteed minimum bandwidth to the flow, and restrictions on changes in inter-packet spacing.

Some network architectures and their quality-of-service (QoS) guarantees:

Architecture	Service model	Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	constant bit rate	constant rate	yes	yes	yes
ATM	available bit rate	guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

The Internet's BEST-EFFORT service model gives no guarantees on (i) successful datagram delivery to the destination, (ii) timing or order of delivery, or (iii) bandwidth available to an end-end flow. Yet it has succeeded, for several reasons. The simplicity of the mechanism allowed the Internet to be widely deployed and adopted. Sufficient provisioning of bandwidth makes real-time applications (interactive voice, video) "good enough" for "most of the time". Replicated, application-layer distributed services (datacenters, content distribution networks) that connect close to clients' networks let services be provided from multiple locations. Congestion control of "elastic" services helps as well.

4.2 What's inside a router

A generic router (Figure 50) has input ports, a high-speed switching fabric, and output ports, which form the forwarding data plane, implemented in hardware and operating on a nanosecond timescale. A routing processor runs the routing and management control plane, implemented in software and operating on a millisecond timescale.

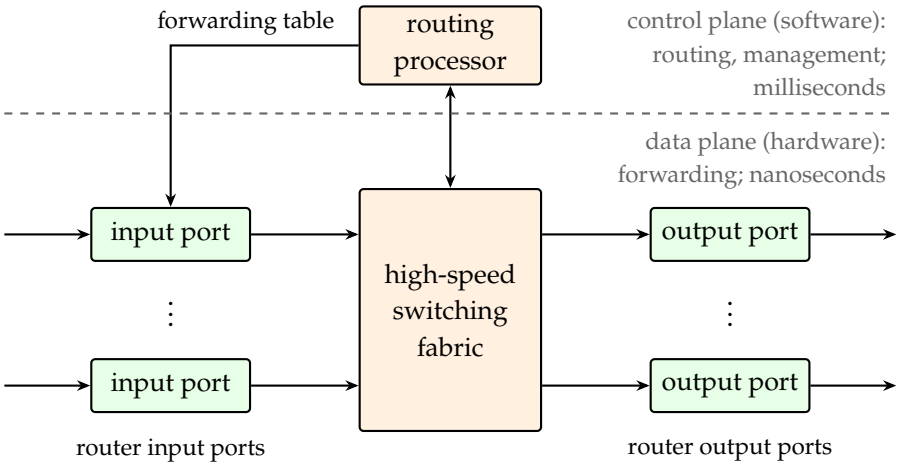


Figure 50: Router architecture. Datagrams flow from input ports through the switching fabric to output ports; the routing processor computes the forwarding table used by the input ports.

Input ports. An input port (Figure 51a) performs line termination (physical-layer, bit-level reception), runs the link-layer protocol (for example Ethernet), and then does lookup, forwarding,

and queueing. This is DECENTRALIZED SWITCHING: using header field values, the port looks up the output port in a forwarding table held in input-port memory ("match plus action"). The goal is to complete input-port processing at "line speed". INPUT PORT QUEUEING occurs if datagrams arrive faster than the forwarding rate into the switch fabric. Lookup comes in two forms.

- DESTINATION-BASED FORWARDING forwards based only on the destination IP address (the traditional approach).
- GENERALIZED FORWARDING forwards based on any set of header field values.

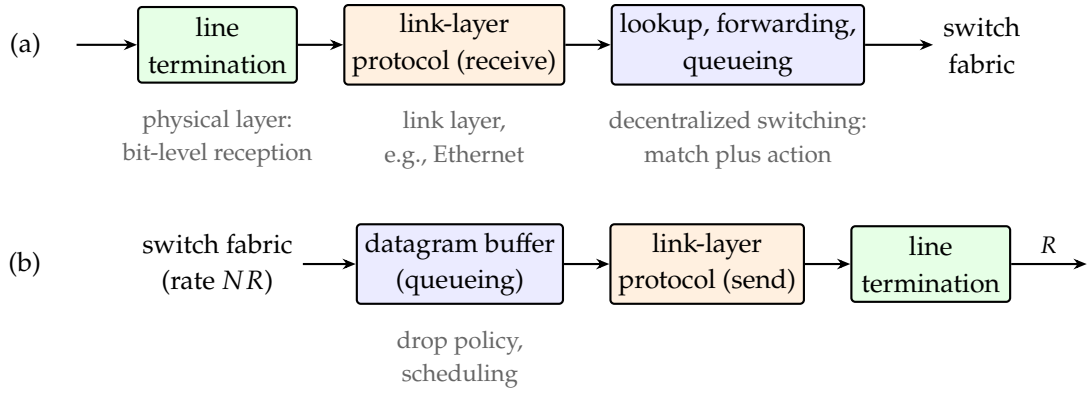


Figure 51: (a) Input-port and (b) output-port processing.

Destination-based forwarding. A forwarding table can map ranges of destination addresses to link interfaces:

Destination address range	Link interface
`11001000 00010111 00010000 00000000` through `11001000 00010111 00010111 11111111`	0
`11001000 00010111 00011000 00000000` through `11001000 00010111 00011000 11111111`	1
`11001000 00010111 00011001 00000000` through `11001000 00010111 00011111 11111111`	2
otherwise	3

But ranges need not divide up so nicely. If, say, the addresses `11001000 00010111 00010000 00000100` through `11001000 00010111 00010000 00000111` had to go to interface 3, they would cut a hole in the middle of interface 0's range.

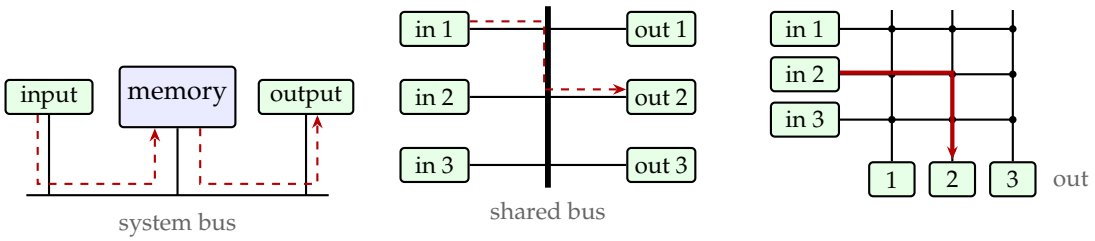
Definition: LONGEST PREFIX MATCHING: when looking for the forwarding-table entry for a given destination address, use the longest address prefix that matches the destination address.

With prefixes, the same table becomes

Destination address prefix	Link interface
`11001000 00010111 00010*** ********`	0
`11001000 00010111 00011000 ********`	1
`11001000 00010111 00011*** ********`	2
otherwise	3

The destination `11001000 00010111 00010110 10100001` matches only the first entry, so it leaves on interface 0. The destination `11001000 00010111 00011000 10101010` matches both the second entry (24 bits) and the third (21 bits); the longer match wins, so it leaves on interface 1. Longest prefix matching is often performed with TERNARY CONTENT ADDRESSABLE MEMORIES (TCAMs). They are content addressable: a TCAM presented with an address retrieves the matching entry in one clock cycle, regardless of table size. A Cisco Catalyst holds about 1M routing-table entries in TCAM.

Switching fabrics. The fabric transfers each packet from its input link to the appropriate output link. Its SWITCHING RATE, the rate at which packets can be transferred from inputs to outputs, is often measured as a multiple of the input/output line rate: with N inputs of rate R, a switching rate of NR is desirable. There are three major types (Figure 52).



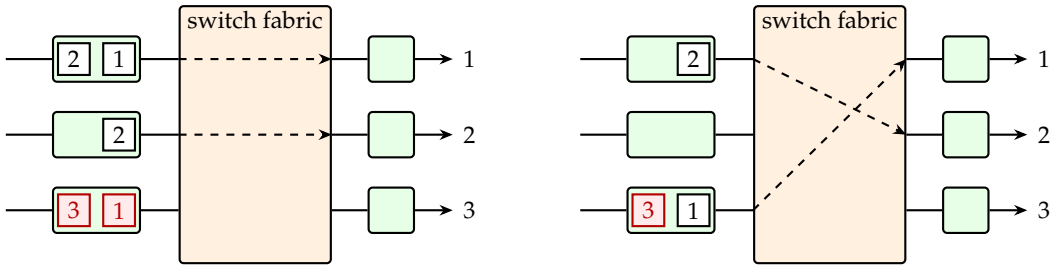
(a) memory (b) bus (c) crossbar

Figure 52: Switching via (a) memory, (b) a shared bus, and (c) an interconnection network (a 3×3 crossbar). The red path is one transfer.

- *Memory.* First-generation routers were traditional computers with switching under direct control of the CPU. The packet is copied into system memory and then out to the output port, so speed is limited by memory bandwidth (two bus crossings per datagram).
- *Bus.* A datagram moves from input-port memory to output-port memory over a shared bus. Because of BUS CONTENTION, the switching speed is limited by the bus bandwidth; a 32 Gbps bus (Cisco 5600) is fast enough for access routers.
- *Interconnection network.* Crossbars, Clos networks, and other interconnection networks were first developed to connect processors in a multiprocessor. A MULTISTAGE SWITCH builds an $n \times n$ switch from multiple stages of smaller switches (for example an 8×8 switch from 3×3 crossbars). Such fabrics exploit parallelism: a datagram is fragmented into fixed-length cells on entry, the cells are switched through the fabric, and the datagram is reassembled at the exit.

Several switching "planes" in parallel give further speedup and scale: the Cisco CRS router uses 8 switching planes, each a 3-stage interconnection network, for up to hundreds of Tbps of switching capacity.

Input port queueing. If the switch fabric is slower than all the input ports combined, queues can build at the inputs, causing queueing delay and loss from input-buffer overflow. Input queueing also suffers HEAD-OF-THE-LINE (HOL) BLOCKING: a queued datagram at the front of a queue prevents the others in that queue from moving forward, even when their own output ports are free (Figure 53).



(a) output-port contention: only one of the two head packets for output 1 is transferred; the other is blocked

(b) one packet time later: the packet for output 3 still waits behind the packet for output 1 (head-of-the-line blocking)

Figure 53: Head-of-the-line blocking. Each packet is labeled with its destination output port; blocked packets are red.

Output port queueing. An output port (Figure 51b) needs buffering when datagrams arrive from the fabric (at rate up to NR) faster than the link transmission rate R. For instance, if in one packet time two inputs each send a packet to the same output, one of them must wait in that output's buffer, and more packets for that output may keep arriving. Output queueing therefore brings queueing delay, and loss on buffer overflow. Two policies matter here: a DROP POLICY decides which datagrams to drop when no free buffers remain (datagrams can be lost to congestion, lack of buffers), and a SCHEDULING DISCIPLINE chooses among the queued datagrams for transmission (priority scheduling decides who gets the best performance, which raises the issue of network neutrality).

How much buffering? The rule of thumb of RFC 3439 is an average buffering equal to a "typical" RTT (say 250 msec) times the link capacity C: for $C = 10$ Gbps this is $0.25\text{ s} \times 10\text{ Gbps} = 2.5$ Gbit. A more recent recommendation, with N flows, is a buffer of

$$\frac{\text{RTT} \cdot C}{\sqrt{N}}.$$

Too much buffering can increase delays, particularly in home routers: long RTTs give poor performance for real-time applications and sluggish TCP response. As in delay-based congestion control, the aim is to keep the bottleneck link just full enough (busy) but no fuller.

Buffer management. Abstractly, an output port is a queue (a waiting area) in front of a link of rate R (the server). Buffer management decides which packets to drop and which to mark.

- *Drop*: which packet to add, and which to drop when buffers are full. TAIL DROP drops the arriving packet; *priority* dropping drops or removes packets on a priority basis.
- *Marking*: which packets to mark to signal congestion (ECN, RED).

Packet scheduling. PACKET SCHEDULING decides which packet to send next on the link. In FIRST COME, FIRST SERVED (FCFS, also called first-in-first-out, FIFO) scheduling, packets are transmitted in their order of arrival to the output port. In PRIORITY SCHEDULING, arriving traffic is classified and queued by class (any header fields can be used for classification), and the link sends a packet from the highest-priority queue that has buffered packets, FCFS within a priority class (Figure 54).

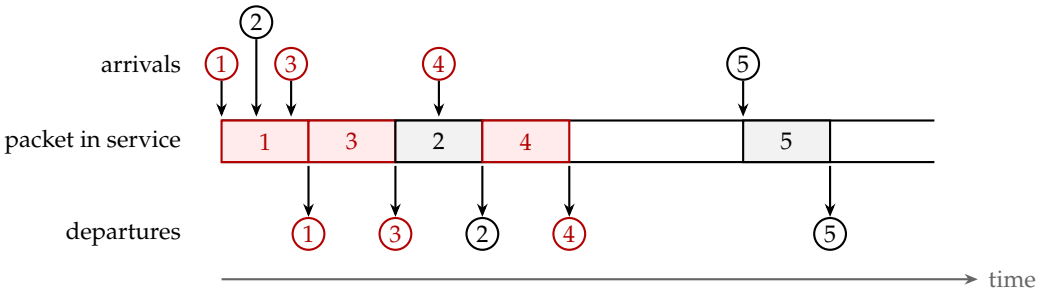


Figure 54: Priority scheduling. Packets 1, 3, 4 (red) have high priority; packets 2 and 5 have low priority. Packet 3 is sent before packet 2, which arrived earlier; packet 4 arrives while packet 2 is in service and waits for it to finish.

In ROUND ROBIN (RR) scheduling, arriving traffic is again classified and queued by class, and the server cyclically and repeatedly scans the class queues, sending one complete packet from each class (if available) in turn. WEIGHTED FAIR QUEUEING (WFQ) generalizes round robin: each class i has a weight w_i and receives a weighted amount of service in each cycle, the fraction

$$\frac{w_i}{\sum_j w_j}$$

of the link (Figure 55). This gives each traffic class a minimum bandwidth guarantee.

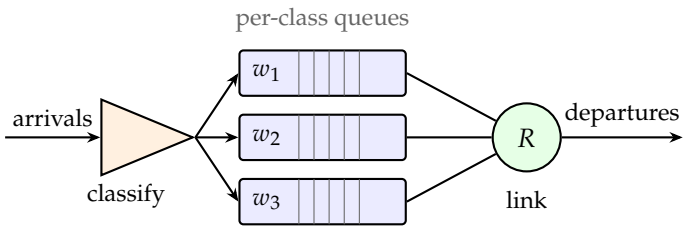


Figure 55: Per-class queues served by a link of rate R; under WFQ, class i has weight w_i.

Network neutrality. Network neutrality asks how an ISP should share and allocate its resources. It is partly technical (packet scheduling and buffer management are the mechanisms), partly a matter of social and economic principles (protecting free speech; encouraging innovation and competition), and partly enforced through legal rules and policies, which differ between countries. The 2015 US FCC *Order on Protecting and Promoting an Open Internet* set three "clear, bright line" rules:

- *no blocking*: "shall not block lawful content, applications, services, or non-harmful devices, subject to reasonable network management";
- *no throttling*: "shall not impair or degrade lawful Internet traffic on the basis of Internet content, application, or service, or use of a non-harmful device, subject to reasonable network management";
- *no paid prioritization*: "shall not engage in paid prioritization".

The order was rolled back in 2017, and that rollback was itself rolled back in 2024. Whether an ISP is a "telecommunications service" or an "information service" provider really matters from a regulatory standpoint. Under the US Telecommunication Act of 1934 and 1996, Title II imposes "common carrier duties" (reasonable rates, non-discrimination) on telecommunications services and requires regulation; Title I applies to information services, which have no common carrier duties (they are not regulated) but over which the FCC has authority "as may be necessary in the execution of its functions".

4.3 IP: the Internet Protocol

In hosts and routers, the Internet's network layer sits between the transport layer (TCP, UDP) and the link layer. It has three parts: path-selection algorithms, implemented in routing protocols (OSPF, BGP) or in an SDN controller, which compute the forwarding table; the IP PROTOCOL, which defines the datagram format, addressing, and packet-handling conventions; and the ICMP PROTOCOL, for error reporting and router "signaling".

Datagram format. Figure 56 shows an IPv4 datagram.

- *ver*: the IP protocol version number; *head. len*: the header length; *length*: the total datagram length in bytes (maximum 64K bytes, typically 1500 bytes or less).
- *type of service*: the diffserv bits (0:5) and the ECN bits (6:7).
- *16-bit identifier*, *flgs*, *fragment offset*: used for fragmentation and reassembly (below).
- *time to live* (TTL): the remaining maximum number of hops, decremented at each router.
- *upper layer*: the upper-layer protocol (e.g., TCP or UDP) to deliver the payload to.

- *header checksum*; 32-bit source and destination IP addresses; *options* (if any), e.g., a timestamp or a record of the route taken.

The overhead is 20 bytes of TCP header plus 20 bytes of IP header, so 40 bytes plus the application-layer overhead for TCP over IP.

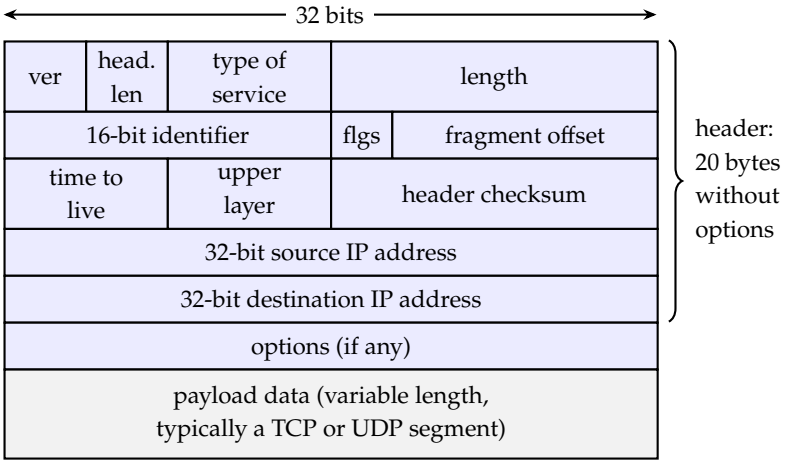


Figure 56: IPv4 datagram format.

Fragmentation and reassembly. Network links have an MTU (maximum transfer size), the largest possible link-level frame, and different link types have different MTUs. A large IP datagram is therefore divided ("fragmented") within the network: one datagram becomes several datagrams, which are "reassembled" only at the final destination. The identifier, flag, and offset bits of the IP header identify and order the related fragments.

For example, a 4000-byte datagram (a 20-byte header and 3980 bytes of data) crossing a link with MTU 1500 bytes becomes three fragments with the same identifier x. Each fragment carries at most $1500 - 20 = 1480$ data bytes, and the offset counts 8-byte units, so the second fragment starts at offset $1480/8 = 185$ and the third at $2960/8 = 370$; the flag is 1 on every fragment but the last:

	length	ID	fragflag	offset
original datagram	4000	x	0	0
fragment 1	1500	x	1	0
fragment 2	1500	x	1	185
fragment 3	1040	x	0	370

The data bytes add up: $1480 + 1480 + 1020 = 3980$.

Addresses and interfaces. An IP ADDRESS is a 32-bit identifier associated with each host or router INTERFACE, the connection between a host or router and a physical link. Routers typically have multiple interfaces; a host typically has one or two (e.g., wired Ethernet and wireless 802.11).

Addresses are written in dotted-decimal notation, one decimal number per byte:

$$223.1.1.1 = \texttt{11011111 00000001 00000001 00000001}.$$

Interfaces that reach each other with no intervening router are connected, for example, by Ethernet switches (wired) or a WiFi base station (wireless); for the network layer it does not matter how.

Definition: A SUBNET is a set of device interfaces that can physically reach each other without passing through an intervening router. IP addresses have structure: the SUBNET PART, the high-order bits that devices in the same subnet have in common, and the HOST PART, the remaining low-order bits. To define the subnets, detach each interface from its host or router, creating "islands" of isolated networks; each isolated network is a subnet.

Figure 57 shows a network of three subnets with SUBNET MASK /24: the high-order 24 bits are the subnet part, so the subnets are 223.1.1.0/24, 223.1.2.0/24, and 223.1.3.0/24. In a network of three routers joined pairwise, each router-to-router link is also a subnet: such a network with host subnets 223.1.1/24, 223.1.2/24, 223.1.3/24 and router links 223.1.7/24, 223.1.8/24, 223.1.9/24 has six subnets.

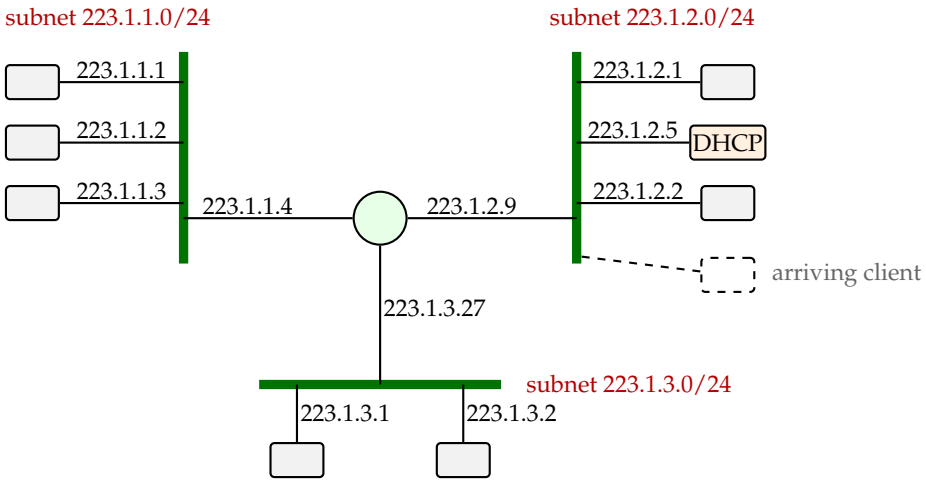


Figure 57: A router with interfaces 223.1.1.4, 223.1.2.9, 223.1.3.27 joining three subnets. A DHCP server (223.1.2.5) on subnet 223.1.2.0/24 serves a client arriving there.

Definition: CIDR (Classless InterDomain Routing, pronounced "cider") allows the subnet portion of an address to have arbitrary length. The address format is $a.b.c.d/x$, where x is the number of bits in the subnet portion. For example, in 200.23.16.0/23, the first 23 bits of `11001000 00010111 00010000 00000000` are the subnet part and the remaining 9 bits are the host part.

Getting an address. There are two questions: how a host gets an IP address within its network (the host part), and how a network gets an IP address for itself (the network part). A host's address is either hard-coded by the system administrator in a configuration file (e.g., `/etc/rc.config` in

UNIX) or obtained dynamically from a server by DHCP, the Dynamic Host Configuration Protocol ("plug and play").

DHCP. With DHCP a host dynamically obtains an IP address from a network server when it "joins" the network. A host can renew its lease on an address in use; addresses can be reused (a host holds an address only while connected or on); and mobile users can join and leave the network. The exchange has four messages:

1. the host broadcasts a DHCP DISCOVER message (optional);
2. the DHCP server responds with a DHCP OFFER message (optional);
3. the host requests an IP address with a DHCP REQUEST message;
4. the DHCP server sends the address in a DHCP ACK message.

The DHCP server is typically co-located in the router, serving all subnets to which the router is attached. Figure 58 traces the exchange for the client arriving in Figure 57. Every message is broadcast to 255.255.255.255; the client uses port 68 and the server port 67, and the client, which has no address yet, sends from 0.0.0.0. The transaction ID ties each request to its reply, and `yiaddr` ("your IP address") carries the offered address 223.1.2.4, with a lifetime of 3600 seconds. The discover and offer steps can be skipped "if a client remembers and wishes to reuse a previously allocated network address" (RFC 2131).

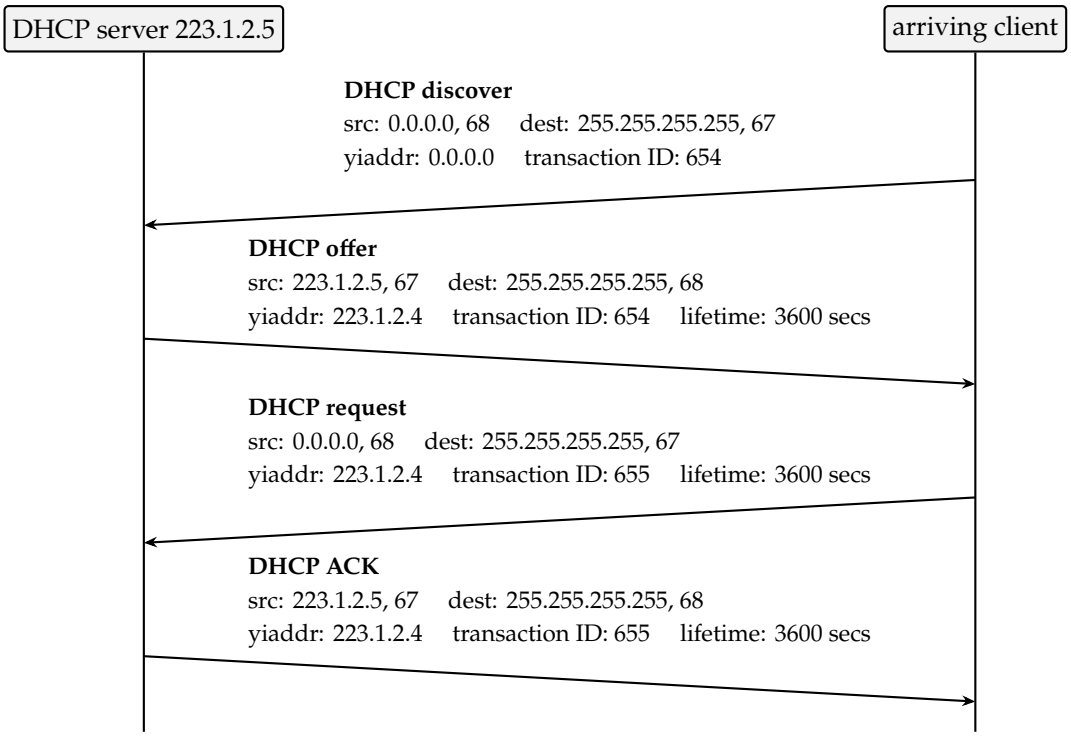


Figure 58: DHCP discover, offer, request, and ACK between the DHCP server 223.1.2.5 and an arriving client.

DHCP can return more than the allocated IP address on the subnet: the address of the client's first-hop router, the name and IP address of the DNS server, and the network mask (which indicates the network and host portions of the address). For example, a connecting laptop uses DHCP to get its IP address, its first-hop router's address, and the DNS server's address. Its DHCP request message is encapsulated in UDP, in IP, and in Ethernet; the Ethernet frame is broadcast on the LAN (destination `FFFFFFFFFFFF`) and received at the router running the DHCP server, which demultiplexes it from Ethernet up to IP, UDP, and finally DHCP. The server formulates a DHCP ACK containing the client's IP address, the IP address of its first-hop router, and the name and IP address of the DNS server; the encapsulated reply is forwarded to the client and demultiplexed up to DHCP there. The client then knows its IP address, the name and IP address of its DNS server, and the IP address of its first-hop router.

Example: *A captured DHCP exchange on a home LAN.* The request (message type Boot Request, transaction ID `0x6b3a11b7`, client MAC address `00:16:d3:23:68:8a`) is a DHCP Request asking for the IP address 192.168.1.101, with host name `nomad` and a parameter request list that includes subnet mask, domain name, router, and domain name server. The reply (Boot Reply, the same transaction ID) is a DHCP ACK from server 192.168.1.1 that lists client address 192.168.1.101, subnet mask 255.255.255.0, router 192.168.1.1, domain name servers 68.87.71.226, 68.87.73.242, 68.87.64.146, and domain name `hsd1.ma.comcast.net.`

Getting a network prefix. A network gets the subnet part of its addresses as an allocated portion of its provider ISP's address space. An ISP with block 200.23.16.0/20 can allocate it in 8 blocks of size /23:

	Address (prefix underlined)	Prefix
ISP's block	11001000 00010111 00010000 00000000	200.23.16.0/20
Organization 0	11001000 00010111 00010000 00000000	200.23.16.0/23
Organization 1	11001000 00010111 00010010 00000000	200.23.18.0/23
Organization 2	11001000 00010111 00010100 00000000	200.23.20.0/23
...	...	...
Organization 7	11001000 00010111 00011110 00000000	200.23.30.0/23

Hierarchical addressing allows efficient advertisement of routing information (route aggregation): Fly-By-Night-ISP tells the rest of the Internet "send me anything with addresses beginning 200.23.16.0/20", covering all eight organizations with one route, while ISPs-R-Us advertises 199.31.0.0/16. Suppose Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us. ISPs-R-Us now advertises a *more specific* route, "or 200.23.18.0/23" (Figure 59). A destination in 200.23.18.0/23 matches both advertised prefixes, and the longest prefix match (/23 rather than /20) sends its traffic to ISPs-R-Us; this is why routers use longest prefix matching.

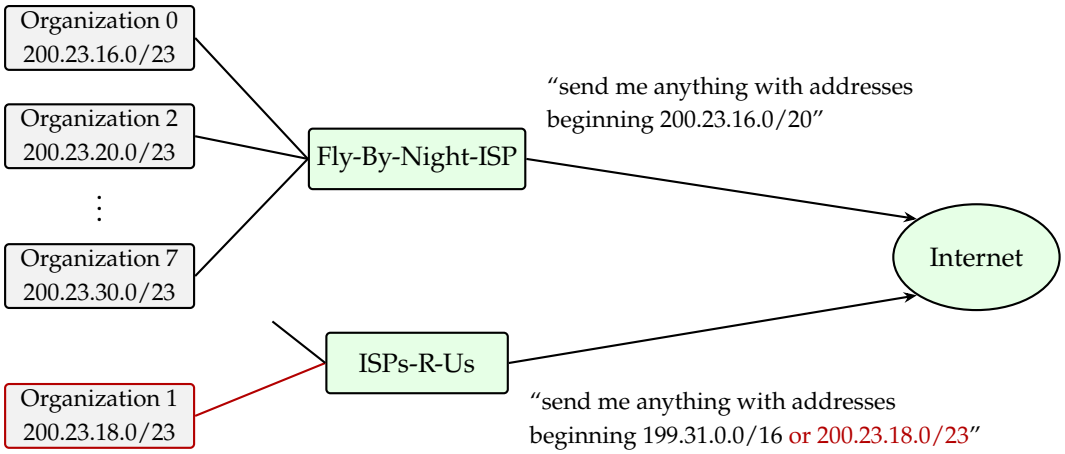


Figure 59: Route aggregation and a more specific route, after Organization 1 has moved from Fly-By-Night-ISP to ISPs-R-Us.

An ISP gets its block of addresses from ICANN (Internet Corporation for Assigned Names and Numbers), which allocates IP addresses through 5 regional registries (RRs), who may then allocate to local registries, and which also manages the DNS root zone, including the delegation of individual top-level domain (`.com`, `.edu`, ...) management. There are not enough 32-bit addresses: ICANN allocated the last chunk of IPv4 addresses to the RRs in 2011. NAT helps with IPv4 address-space exhaustion, and IPv6 has a 128-bit address space. ("Who the hell knew how much address space we needed?", Vint Cerf, reflecting on the decision to make IPv4 addresses 32 bits long.)

Network address translation. With NAT, all devices in a local network share just one IPv4 address as far as the outside world is concerned. The devices have 32-bit addresses in a "private" IP address space (prefixes 10/8, 172.16/12, 192.168/16) that can only be used in the local network. All datagrams leaving the local network have the same source NAT IP address but different source port numbers; datagrams inside the network use local addresses for source and destination as usual. Advantages:

- just one IP address is needed from the provider ISP for all devices;
- addresses of hosts in the local network can change without notifying the outside world;
- the ISP can change without changing the addresses of devices in the local network;
- security: devices inside the local network are not directly addressable or visible from the outside world.

A NAT router must, transparently,

- replace the (source IP address, port #) of every outgoing datagram with (NAT IP address, new port #); remote clients and servers then respond using (NAT IP address, new port #) as the destination;

- remember in a NAT TRANSLATION TABLE every such translation pair;
- replace (NAT IP address, new port #) in the destination fields of every incoming datagram with the corresponding (source IP address, port #) stored in the table.

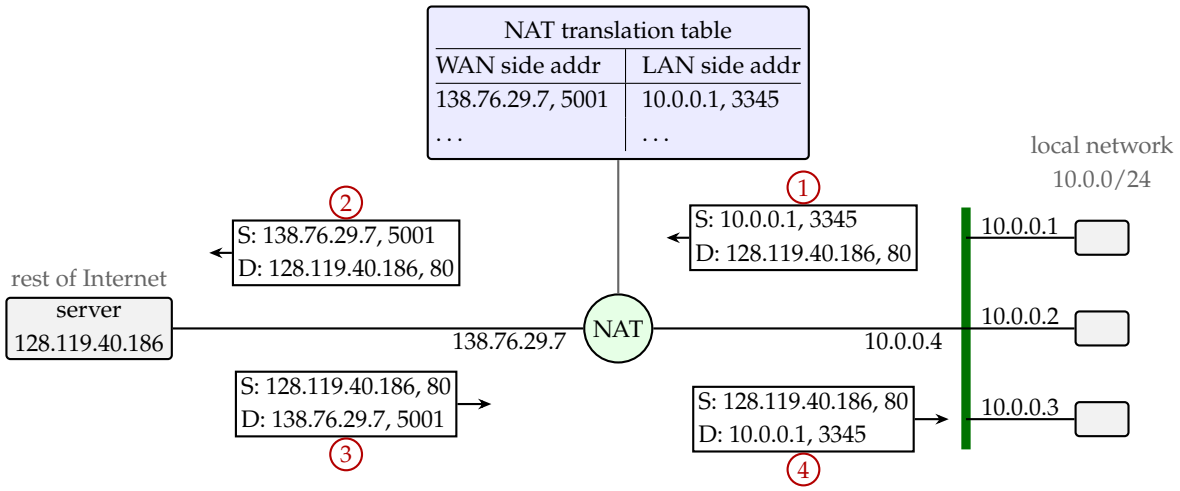


Figure 60: Network address translation for a local network 10.0.0/24 behind the single public address 138.76.29.7.

In Figure 60, host 10.0.0.1 sends a datagram from port 3345 to 128.119.40.186, port 80 (step 1). The NAT router changes the source to 138.76.29.7, port 5001, and records the pair in its table (step 2). The reply arrives addressed to 138.76.29.7, port 5001 (step 3), and the router rewrites the destination to 10.0.0.1, port 3345 (step 4).

NAT has been controversial: routers "should" only process up to layer 3; the address "shortage" should be solved by IPv6; NAT violates the end-to-end argument (port numbers are manipulated by a network-layer device); and NAT traversal is needed when a client wants to connect to a server behind a NAT. But NAT is here to stay: it is used extensively in home and institutional networks and in 4G/5G cellular networks.

IPv6. The initial motivation for IPv6 was that the 32-bit IPv4 address space would be completely allocated. Additional motivations were to speed processing and forwarding with a 40-byte fixed-length header and to enable different network-layer treatment of "flows". In the IPv6 datagram (Figure 61), *priority* identifies priority among datagrams in a flow, and the *flow label* identifies datagrams in the same "flow" (a concept that is not well defined); addresses are 128 bits long. Compared with IPv4, IPv6 has no checksum (to speed processing at routers), no fragmentation or reassembly, and no options (these are available as an upper-layer, next-header protocol at routers).

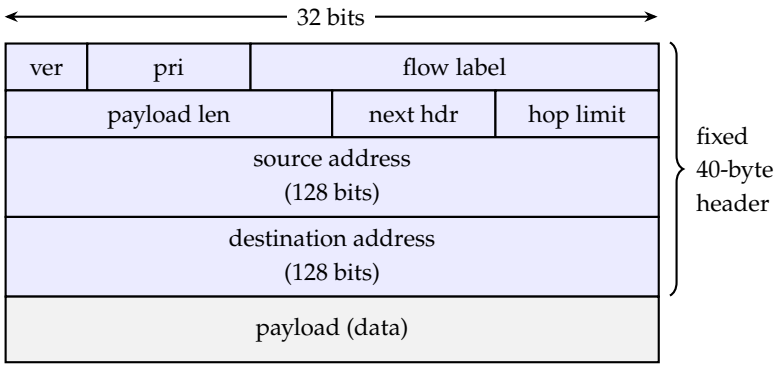


Figure 61: IPv6 datagram format.

Not all routers can be upgraded simultaneously: there are no "flag days", so the network must operate with a mix of IPv4 and IPv6 routers. The solution is TUNNELING: an IPv6 datagram is carried as the payload of an IPv4 datagram among IPv4 routers ("packet within a packet"); tunneling is also used extensively in other contexts (4G/5G). Normally an IPv6 datagram travels as the payload of a link-layer frame between two IPv6 routers. When an IPv4 network connects two IPv6 routers B and E, B places the IPv6 datagram inside an IPv4 datagram addressed from B to E, the IPv4 routers forward it, and E extracts the IPv6 datagram (Figure 62). The inner source and destination remain A and F throughout.

Adoption has been slow. About 40% of Google's clients accessed its services via IPv6 in 2023, and according to NIST about one third of all US government domains are IPv6 capable. Deployment has taken a long (long!) time, 25 years and counting, while application-level changes in the same period (the WWW, social media, streaming media, gaming, telepresence) happened far faster.

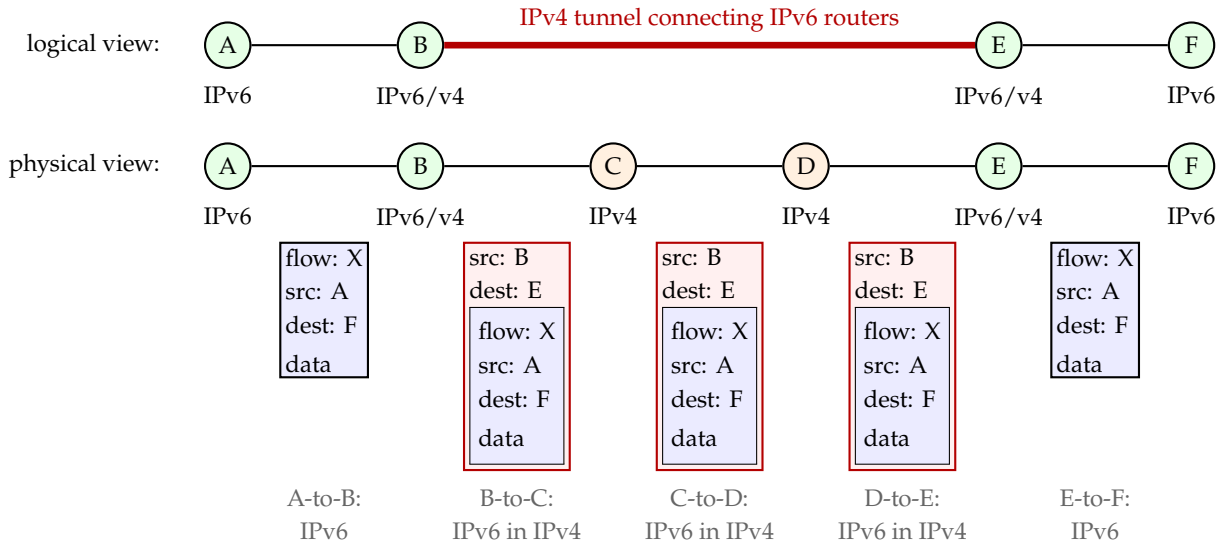


Figure 62: Tunneling IPv6 through IPv4: logical and physical views, and the datagram on each hop. B and E speak both IPv6 and IPv4; C and D speak only IPv4.

4.4 Generalized forwarding

Each router contains a forwarding table, also called a FLOW TABLE, and implements a MATCH PLUS ACTION abstraction: it matches bits in the arriving packet and takes an action. In destination-based forwarding the action is to forward based on the destination IP address. In generalized forwarding many header fields can determine the action, and many actions are possible: drop, copy, modify, or log the packet.

Definition: A FLOW is defined by header field values (in link-, network-, and transport-layer fields). GENERALIZED FORWARDING uses simple packet-handling rules:

- *match*: pattern values in packet header fields;
- *actions* for a matched packet: drop, forward, modify, or send the matched packet to the controller;
- *priority*: disambiguates overlapping patterns;
- *counters*: number of bytes and number of packets.

A router's flow table defines its match plus action rules.

For example (with * a wildcard), the flow table

Match	Action
src = *.*.*.*, dest = 3.4.*.*	forward(2)
src = 1.2.*.*, dest = *.*.*.*	drop
src = 10.1.2.3, dest = *.*.*.*	send to controller

forwards datagrams destined to 3.4.*.* out of port 2, drops datagrams from 1.2.*.*, and sends datagrams from 10.1.2.3 to the controller.

OpenFlow. An OPENFLOW flow-table entry has a match, an action, and stats (packet and byte counters). The action is one of: forward the packet to port(s); drop the packet; modify fields in the header(s); encapsulate and forward to the controller. The header fields that can be matched are, at the link layer, the ingress port, source MAC, destination MAC, Ethernet type, VLAN ID, and VLAN priority; at the network layer, the IP source, IP destination, IP protocol, and IP ToS; and at the transport layer, the TCP/UDP source and destination ports.

Examples (every field not listed is *):

Match	Action	Effect
IP Dst = 51.6.0.8	port6	destination-based forwarding to output port 6
TCP d-port = 22	drop	firewall: block all datagrams to TCP port 22 (ssh)
IP Src = 128.119.1.1	drop	firewall: block all datagrams sent by host 128.119.1.1
MAC dst = 22:A7:23:11:E1:02	port3	layer-2 destination-based forwarding to port 3

The match plus action abstraction unifies different kinds of devices:

Device	Match	Action
router	longest destination IP prefix	forward out a link
switch	destination MAC address	forward or flood
firewall	IP addresses and TCP/UDP port numbers	permit or deny
NAT	IP address and port	rewrite address and port

Orchestrated tables can create network-wide behavior. In Figure 63, datagrams from hosts h5 and h6 should be sent to h3 or h4 via s1 and from there to s2. Switch s3 forwards datagrams from 10.3.*.* to 10.2.*.* out of port 3 (to s1); s1 forwards those arriving on port 1 out of port 4 (to s2); s2 forwards those arriving on port 2 to 10.2.0.3 out of port 3 and to 10.2.0.4 out of port 4.

In summary, generalized forwarding matches bits in arriving packet headers in any layer (link, network, transport) and takes local actions (drop, forward, modify, or send the matched packet to the controller), from which network-wide behaviors can be "programmed". It is a simple form of "network programmability": programmable, per-packet "processing". Its historical roots are in active networking; today more general programming is done with P4 (see p4.org).

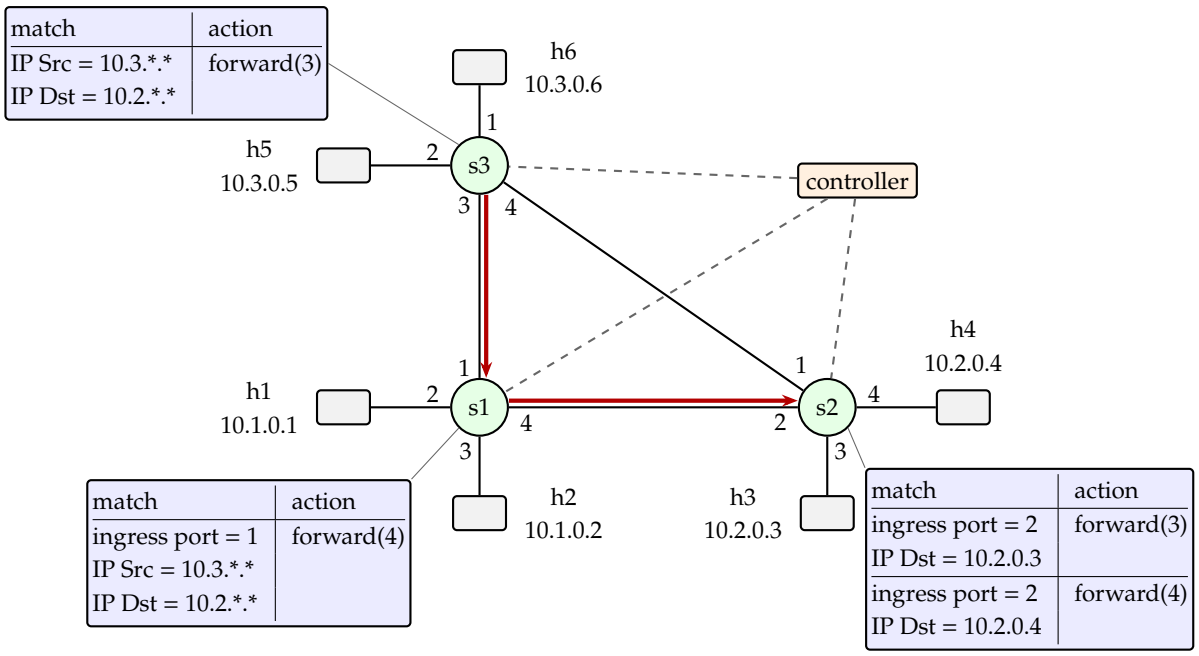


Figure 63: OpenFlow example: the flow tables of s3, s1, s2 route datagrams from h5 and h6 to h3 or h4 along the red path. Small numbers are port numbers; dashed lines go to the controller.

4.5 Middleboxes and Internet architecture

Definition: A MIDDLEBOX (RFC 3234) is "any intermediary box performing functions apart from normal, standard functions of an IP router on the data path between a source host and destination host".

Middleboxes are everywhere: NAT (home, cellular, institutional networks); firewalls and intrusion detection systems (corporate, institutional, service providers, ISPs); load balancers (corporate, service provider, data center, mobile networks); caches (service provider, mobile, CDNs); and application-specific boxes (service providers, institutional, CDN). Initially proprietary (closed) hardware, they are moving toward "whitebox" hardware implementing an open API, with programmable local actions via match plus action and innovation and differentiation in software; toward SDN, (logically) centralized control and configuration management, often in a private or public cloud; and toward NETWORK FUNCTIONS VIRTUALIZATION (NFV), programmable services over white-box networking, computation, and storage.

The IP hourglass. The Internet has a "thin waist": many protocols in the physical (copper, radio, fiber), link (Ethernet, PPP, WiFi, Bluetooth, PDCP), transport (TCP, UDP), and application (HTTP, SMTP, RTP, QUIC, DASH) layers, but one network-layer protocol, IP, which must be implemented by every one of the billions of Internet-connected devices. In the Internet's middle age, the hourglass has grown "love handles" at the waist: middleboxes (NAT, caching, NFV, firewalls) operating inside the network.

Architectural principles. RFC 1958 observes: "Many members of the Internet community would argue that there is no architecture, but only a tradition, which was not written down for the first 25 years (or at least not by the IAB). However, in very general terms, the community believes that the goal is connectivity, the tool is the Internet Protocol, and the intelligence is end to end rather than hidden in the network." Its three cornerstone beliefs are simple connectivity, the IP protocol as the narrow waist, and intelligence and complexity at the network edge.

Some functionality, such as reliable data transfer or congestion control, can be implemented in the network or at its edge: reliable data transfer, for example, either end to end between the transport layers of the two hosts, or hop by hop in every router along the path. The END-END ARGUMENT (Saltzer, Reed, and Clark, 1981) favors the edge: "The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.) We call this line of reasoning against low-level function implementation the 'end-to-end argument.' "

Where the intelligence lives has shifted. In the 20th-century telephone network it sat in the network switches; in the Internet before about 2005, at the edge; since then, network devices have become programmable, while intelligence, computing, and massive application-level infrastructure sit at the edge. Forwarding tables (destination-based forwarding) and flow tables (generalized forwarding) are computed by the control plane.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 4 slides (9th edition).

Lecture 5: The Network Layer: The Control Plane

5.1 Introduction

The network layer has two functions. Forwarding, the data plane, moves packets from a router's input to the appropriate router output. Routing, the control plane, determines the route taken by packets from source to destination. There are two ways to structure the control plane.

- Per-router control (traditional): individual routing-algorithm components in each and every router interact in the control plane, and each router computes its own forwarding table, which maps values in an arriving packet's header to an output link.
- Logically centralized control (software-defined networking, SDN): a remote controller computes forwarding tables and installs them in the routers, through a control agent (CA) in each router.

5.2 Routing algorithms

The goal of a routing protocol is to determine "good" paths (equivalently, routes) from sending hosts to the receiving host through the network of routers. A path is the sequence of routers that packets traverse from a given initial source host to the final destination host; "good" means least "cost", "fastest", or "least congested". Routing is one of the top ten networking challenges.

Definition: The network is a graph $G = (N, E)$, where N is the set of routers and E the set of links. The link cost $c_{a,b}$ is the cost of the direct link connecting a and b, with $c_{a,b} = \infty$ if a and b are not neighbors. The network operator defines the costs: they could always be 1, or be inversely related to bandwidth, or inversely related to congestion.

In the graph of Figure 64a,

$$N = \{u, v, w, x, y, z\}, \quad E = \{(u,v), (u,w), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)\},$$

and, for example, $c_{w,z} = 5$ and $c_{u,z} = \infty$.

Routing algorithms are classified along two axes.

- *Global or decentralized information.* In a global algorithm all routers have the complete topology and link-cost information: link-state (LS) algorithms. In a decentralized algorithm, routers initially know only the link costs to their attached neighbors, and compute routes by an iterative process of computation and exchange of information with neighbors: distance-vector (DV) algorithms.

- *How fast routes change.* In STATIC routing, routes change slowly over time. In DYNAMIC routing, routes change more quickly, through periodic updates or in response to link-cost changes.

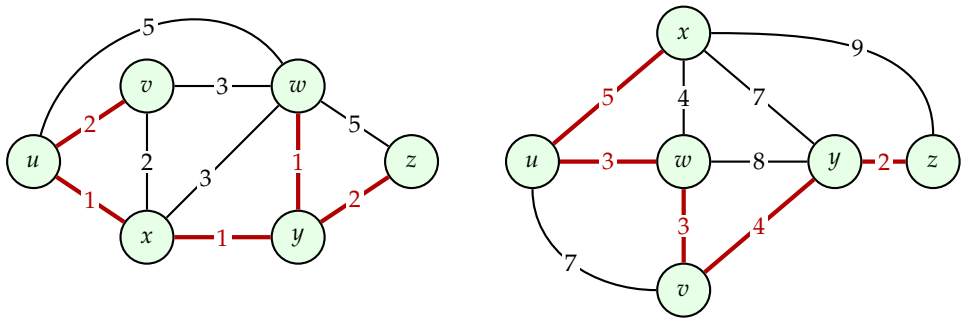


Figure 64: (a) The graph of the routing examples, with link costs; red links form the least-cost-path tree from u. (b) The graph of the second Dijkstra example, with its least-cost-path tree from u in red.

Dijkstra's link-state algorithm. The algorithm is centralized: the network topology and all link costs are known to all nodes, which is accomplished by a "link-state broadcast", so all nodes have the same information. It computes the least-cost paths from one node (the "source") to all other nodes, giving the forwarding table for that node. It is iterative: after k iterations the least-cost paths to k destinations are known. Notation: $D(v)$ is the current estimate of the cost of the least-cost path from the source to destination v; $p(v)$ is the predecessor of v along that path; N' is the set of nodes whose least-cost path is definitively known.

Algorithm 5.1: Dijkstra's link-state algorithm at source u

Input: Link costs $c_{a,b}$ of the whole network (∞ between non-neighbors).
Output: Least costs $D(v)$ and predecessors $p(v)$ for every node v.

```
1  N' ← {u}
2  for each node v do
3  |  if v adjacent to u then D(v) ← c_{u,v}, p(v) ← u else D(v) ← ∞   // only neighbors' costs known
4  repeat
5  |  find w ∉ N' such that D(w) is a minimum
6  |  add w to N'
7  |  for each v adjacent to w and not in N' do
8  |  |  if D(w) + c_{w,v} < D(v) then          // old path, or known path to w plus c_{w,v}
9  |  |  |  D(v) ← D(w) + c_{w,v}, p(v) ← w
10 until all nodes are in N'
```

Example: *Dijkstra on Figure 64a, source u.* Each row shows $D(\cdot), p(\cdot)$ after a step; a column ends when its node joins N'.

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	$2, u$	$5, u$	$1, u$	∞	∞
1	ux	$2, u$	$4, x$		$2, x$	∞
2	uxy	$2, u$	$3, y$			$4, y$
3	$uxyv$		$3, y$			$4, y$
4	$uxyvw$					$4, y$
5	$uxyvwz$					

In step 1, x joins and $D(v) = \min(2, 1+2) = 2$, $D(w) = \min(5, 1+3) = 4$, $D(y) = \min(\infty, 1+1) = 2$. In step 2, y joins (the tie with v is broken arbitrarily) and $D(w) = \min(4, 2+1) = 3$, $D(z) = \min(\infty, 2+2) = 4$. In step 3, v joins and $D(w) = \min(3, 2+3) = 3$; in step 4, w joins and $D(z) = \min(4, 3+5) = 4$. Tracing predecessors gives the tree of Figure 64a, and the forwarding table in u sends destination v on link (u, v) and every other destination (x, y, w, z) on link (u, x).

Example: *A second example* (Figure 64b, source u):

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	$7, u$	$3, u$	$5, u$	∞	∞
1	uw	$6, w$		$5, u$	$11, w$	∞
2	uwx	$6, w$			$11, w$	$14, x$
3	$uwxv$				$10, v$	$14, x$
4	$uwxvy$					$12, y$
5	$uwxvyz$					

The least-cost-path tree is constructed by tracing predecessor nodes, and ties can exist (they can be broken arbitrarily).

With n nodes, each of the n iterations must check all nodes w not in N', so the algorithm makes $n(n+1)/2$ comparisons: $O(n^2)$ complexity (more efficient implementations, $O(n \log n)$, are possible). For message complexity, each router must broadcast its link-state information to the other n routers. Efficient broadcast algorithms need $O(n)$ link crossings to disseminate a broadcast message from one source, so each router's message crosses $O(n)$ links, and the overall message complexity is $O(n^2)$.

Oscillations. When link costs depend on traffic volume, route oscillations are possible. Route to destination a in a ring of four nodes a, b, c, d (links ab, bc, cd, da), with traffic entering at b, c, d at rates $1, e < 1, 1$; link costs are directional and equal to the traffic carried. Initially b and d send directly to a and c sends through b, so $b \to a$ costs $1+e$ and $d \to a$ costs 1. Given these costs, b and c both switch to the paths through d: now $d \to a$ costs $2+e$, $c \to d$ costs $1+e$, $b \to c$ costs 1, and $b \to a$ costs 0. Given these costs, all traffic switches to the opposite direction ($d \to c \to b \to a$, with $b \to a$ costing $2+e$), and the next recomputation switches it back again, indefinitely.

Distance vector. The DV algorithm is based on the Bellman-Ford equation (dynamic programming). Let $D_x(y)$ be the cost of the least-cost path from x to y. Then

$$D_x(y) = \min_v \{\, c_{x,v} + D_v(y) \,\},$$

where the minimum is taken over all neighbors v of x, $c_{x,v}$ is the direct cost of the link from x to v, and $D_v(y)$ is v's least-cost-path cost to y.

Example: In Figure 64a, suppose u's neighbors know $D_v(z) = 5$, $D_x(z) = 3$, $D_w(z) = 3$. Then $D_u(z) = \min\{c_{u,v} + D_v(z),\ c_{u,x} + D_x(z),\ c_{u,w} + D_w(z)\} = \min\{2+5,\ 1+3,\ 5+3\} = 4$. The node achieving the minimum, x, is the next hop on the estimated least-cost path to z.

The key idea is that from time to time each node sends its own distance-vector estimate to its neighbors. When x receives a new DV estimate from any neighbor, it updates its own DV using the Bellman-Ford equation,

$$D_x(y) \leftarrow \min_v \{\, c_{x,v} + D_v(y) \,\} \quad \text{for each node } y \in N.$$

Under minor, natural conditions the estimates $D_x(y)$ converge to the actual least costs $d_x(y)$. The algorithm is *iterative and asynchronous*: each local iteration is caused by a local link-cost change or by a DV update message from a neighbor. It is *distributed and self-stopping*: each node notifies its neighbors only when its DV changes, the neighbors then notify their neighbors only if necessary, and if no notification is received, no actions are taken.

Algorithm 5.2: Distance-vector algorithm at node x

```
1 for each node y do D_x(y) ← c_{x,y}                    // D_x(x) = 0; ∞ for non-neighbors
2 send D_x to all neighbors
3 while true do
4     wait for a change in a local link cost or a DV message from a neighbor
5     for each node y do D_x(y) ← min_v{c_{x,v} + D_v(y)}        // latest DVs D_v of neighbors v
6     if D_x(y) changed for any destination y then send D_x to all neighbors
```

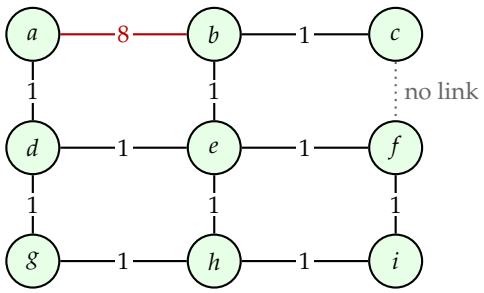


Figure 65: The network of the distance-vector example: all links cost 1 except a–b (cost 8), and there is no c–f link.

Example: *Synchronous DV iterations on Figure 65.* At $t = 0$ every node has distance estimates to its nearest neighbors only, for example $D_a(a) = 0$, $D_a(b) = 8$, $D_a(d) = 1$, and $D_a(y) = \infty$ otherwise, and

every node sends its DV to its neighbors. At each step $t = 1, 2, \ldots$ every node receives DVs from its neighbors, computes its new local DV, and sends it to its neighbors. At $t = 1$, b receives the DVs of a, c, e and computes

$$D_b(a) = \min\{c_{b,a} + D_a(a),\ c_{b,c} + D_c(a),\ c_{b,e} + D_e(a)\} = \min\{8, \infty, \infty\} = 8,$$
$$D_b(d) = \min\{8 + 1,\ 1 + \infty,\ 1 + 1\} = 2, \qquad D_b(f) = D_b(h) = \min\{\infty, \infty, 1 + 1\} = 2,$$

together with $D_b(c) = 1$, $D_b(e) = 1$, $D_b(g) = D_b(i) = \infty$. Node c receives only b's DV and computes $D_c(a) = 1 + 8 = 9$, $D_c(b) = 1$, $D_c(e) = 1 + 1 = 2$, and ∞ for d, f, g, h, i. Node e receives the DVs of b, d, f, h and obtains $D_e = (2, 1, 2, 1, 0, 1, 2, 1, 2)$ for destinations $a, \ldots, i$; for instance $D_e(a) = \min\{1 + 8,\ 1 + 1\} = 2$ through d.

Iterative communication and computation steps diffuse information through the network: c's state at $t = 0$ is at c only; at $t = 1$ it has propagated to b and may influence DV computations up to one hop away; at $t = 2$ it may influence computations two hops away (a and e), at $t = 3$ three hops away (d, f, h), and at $t = 4$ four hops away (g, i).

Example: *Three nodes.* Take links x–y of cost 2, y–z of cost 1, and x–z of cost 7. Initially $D_x = (0, 2, 7)$, $D_y = (2, 0, 1)$, $D_z = (7, 1, 0)$ (costs to x, y, z). After receiving its neighbors' DVs, x computes $D_x(y) = \min\{2 + 0,\ 7 + 1\} = 2$ and $D_x(z) = \min\{2 + 1,\ 7 + 0\} = 3$; z similarly computes $D_z(x) = \min\{7 + 0,\ 1 + 2\} = 3$, while D_y does not change. After one more exchange no DV changes: $D_x = (0, 2, 3)$, $D_y = (2, 0, 1)$, $D_z = (3, 1, 0)$.

Link-cost changes. A node detects a local link-cost change, updates its routing information and recalculates its local DV, and, if its DV changes, notifies its neighbors. Take links x–y of cost 4, y–z of cost 1, and x–z of cost 50, so that $D_y(x) = 4$ and $D_z(x) = 5$.

- *"Good news travels fast."* The cost of x–y drops to 1. At t_0, y detects the change, updates its DV, and informs its neighbors. At t_1, z receives the update, computes its new least cost to x, $\min\{50, 1 + 1\} = 2$, and sends its neighbors its DV. At t_2, y receives z's update; y's least costs do not change, so y sends no message to z.
- *"Bad news travels slow"* (the COUNT-TO-INFINITY PROBLEM). The cost of x–y rises to 60. Node y sees the direct link cost 60, but z has said it has a path at cost 5, so y computes a new cost 6 via z and notifies z. Then z learns that its path via y costs 6, computes 7 via y, and notifies y; y computes 8 via z; z computes 9 via y; and so on, two units per exchange, until z's cost through y would exceed its direct cost 50. Distributed algorithms are tricky.

Comparison of LS and DV. Robustness asks what happens if a router malfunctions or is compromised.

	Link state	Distance vector
Messages	n routers, $O(n^2)$ messages sent	exchange between neighbors only
Convergence	$O(n^2)$ algorithm, $O(n^2)$ messages; may have oscillations	convergence time varies; may have routing loops; count-to-infinity problem
Robustness	a router can advertise an incorrect link cost, but each router computes only its own table	a router can advertise an incorrect path cost ("I have a really low-cost path to everywhere"), BLACK-HOLING traffic; each router's DV is used by others, so errors propagate through the network

5.3 Intra-AS routing in the Internet: OSPF

The routing studied so far is idealized: all routers are identical and the network is "flat". In practice neither holds. *Scale*: with billions of destinations, routers cannot store all destinations in routing tables, and routing-table exchange would swamp the links. *Administrative autonomy*: the Internet is a network of networks, and each network's administrator may want to control routing in its own network.

Definition: The Internet aggregates routers into regions called AUTONOMOUS SYSTEMS (ASes), also known as domains. INTRA-AS (intra-domain) routing is routing among routers within the same AS: all routers in an AS must run the same intra-domain protocol, while routers in different ASes can run different ones. INTER-AS (inter-domain) routing is routing among ASes. A GATEWAY ROUTER, at the edge of its own AS, has links to routers in other ASes; gateways perform inter-domain routing as well as intra-domain routing.

A router's forwarding table is configured by both kinds of routing: intra-AS routing determines the entries for destinations within the AS, and inter-AS and intra-AS routing together determine the entries for external destinations. Suppose a router in AS1 receives a datagram destined outside AS1. It should forward the packet to a gateway router in AS1, but which one? AS1's inter-domain routing must learn which destinations are reachable through each neighboring AS (say AS2 and AS3) and propagate this reachability information to all routers in AS1.

The most common intra-AS routing protocols are:

- RIP (Routing Information Protocol, RFC 1723): classic DV, with DVs exchanged every 30 s; no longer widely used.
- EIGRP (Enhanced Interior Gateway Routing Protocol): DV based; Cisco-proprietary for decades, open since 2013 (RFC 7868).
- OSPF (Open Shortest Path First, RFC 2328): link-state routing. The IS-IS protocol (an ISO

standard, not an RFC standard) is essentially the same as OSPF.

OSPF is "open", meaning publicly available, and is classic link state: each router floods OSPF link-state advertisements to all other routers in the entire AS, directly over IP rather than over TCP or UDP. Multiple link-cost metrics are possible (bandwidth, delay). Each router has the full topology and uses Dijkstra's algorithm to compute its forwarding table. For security, all OSPF messages are authenticated, to prevent malicious intrusion.

Definition: Hierarchical OSPF uses a two-level hierarchy of local areas and a backbone. Link-state advertisements are flooded only within an area, or within the backbone. Each node has the detailed topology of its area and knows only the direction to other destinations. Area border routers "summarize" distances to destinations in their own area and advertise them in the backbone. Local (internal) routers flood link state within their area only, compute routing within the area, and forward packets to the outside via an area border router. Backbone routers run OSPF limited to the backbone, and boundary routers connect to other ASes.

5.4 Routing among the ISPs: BGP

Definition: The Border Gateway Protocol (BGP) is the de facto inter-domain routing protocol, the "glue that holds the Internet together". It allows a subnet to advertise its existence, and the destinations it can reach, to the rest of the Internet: "I am here, here is who I can reach, and how". BGP provides each AS a means to

- obtain destination-network reachability information from neighboring ASes (eBGP);
- determine routes to other networks based on reachability information and policy;
- propagate reachability information to all AS-internal routers (iBGP);
- advertise destination reachability information to neighboring networks.

Definition: In a BGP session, two BGP routers (peers) exchange BGP messages over a semi-permanent TCP connection, advertising paths to different destination network prefixes: BGP is a path-vector protocol. When AS3's gateway 3a advertises the path AS3,X to AS2's gateway 2c, AS3 promises AS2 that it will forward datagrams towards X.

BGP messages (RFC 4271) are exchanged between peers over the TCP connection: `OPEN` opens the TCP connection to the remote BGP peer and authenticates the sending peer; `UPDATE` advertises a new path (or withdraws an old one); `KEEPALIVE` keeps the connection alive in the absence of updates, and also acknowledges an `OPEN` request; `NOTIFICATION` reports errors in a previous message and is also used to close the connection.

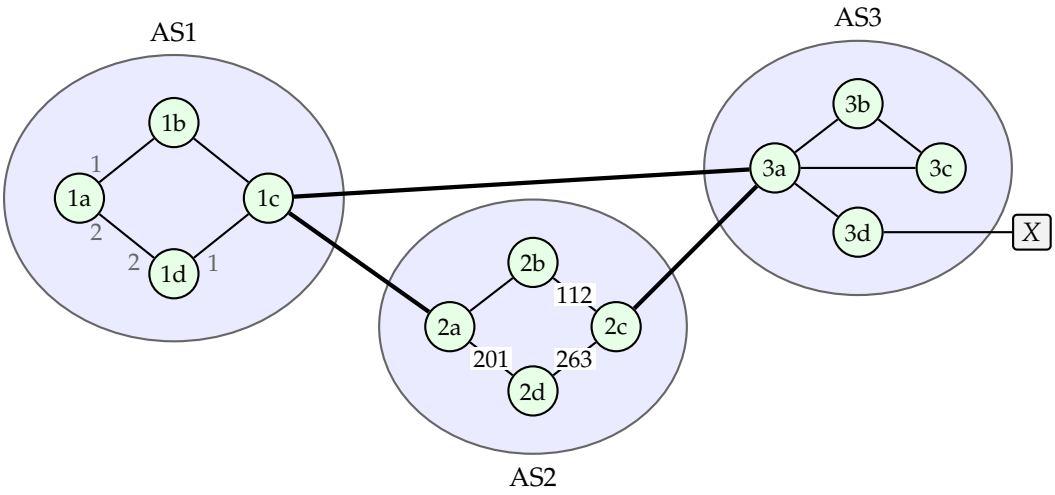


Figure 66: Three interconnected ASes. Thick links are the inter-AS links (1c–2a, 1c–3a, 2c–3a); the destination network X is reached through 3d. Numbers in AS2 are OSPF link weights; small grey numbers at 1a and 1d are interface numbers.

In Figure 66, eBGP sessions run over the inter-AS links between gateway routers, and iBGP sessions logically connect the routers inside each AS. Gateway routers (1c, 2a, 2c, 3a) run both eBGP and iBGP.

A BGP advertised route is a prefix (the destination being advertised) plus attributes. Two important attributes are AS-PATH, the list of ASes through which the prefix advertisement has passed, and NEXT-HOP, which indicates the specific internal-AS router to the next-hop AS. Routing is POLICY BASED: a gateway receiving a route advertisement uses its IMPORT POLICY to accept or decline the path (for example, never route through AS Y), and AS policy also determines whether to advertise a path to other neighboring ASes.

Path advertisement. In Figure 66, AS2 router 2c receives the path advertisement AS3,X (via eBGP) from AS3 router 3a. Based on AS2 policy, 2c accepts the path and propagates it (via iBGP) to all AS2 routers, and, based on AS2 policy, AS2 router 2a advertises (via eBGP) the path AS2,AS3,X to AS1 router 1c. A gateway router may learn about multiple paths to a destination: 1c learns AS2,AS3,X from 2a and AS3,X from 3a. Based on policy, 1c chooses the path AS3,X and advertises it within AS1 via iBGP.

Populating forwarding tables. Routers 1a, 1b, 1d learn via iBGP from 1c that the path to X goes through 1c. At 1d, OSPF intra-domain routing says that to get to 1c it should use interface 1, so 1d forwards traffic to X on interface 1. At 1a, OSPF says to use interface 2 to reach 1c, so X is reached on interface 2.

Hot potato routing. Router 2d learns via iBGP that it can route to X via 2a (path AS1,AS3,X) or via 2c (path AS3,X). HOT POTATO ROUTING chooses the local gateway with the least intra-domain cost, without worrying about inter-domain cost: 2d chooses 2a (OSPF weight 201 rather than 263), even though that path has more AS hops to X.

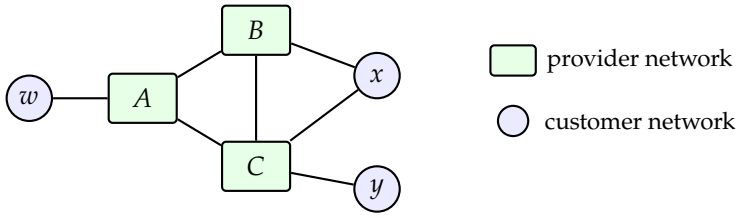


Figure 67: Provider networks A, B, C and customer networks w, x, y; x is dual-homed to B and C.

Achieving policy via advertisements. A typical real-world policy is that an ISP only wants to route traffic to and from its customer networks: it does not want to carry transit traffic between other ISPs. In Figure 67, A advertises the path Aw to B and to C. B chooses not to advertise BAw to C: B gets no "revenue" for routing $CBAw$, since none of C, A, w are B's customers. So C does not learn about the path $CBAw$, and C will route CAw (not using B) to get to w. Customers enforce policy the same way: x is DUAL-HOMED (attached to two networks), and if x does not want to carry traffic from B to C, it will not advertise to B a route to C.

BGP route selection. A router may learn about more than one route to a destination AS; it selects a route based on, in order:

1. the local preference value attribute (a policy decision);
2. the shortest AS-PATH;
3. the closest NEXT-HOP router (hot potato routing);
4. additional criteria.

Why different intra- and inter-AS routing? *Policy*: inter-AS, an administrator wants control over how its traffic is routed and who routes through its network; intra-AS, there is a single administrator, so policy is less of an issue. *Scale*: hierarchical routing saves table size and reduces update traffic. *Performance*: intra-AS routing can focus on performance, while inter-AS policy dominates over performance.

5.5 The SDN control plane

Historically the Internet network layer was implemented with a distributed, per-router control approach: a monolithic router contains switching hardware and runs a proprietary implementation

of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in a proprietary router operating system (such as Cisco IOS), and different "middleboxes" implement other network-layer functions (firewalls, load balancers, NAT boxes). Around 2005 interest renewed in rethinking the network control plane, and SOFTWARE-DEFINED NETWORKING (SDN) uses a remote controller that computes forwarding tables and installs them in routers.

A logically centralized control plane offers

- easier network management: it avoids router misconfigurations and gives greater flexibility of traffic flows;
- programmable routers through table-based forwarding (the OpenFlow API): centralized "programming", which computes tables centrally and distributes them, is easier than distributed "programming", in which tables result from a distributed algorithm (protocol) implemented in each and every router;
- an open (non-proprietary) implementation of the control plane, which fosters innovation.

The change resembles the mainframe-to-PC revolution: a vertically integrated, closed, proprietary industry (specialized hardware, operating system, and applications) with slow innovation became a horizontal one with open interfaces (a microprocessor, an open interface, several operating systems, many applications) and rapid innovation.

Traffic engineering with traditional routing. In Figure 64a, if the operator wants u-to-z traffic to flow along $uvwz$ rather than $uxyz$, it must redefine the link weights so that the routing algorithm computes routes accordingly (or use a new routing algorithm): link weights are the only control "knobs". If it wants to split u-to-z traffic along $uvwz$ and $uxyz$ (load balancing), it cannot (without a new routing algorithm). And if w wants to route two classes of traffic to z differently, it cannot with destination-based forwarding and LS or DV routing. Generalized forwarding and SDN can achieve any routing desired.

Definition: SDN has four defining features: (1) generalized "flow-based" forwarding (such as OpenFlow); (2) separation of the control and data planes; (3) control-plane functions external to the data-plane switches; (4) programmable control applications (routing, access control, load balancing, . . .).

The components of an SDN (Figure 68) are:

- *Data-plane switches*: fast, simple, commodity switches implementing generalized data-plane forwarding in hardware. The flow (forwarding) table is computed and installed under controller supervision. An API for table-based switch control (such as OpenFlow) defines what is controllable and what is not, along with a protocol for communicating with the controller (such as OpenFlow).
- *The SDN controller* (network operating system): maintains network state information; interacts

with network-control applications "above" via the NORTHBOUND API and with network switches "below" via the SOUTHBOUND API; and is implemented as a distributed system for performance, scalability, fault tolerance, and robustness. Internally it has a communication layer (between the controller and the controlled switches), network-wide state management (the state of links, switches, and services, kept in a distributed database), and an interface layer that offers abstractions and an API to network-control applications.

- *Network-control applications*: the "brains" of control, implementing control functions using the lower-level services and API provided by the controller. They are unbundled: a third party, distinct from the routing vendor or the SDN controller, can provide them.

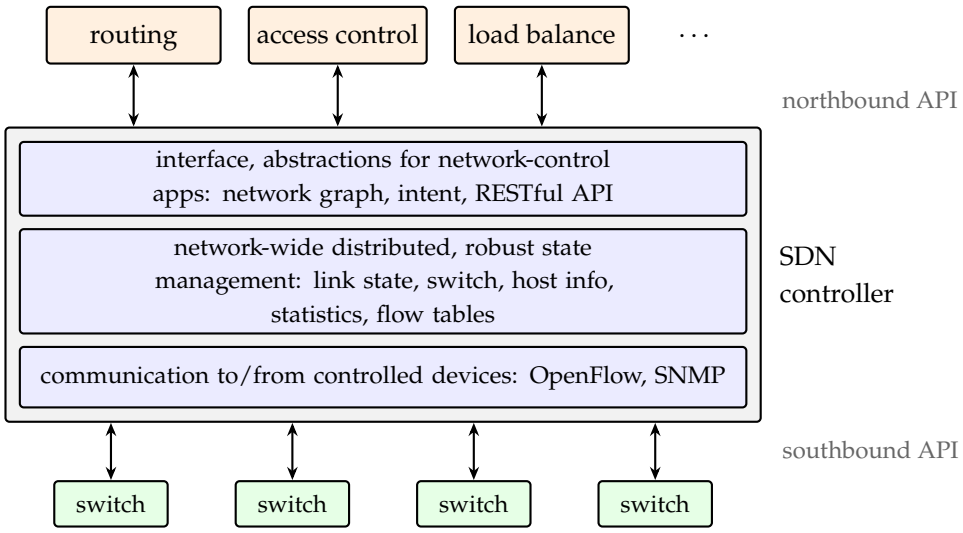


Figure 68: SDN architecture. Network-control applications use the northbound API of the SDN controller, whose three layers manage communication with the controlled switches (southbound API), network-wide state, and the abstractions offered to applications.

OpenFlow. The OpenFlow protocol operates between a controller and a switch, uses TCP to exchange messages (with optional encryption), and is distinct from the OpenFlow API used to specify generalized forwarding actions. It has three classes of messages: controller-to-switch, asynchronous (switch-to-controller), and symmetric (miscellaneous). Key controller-to-switch messages are `features` (the controller queries the switch's features and the switch replies), `configure` (the controller queries or sets switch configuration parameters), `modify-state` (add, delete, or modify flow entries in the OpenFlow tables), and `packet-out` (the controller sends this packet out of a specific switch port). Key switch-to-controller messages are `packet-in` (transfer a packet, and its control, to the controller), `flow-removed` (a flow-table entry was deleted at the switch), and `port status` (inform the controller of a change on a port). Network operators do not "program" switches by creating and sending OpenFlow messages directly; they use higher-level abstractions at the controller.

Example: *Control/data-plane interaction.* Switch s_1 experiences a link failure.

1. s_1 notifies the controller with an OpenFlow `port status` message.
2. The SDN controller receives the message and updates its link-status information.
3. Dijkstra's routing application, which previously registered to be called whenever link status changes, is called.
4. It accesses the network-graph and link-state information in the controller and computes new routes.
5. The link-state routing application interacts with the flow-table-computation component in the controller, which computes the new flow tables needed.
6. The controller uses OpenFlow to install the new tables in the switches that need updating.

Controllers in practice. ONOS is an early, influential open-source SDN controller. Its control applications (traffic engineering, firewalling, load balancing, . . .) are separate from the controller; its northbound abstractions include a REST API and an INTENT FRAMEWORK, a high-level specification of a service (*what* rather than *how*); its southbound protocols include OpenFlow, Netconf, and OVSDB; and it puts considerable emphasis on its distributed core (statistics, devices, hosts, links, paths, flow rules, topology) for service reliability, replication, and performance scaling. Google's ORION (NSDI 2021) is the SDN control plane for Google's datacenter (Jupiter) and wide-area (B4) networks, providing intradomain services within Google's network. Routing (intradomain, iBGP) and traffic engineering are applications on top of the Orion core; edge-to-edge flow-based controls (such as CoFlow scheduling) meet contract SLAs; management uses publish-subscribe distributed microservices in the Orion core, with OpenFlow for switch signaling and monitoring. As Amin Vahdat of Google puts it, the location and role of every element of such a network is known ahead of time, so roles are configured into a logically centralized controller instead of being learned by booting nodes from their neighbors: management shifts from configuring individual elements to configuring one logical network, simplifying both the technical problem and the cognitive load on network managers.

Challenges. SDN's control plane must be hardened into a dependable, reliable, performance-scalable, secure distributed system: robust to failures (leveraging the strong theory of reliable distributed systems), with dependability and security "baked in" from day one. Networks and protocols must meet mission-specific requirements (real-time, ultra-reliable, ultra-secure); SDN must scale beyond a single AS; and SDN is critical in 5G cellular networks. SDN-computed versus router-computed forwarding tables is just one example of logically-centralized computation versus protocol computation; one could imagine SDN-computed congestion control, in which the controller sets sender rates based on congestion levels reported by routers.

5.6 ICMP: the Internet Control Message Protocol

Definition: The Internet Control Message Protocol (ICMP) is used by hosts and routers to communicate network-level information: error reporting (unreachable host, network, port, protocol) and echo request/reply (used by `ping`). It sits at the network layer, "above" IP: ICMP messages are carried in IP datagrams. An ICMP message has a type and a code, plus the first 8 bytes of the IP datagram causing the error. Source quench (type 4) was meant for congestion control and is not used.

Type	Code	Description	Type	Code	Description
0	0	echo reply (ping)	3	7	dest. host unknown
3	0	dest. network unreachable	4	0	source quench (not used)
3	1	dest. host unreachable	8	0	echo request (ping)
3	2	dest. protocol unreachable	9	0	route advertisement
3	3	dest. port unreachable	10	0	router discovery
3	6	dest. network unknown	11	0	TTL expired
			12	0	bad IP header

Traceroute. The source sends sets of UDP segments (three probes per set) to the destination: the first set with TTL = 1, the second with TTL = 2, and so on. A datagram in the nth set arrives at the nth router, which discards it and sends the source an ICMP message (type 11, code 0), possibly including the router's name and IP address. When the ICMP message arrives, the source records the RTT. Eventually a UDP segment arrives at the destination host, which returns an ICMP "port unreachable" message (type 3, code 3), and the source stops.

5.7 Network management and SNMP, NETCONF/YANG

An autonomous system (a "network") has thousands of interacting hardware and software components; other complex systems, such as a jet airplane or a nuclear power plant, likewise require monitoring, configuration, and control.

Definition: Network management "includes the deployment, integration and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost."

Its components are the managing server (controller), an application typically with network managers (humans) in the loop; the managed devices, equipment with manageable, configurable hardware and software components, each running an agent; the *data*, meaning device "state": configuration data, operational data, and device statistics; and a network management protocol, used by the managing server to query, configure, and manage devices, and by devices to inform the managing server of data and events. Operators manage networks in three ways:

- *CLI* (command-line interface): the operator issues (types, scripts) commands directly to individual devices, for example via ssh.
- *SNMP/MIB*: the operator queries and sets device data (the MIB) using the Simple Network Management Protocol (SNMP).
- *NETCONF/YANG*: more abstract, network-wide, and holistic, with an emphasis on multi-device configuration management. YANG is a data modeling language, and NETCONF communicates YANG-compatible actions and data to, from, and among remote devices.

SNMP. MIB information and commands are conveyed in two ways: *request/response* mode (the managing server sends a request, the agent responds) and *trap* mode (the agent sends a trap message to the managing server).

Message type	Function
`GetRequest`, `GetNextRequest`, `GetBulkRequest`	manager-to-agent: "get me data" (data instance, next data in list, block of data)
`SetRequest`	manager-to-agent: set MIB value
`Response`	agent-to-manager: value, response to a request
`Trap`	agent-to-manager: inform manager of an exceptional event

Get/set messages (PDU types 0–3) carry a header with the PDU type, request ID, error status (0–5), and error index, followed by name/value pairs of the variables to get or set. Trap messages (PDU type 4) carry a header with the PDU type, enterprise, agent address, trap type (0–7), specific code, and time stamp, followed by name/value pairs of trap information.

Definition: A managed device's operational (and some configuration) data are gathered into a device MIB (Management Information Base) module; about 400 MIB modules are defined in RFCs, and there are many more vendor-specific MIBs. MIB data are defined in the Structure of Management Information (SMI), a data-definition language.

Example MIB variables for UDP:

Object ID	Name	Type	Comments
1.3.6.1.2.1.7.1	UDPInDatagrams	32-bit counter	total datagrams delivered
1.3.6.1.2.1.7.2	UDPNoPorts	32-bit counter	undeliverable: no application at port
1.3.6.1.2.1.7.3	UDPInErrors	32-bit counter	undeliverable: all other reasons
1.3.6.1.2.1.7.4	UDPOutDatagrams	32-bit counter	total datagrams sent
1.3.6.1.2.1.7.5	udpTable	SEQUENCE	one entry per port in use

NETCONF. NETCONF actively manages and configures devices network-wide, between the managing server and managed devices: it retrieves, sets, modifies, and activates configurations, commits actions atomically over multiple devices, queries operational data and statistics, and

subscribes to notifications from devices. It follows a remote procedure call (RPC) paradigm: protocol messages are encoded in XML and exchanged over a secure, reliable transport protocol (such as TLS). A session begins with <hello> (session initiation and capabilities exchange), continues with <rpc>/<rpc-reply> exchanges and asynchronous <notification> messages, and ends with <close-session>.

The following RPC message (message id 101) changes the running configuration, setting the MTU of interface Ethernet0/0 to 1500:

```
<?xml version="1.0" encoding="UTF-8"?>
<rpc message-id="101"
  xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target><running/></target>
    <config>
      <top xmlns="http://example.com/schema/1.2/config">
        <interface>
          <name>Ethernet0/0</name>
          <mtu>1500</mtu>
        </interface>
      </top>
    </config>
  </edit-config>
</rpc>
```

Selected NETCONF operations:

Operation	Description
<get-config>	retrieve all or part of a given configuration (a device may have several)
<get>	retrieve all or part of both configuration state and operational state data
<edit-config>	change a specified (possibly running) configuration; the device's <rpc-reply> contains <ok> or <rpc-error> with rollback
<lock>, <unlock>	lock (unlock) the configuration datastore at the managed device, to lock out NETCONF, SNMP, or CLI commands from other sources
<create-subscription>	enable event-notification subscription (<notification> messages) from the device

Definition: YANG is a data modeling language that specifies the structure, syntax, and semantics of NETCONF management data, with built-in data types like SMI. XML describing a device and its capabilities can be generated from a YANG description and sent inside NETCONF RPC messages such as <edit-config>, and YANG can express constraints among data that a valid configuration must satisfy (correctness and consistency).

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 5 slides (9th edition).

Lecture 6: The Link Layer and LANs

6.1 Introduction to the link layer

Hosts and routers are NODES; the communication channels that connect adjacent nodes along a communication path are LINKS (wired, wireless, LANs). A layer-2 packet is a FRAME, which encapsulates a datagram. The link layer has the responsibility of transferring a datagram from one node to a physically adjacent node over a link.

A datagram is transferred by different link protocols over different links, for example WiFi on the first link and Ethernet on the next, and each link protocol provides different services (it may or may not provide reliable data transfer over the link). In a transportation analogy, a trip from Princeton to Lausanne goes by limo to JFK, by plane to Geneva, and by train to Lausanne: the tourist is a datagram, each transport segment is a communication link, the transportation mode is a link-layer protocol, and the travel agent is the routing algorithm.

Link-layer services:

- *Framing and link access*: encapsulate a datagram into a frame, adding a header and trailer, with channel access if the medium is shared. MAC ADDRESSES in frame headers identify the source and destination (they are different from IP addresses).
- *Reliable delivery between adjacent nodes*, by the same techniques as reliable transport. It is seldom used on low bit-error links, but wireless links have high error rates. (Why have both link-level and end-to-end reliability?)
- *Flow control*: pacing between adjacent sending and receiving nodes.
- *Error detection*: errors are caused by signal attenuation and noise; the receiver detects errors and signals retransmission or drops the frame. *Error correction*: the receiver identifies and corrects bit errors without retransmission.
- *Half duplex and full duplex*: with half duplex, nodes at both ends of a link can transmit, but not at the same time.

The link layer is implemented in each and every host, on a chip or in a NETWORK INTERFACE CARD (NIC) that implements the link and physical layers and attaches to the host's system buses (for example PCI): a combination of hardware, software, and firmware. The sending side encapsulates the datagram in a frame and adds error-checking bits, reliable data transfer, flow control, and so on; the receiving side looks for errors and performs reliable data transfer and flow control, then extracts the datagram and passes it to the upper layer.

6.2 Error-detection and -correction techniques

The sender protects data D (which may include header fields) with ERROR DETECTION AND CORRECTION BITS (EDC, redundancy), and sends D and EDC over a bit-error-prone link. The receiver checks the received D' and EDC': if all bits are OK it passes the datagram up, and otherwise it has detected an error. Error detection is not 100% reliable: a protocol may miss some errors, but rarely, and a larger EDC field yields better detection and correction.

Parity checking. With SINGLE-BIT PARITY, the sender sets the parity bit so that there is an even (or, for odd parity, odd) number of 1s; the receiver computes the parity of the d received bits and compares it with the received parity bit, detecting any single-bit error. For example, with even parity the data 0111000110101011 (nine 1s) gets parity bit 1. TWO-DIMENSIONAL PARITY arranges the data bits $d_{i,j}$ in rows and adds a parity bit to every row and every column (plus a parity bit over the parity bits). A single-bit error then causes a parity error in exactly one row and one column, which locates the bit, so it can be both detected and corrected without retransmission:

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

no errors (even parity)

1	0	1	0	1	1	
1	**0**	1	1	0	0	← parity error
0	1	1	1	0	1	
0	0	1	0	1	0	
	↑					

parity error

correctable single-bit error

Internet checksum. The goal is to detect flipped bits in a transmitted segment. The sender treats the contents of a UDP segment (including UDP header fields and IP addresses) as a sequence of 16-bit integers, computes their one's complement sum, and puts it in the UDP checksum field. The receiver computes the checksum of the received segment and compares it with the checksum field: if they differ, an error is detected; if they are equal, no error is detected (but there may be errors nonetheless).

Cyclic redundancy check. A CYCLIC REDUNDANCY CHECK (CRC) is a more powerful error-detection code, widely used in practice (Ethernet, 802.11 WiFi). Think of the d data bits D as a binary number, and let G be an $(r+1)$-bit pattern, the GENERATOR, specified in the CRC standard. The sender computes r CRC bits R such that the $(d+r)$-bit pattern

$$\langle D, R \rangle = D \cdot 2^r \text{ XOR } R$$

is exactly divisible by G (modulo 2). The receiver knows G and divides $\langle D, R \rangle$ by G: a nonzero remainder means an error was detected. A CRC detects all burst errors of fewer than $r+1$ bits. To find R, the sender wants $D \cdot 2^r$ XOR $R = nG$, or equivalently (XOR-ing both sides with R)

$D \cdot 2^r = nG$ XOR R: if we divide $D \cdot 2^r$ by G, the remainder is R, so

$$R = \text{remainder}\left[\frac{D \cdot 2^r}{G}\right].$$

Example: Take $D = 101110$, $G = 1001$, $r = 3$. Dividing $D \cdot 2^3 = 101110000$ by G with modulo-2 (XOR) subtraction gives quotient 101011 and remainder $R = 011$:

$$\begin{array}{r} \\ 1001 \,)\, \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \begin{array}{l} 1\,0\,1\,0\,1\,1 \\ \hline 1\,0\,1\,1\,1\,0\,0\,0\,0 \\ 1\,0\,0\,1 \\ \hline 0\,1\,0\,1 \\ 0\,0\,0\,0 \\ \hline 1\,0\,1\,0 \\ 1\,0\,0\,1 \\ \hline 0\,1\,1\,0 \\ 0\,0\,0\,0 \\ \hline 1\,1\,0\,0 \\ 1\,0\,0\,1 \\ \hline 1\,0\,1\,0 \\ 1\,0\,0\,1 \\ \hline 0\,1\,1 \end{array}$$

The sender transmits 101110 011, which is divisible by 1001.

6.3 Multiple access links and protocols

There are two types of links: POINT-TO-POINT (a link between an Ethernet switch and a host, PPP for dial-up access) and BROADCAST (a shared wire or medium: old-school Ethernet, upstream HFC in a cable access network, 802.11 wireless LANs, 4G/5G, satellite; humans at a cocktail party share air acoustically). On a single shared broadcast channel, two or more simultaneous transmissions by nodes interfere: a COLLISION occurs if a node receives two or more signals at the same time.

Definition: A MULTIPLE ACCESS PROTOCOL is a distributed algorithm that determines how nodes share a channel, that is, when a node can transmit. Communication about channel sharing must use the channel itself: there is no out-of-band channel for coordination.

Given a multiple access channel of rate R bps, an ideal protocol would satisfy: (1) when one node wants to transmit, it can send at rate R; (2) when M nodes want to transmit, each can send at average rate R/M; (3) it is fully decentralized, with no special node to coordinate transmissions and no synchronization of clocks or slots; (4) it is simple. MAC protocols fall into three broad classes:

- *channel partitioning*: divide the channel into smaller "pieces" (time slots, frequency, code) and allocate a piece to a node for exclusive use;

- *random access*: the channel is not divided, collisions are allowed, and nodes "recover" from collisions;
- *taking turns*: nodes take turns, but nodes with more to send can take longer turns.

Channel partitioning. In TDMA (time division multiple access), access to the channel is in "rounds": each station gets a fixed-length slot (length = packet transmission time) in each round, and unused slots go idle. In FDMA (frequency division multiple access), the channel spectrum is divided into frequency bands, each station is assigned a fixed band, and unused transmission time in a band goes idle. For example, on a six-station LAN where stations 1, 3, 4 have packets to send, slots (or bands) 2, 5, 6 are idle.

Random access. When a node has a packet to send, it transmits at the full channel data rate R, with no a priori coordination among nodes; two or more transmitting nodes cause a collision. A random access protocol specifies how to detect collisions and how to recover from them (for example, via delayed retransmissions). Examples are ALOHA, slotted ALOHA, CSMA, CSMA/CD, and CSMA/CA.

Definition: Slotted ALOHA assumes that all frames have the same size, time is divided into equal slots (the time to transmit one frame), nodes start to transmit only at slot beginnings, nodes are synchronized, and if two or more nodes transmit in a slot, all nodes detect the collision. When a node obtains a fresh frame, it transmits in the next slot. If there is no collision, the node can send a new frame in the next slot; if there is a collision, the node retransmits the frame in each subsequent slot with probability p until success.

Randomization keeps colliding nodes from colliding again forever. With three nodes, for example, nine slots might go collision, empty, collision, success, empty, collision, empty, success, success. Slotted ALOHA's pros: a single active node can continuously transmit at the full rate of the channel; it is highly decentralized (only the slots in nodes need to be in sync); and it is simple. Its cons: collisions waste slots; slots can be idle; nodes may be able to detect a collision in less than the time to transmit a packet; and clocks must be synchronized.

Efficiency. Efficiency is the long-run fraction of successful slots when there are many nodes, all with many frames to send. Suppose N nodes each transmit in a slot with probability p. A given node succeeds in a slot with probability $p(1-p)^{N-1}$, so some node succeeds with probability

$$E(p) = Np(1-p)^{N-1}.$$

Since $E'(p) = N(1-p)^{N-2}\big[(1-p) - (N-1)p\big]$, the maximizing probability is $p^* = 1/N$, giving $E(p^*) = (1-1/N)^{N-1}$, which tends to $1/e \approx 0.37$ as $N \to \infty$. At best, the channel is used for useful transmissions 37% of the time.

In PURE (UNSLOTTED) ALOHA, a node transmits a frame as soon as it arrives, without synchronization. A frame sent at t_0 (frames take one unit of time) collides with any other frame sent in $[t_0 - 1, t_0 + 1]$, so

$$P(\text{success by a given node}) = p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1} = p(1-p)^{2(N-1)},$$

where the factors are the probabilities that the node transmits, that no other node transmits in $[t_0 - 1, t_0]$, and that no other node transmits in $[t_0, t_0 + 1]$. Choosing the optimal p (here $p^* = 1/(2N-1)$) and letting $N \to \infty$, the efficiency $Np(1-p)^{2(N-1)}$ tends to $1/(2e) \approx 0.18$: even worse than slotted ALOHA.

CSMA. Simple CSMA (carrier sense multiple access) listens before transmitting: if the channel is sensed idle, the node transmits its entire frame; if it is sensed busy, the node defers transmission (human analogy: don't interrupt others). Collisions can still occur with carrier sensing, because propagation delay means two nodes may not hear each other's just-started transmission; the entire packet transmission time is then wasted. Distance and propagation delay play a role in determining the collision probability. CSMA/CD (CSMA with collision detection) detects collisions within a short time and aborts the colliding transmissions, reducing the time wasted in collisions (human analogy: the polite conversationalist). Collision detection is easy in wired LANs and difficult with wireless.

Algorithm 6.1: Ethernet CSMA/CD

```
1  the NIC receives a datagram from the network layer and creates a frame
2  m ← 0                                                  // number of collisions so far
3  while true do
4      if channel busy then wait until the channel is idle
5      start frame transmission
6      if entire frame transmitted without detecting another transmission then return (done)
7      abort, and send a jam signal
8      m ← m + 1
9      choose K at random from {0, 1, 2, ..., 2^m − 1}          // binary (exponential) backoff
10     wait K · 512 bit times
```

More collisions thus give a longer backoff interval.

Taking turns. Channel partitioning shares the channel efficiently and fairly at high load, but is inefficient at low load: there is delay in channel access, and a node gets $1/N$ of the bandwidth even if it is the only active node. Random access is efficient at low load (a single node can fully use the channel), but at high load there is collision overhead. "Taking turns" protocols look for the best of both.

- POLLING: a centralized controller "invites" the other nodes to transmit in turn; it is typically used with "dumb" devices, and Bluetooth uses polling. Concerns are polling overhead, latency, and a single point of failure (the controller).

- TOKEN PASSING: a control token message is explicitly passed from one node to the next, sequentially, and a node transmits while holding the token. Concerns are token overhead, latency, and a single point of failure (the token).

Cable access networks. A cable network uses FDM, TDM, and random access together. The cable headend's CMTS (cable modem termination system) transmits Internet frames, TV channels, and control downstream on multiple broadcast FDM channels (up to 1.6 Gbps per channel); there are multiple upstream channels (up to 1 Gbps per channel). In DOCSIS (Data Over Cable Service Interface Specification), upstream time is divided into minislots: a downstream MAP frame assigns upstream minislots to cable modems for their data frames, while requests for upstream slots (and data) are sent by random access, with binary backoff, in contention minislots.

In summary: channel partitioning divides by time, frequency, or code; random access protocols include ALOHA, slotted ALOHA, CSMA, and CSMA/CD (carrier sensing is easy in some technologies, such as wire, and hard in others, such as wireless), with CSMA/CD used in Ethernet and CSMA/CA in 802.11; taking-turns protocols (polling from a central site, token passing) are used in Bluetooth, FDDI, and token ring.

6.4 Switched local area networks

MAC addresses. A 32-bit IP address (for example 128.119.40.136) is a network-layer address for an interface, used for layer-3 forwarding. A MAC ADDRESS (also called LAN, physical, or Ethernet address) is used "locally" to get a frame from one interface to another physically connected interface (in the same subnet, in the IP addressing sense). It is 48 bits for most LANs, burned into the NIC ROM and sometimes software settable, and written in hexadecimal, for example 1A-2F-BB-76-09-AD. Each interface on a LAN has a unique 48-bit MAC address and a locally unique 32-bit IP address. MAC address allocation is administered by IEEE: a manufacturer buys a portion of the MAC address space, to assure uniqueness. A MAC address is like a Social Security Number, an IP address like a postal address. The flat MAC address is portable: an interface can move from one LAN to another, while an IP address depends on the IP subnet to which the node is attached.

Definition: ARP (address resolution protocol) determines an interface's MAC address from its IP address. Each IP node (host or router) on a LAN has an ARP TABLE of IP/MAC address mappings for some LAN nodes, with entries ⟨IP address; MAC address; TTL⟩, where the TTL is the time after which the mapping will be forgotten (typically 20 minutes).

Example: *ARP in action.* Host A (IP 137.196.7.23, MAC 71-65-F7-2B-08-53) wants to send a datagram to B (IP 137.196.7.14) on the same LAN, but B's MAC address is not in A's ARP table.

1. A broadcasts an ARP query containing B's IP address, in an Ethernet frame with destination MAC address FF-FF-FF-FF-FF-FF (source MAC 71-65-F7-2B-08-53, source IP 137.196.7.23, target IP 137.196.7.14). All nodes on the LAN receive it.

2. B replies to A with an ARP response giving its MAC address, 58-23-D7-FA-20-B0, in a frame sent to 71-65-F7-2B-08-53.

3. A receives B's reply and adds the entry ⟨137.196.7.14; 58-23-D7-FA-20-B0; TTL⟩ to its local ARP table.

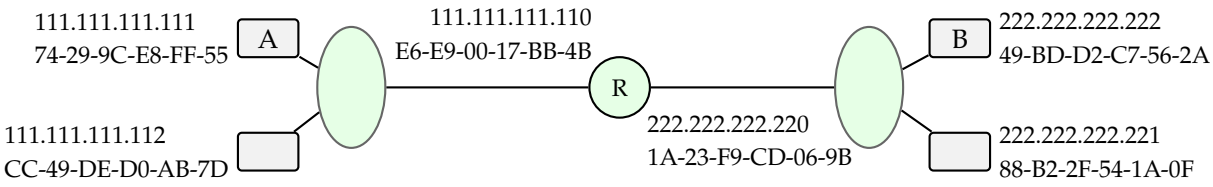


Figure 69: Two subnets joined by router R, with the IP and MAC address of every interface.

Example: *Routing to another subnet* (Figure 69). A sends a datagram to B via R, assuming A knows B's IP address, the IP address of its first-hop router R, and R's MAC address. A creates an IP datagram with source A (111.111.111.111) and destination B (222.222.222.222), and a link-layer frame containing it, with R's MAC address as the frame's destination: MAC source 74-29-9C-E8-FF-55, MAC destination E6-E9-00-17-BB-4B. R receives the frame, removes the datagram, and passes it up to IP. R determines the outgoing interface and passes the datagram, still with IP source A and destination B, to the link layer, which creates a frame with B's MAC address as its destination (MAC source 1A-23-F9-CD-06-9B, MAC destination 49-BD-D2-C7-56-2A) and transmits it. B receives the frame, extracts the IP datagram with destination B, and passes it up the protocol stack to IP. The IP addresses stay fixed end to end, while the MAC addresses change on every link.

Ethernet. Ethernet is the "dominant" wired LAN technology: the first widely used LAN technology, simpler and cheap, and it has kept up with the speed race, from 10 Mbps to 400 Gbps (a single chip can support multiple speeds). Bob Metcalfe, Ethernet's co-inventor, received the 2022 ACM Turing Award. In a BUS topology (popular through the mid 1990s, over coaxial cable), all nodes are in the same collision domain and can collide with each other. The SWITCHED topology now prevails: an active layer-2 switch is in the center, and each "spoke" runs a separate Ethernet protocol, so nodes do not collide with each other.

The sending interface encapsulates an IP datagram (or another network-layer packet) in an Ethernet frame with the fields of Figure 71 (top).

- *Preamble*: 7 bytes of 10101010 followed by one byte of 10101011, used to synchronize the receiver and sender clock rates.
- *Addresses*: 6-byte source and destination MAC addresses. If an adapter receives a frame with a matching destination address, or with the broadcast address (for example an ARP packet), it passes the data in the frame to the network-layer protocol; otherwise it discards the frame.
- *Type*: indicates the higher-layer protocol (mostly IP, but others are possible, such as Novell IPX and AppleTalk), used to demultiplex at the receiver.

- *CRC*: checked at the receiver; if an error is detected, the frame is dropped.

Ethernet is *connectionless* (no handshaking between the sending and receiving NICs) and *unreliable* (the receiving NIC sends no ACKs or NAKs to the sending NIC): data in dropped frames is recovered only if the initial sender uses a higher-layer reliable data transfer protocol (such as TCP); otherwise the dropped data is lost. Ethernet's MAC protocol is unslotted CSMA/CD with binary backoff. There are many different 802.3 Ethernet standards, with a common MAC protocol and frame format, different speeds, and different physical-layer media (copper twisted pair, fiber); the gigabit physical layers, for example, include 1000BASE-T, 1000BASE-CX, 1000BASE-SX, 1000BASE-LX, 1000BASE-LX10, and 1000BASE-BX10.

Switches. A SWITCH is a link-layer device that takes an active role: it stores and forwards Ethernet (or other) frames, examining an incoming frame's MAC address and selectively forwarding the frame to one or more outgoing links; when a frame is to be forwarded on a segment, it uses CSMA/CD to access the segment. Switches are *transparent* (hosts are unaware of them) and *plug-and-play, self-learning* (they need not be configured). Hosts have dedicated, direct connections to the switch, which buffers packets, and the Ethernet protocol is used on each incoming link, so there are no collisions and links are full duplex: each link is its own collision domain. On a switch with six interfaces, A-to-A′ and B-to-B′ can transmit simultaneously without collisions, but A-to-A′ and C-to-A′ cannot happen simultaneously.

Definition: Each switch has a SWITCH TABLE whose entries are (MAC address of host, interface to reach host, time stamp), much like a routing table. The table is built by SELF-LEARNING: when a frame is received, the switch learns the location of the sender (the incoming LAN segment) and records the sender/location pair in its table.

Algorithm 6.2: Switch frame filtering and forwarding

```
record the incoming link and the MAC address of the sending host
index the switch table using the destination MAC address
if an entry is found for the destination then
    if the destination is on the segment from which the frame arrived then drop the frame else
      forward the frame on the interface in the entry
else
    flood                                   // forward on all interfaces except the arriving one
```

Example: On a six-interface switch with an initially empty table, A (on interface 1) sends a frame to A′ (on interface 4). The switch records $(A, 1, 60)$; the location of A′ is unknown, so it floods the frame. When A′ replies to A, the switch records $(A', 4, 60)$, and since A's location is known, it selectively sends the frame on just one link, interface 1.

Self-learning switches can be connected together, and self-learning works exactly as in the single-switch case. In Figure 70, suppose C sends a frame to I, and I responds to C. S_1 learns that C is on its C link and floods the frame (to A, B, and S_4); S_4 learns that C is reached via S_1 and floods to S_2 and S_3; S_2 and S_3 learn that C is reached via S_4 and flood to their hosts, so I receives the frame. I's

reply reaches S_3, which learns I's location and, knowing C's, forwards it only to S_4; S_4 learns that I is via S_3 and forwards to S_1; S_1 learns that I is via S_4 and delivers the frame to C. S_2 does not learn I's location.

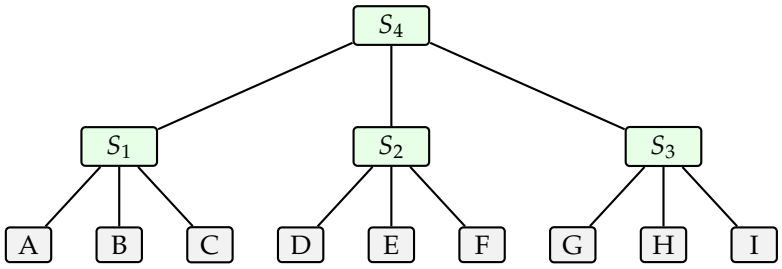


Figure 70: Four interconnected self-learning switches.

A campus network shows the scale: the UMass network has 4 firewalls, 10 routers, more than 2000 network switches, 6000 wireless access points, 30000 active wired network jacks, and 55000 active end-user wireless devices, all built, operated, and maintained by about 15 people. Its hierarchy runs from two border routers (eBGP, inter-domain routing; 10G links off campus, 100G pending) to two core routers (iBGP and IS-IS; 40G and 100G), aggregation routers for buildings, WiFi, the firewall, and the data center (IS-IS, intra-domain routing; 40G), and layer-2 Ethernet switches in building closets and the data center (10G and 1G).

Switches versus routers. Both are store-and-forward: routers are network-layer devices that examine network-layer headers, and switches are link-layer devices that examine link-layer headers. Both have forwarding tables: routers compute them using routing algorithms and IP addresses, while switches learn them using flooding, learning, and MAC addresses.

Virtual LANs. As LANs scale and users change their point of attachment, a single broadcast domain causes problems. *Scaling*: all layer-2 broadcast traffic (ARP, DHCP, unknown MAC addresses) must cross the entire LAN, with efficiency, security, and privacy issues. *Administration*: a CS user who moves to an office in EE is physically attached to the EE switch but wants to remain logically attached to the CS switch.

Definition: Switches supporting VIRTUAL LOCAL AREA NETWORK (VLAN) capabilities can be configured to define multiple virtual LANs over a single physical LAN infrastructure. In a PORT-BASED VLAN, switch ports are grouped (by switch management software) so that a single physical switch operates as multiple virtual switches, for example EE on ports 1–8 and CS on ports 9–15.

VLANs give *traffic isolation*: frames to and from ports 1–8 can only reach ports 1–8. A VLAN can also be defined by the MAC addresses of endpoints rather than by switch port, and membership is dynamic: ports can be dynamically assigned among VLANs. Forwarding between VLANs is done via routing, just as with separate switches; in practice vendors sell combined switches and routers. A VLAN can span multiple switches: a TRUNK PORT carries frames between VLANs defined over

multiple physical switches (for example, ports 2, 3, 5 of a second switch belong to the EE VLAN and ports 4, 6, 7, 8 to the CS VLAN). Frames forwarded within a VLAN between switches cannot be vanilla 802.1 frames, since they must carry VLAN ID information: the 802.1Q protocol adds and removes additional header fields for frames forwarded between trunk ports (Figure 71).

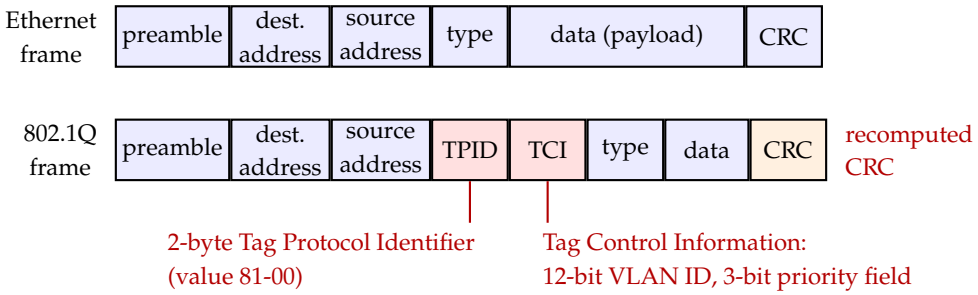


Figure 71: An Ethernet frame and an 802.1Q frame, which adds a tag after the source address and recomputes the CRC.

The 802.1Q tag consists of a 2-byte Tag Protocol Identifier (value 81-00) and Tag Control Information, which contains a 12-bit VLAN ID field and a 3-bit priority field (like the IP TOS field); the CRC is recomputed.

Definition: EVPN (Ethernet VPN, also VXLAN) logically connects layer-2 Ethernet switches to each other using IP as an underlay: Ethernet frames are carried within IP datagrams between sites, a "tunneling scheme to overlay Layer 2 networks on top of Layer 3 networks" that "runs over the existing networking infrastructure and provides a means to 'stretch' a Layer 2 network" (RFC 7348).

For example, a VLAN can span a Sunnyvale data center and a Bangalore data center. An Ethernet frame from host A in Sunnyvale to host B in Bangalore reaches the VXLAN tunnel endpoint (VTEP) x in Sunnyvale, which encapsulates it in a UDP segment inside an IP datagram from x to VTEP y in Bangalore; the datagram crosses the Internet, and y decapsulates the Ethernet frame and delivers it to B.

6.5 Link virtualization: a network as a link layer

Definition: MULTIPROTOCOL LABEL SWITCHING (MPLS) aims at high-speed IP forwarding among a network of MPLS-capable routers, using a fixed-length LABEL (instead of longest prefix matching) for faster lookup. It borrows ideas from the virtual-circuit approach, but the IP datagram still keeps its IP addresses. The MPLS header sits between the Ethernet header and the rest of the frame (which includes the IP header), with fields label (20 bits), Exp (3 bits), S (1 bit), and TTL (8 bits).

MPLS-capable routers, also called LABEL-SWITCHED ROUTERS, forward packets to an outgoing interface based only on the label value, without inspecting the IP address; the MPLS forwarding table is distinct from the IP forwarding table. This gives flexibility: MPLS forwarding decisions can differ from those of IP. Using destination *and* source addresses, flows to the same destination can be

routed differently (traffic engineering), and flows can be re-routed quickly if a link fails, using precomputed backup paths. In IP routing, the path to a destination is determined by the destination address alone; in MPLS routing, an IP/MPLS entry router can use different MPLS routes to the same destination based on, for example, the IP source address or other fields: a flavor of generalized forwarding, ten years before it. For *signaling*, the OSPF and IS-IS link-state flooding protocols are modified to carry information used by MPLS routing (for example link bandwidth and the amount of "reserved" link bandwidth), and the entry MPLS router uses the RSVP-TE signaling protocol to set up MPLS forwarding at downstream routers.

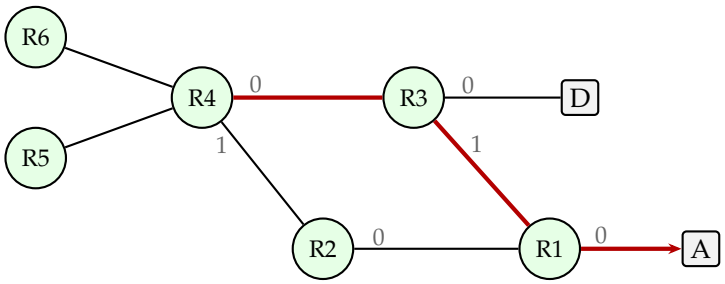


Figure 72: An MPLS network: numbers next to routers are interface numbers, and the red path is the label-switched path R4 → R3 → R1 → A.

Example: The MPLS forwarding tables of Figure 72:

Router	In label	Out label	Destination	Out interface
R4		10	A	0
		12	D	0
		8	A	1
R3	10	6	A	1
	12	9	D	0
R2	8	6	A	0
R1	6	–	A	0

Entry router R4 labels a packet for A with 10 and sends it on interface 0 to R3, which swaps label 10 for 6 and forwards on interface 1 to R1, which removes the label and delivers the packet to A. R4 can instead label a packet for A with 8 and send it on interface 1, via R2, which swaps 8 for 6: two routes to the same destination.

6.6 Data center networking

Data center networks have tens to hundreds of thousands of hosts, often closely coupled, in close proximity: e-business (Amazon), content servers (YouTube, Akamai, Apple, Microsoft), search engines and data mining (Google). The challenges are multiple applications, each serving massive

numbers of clients; reliability; and managing and balancing load, avoiding processing, networking, and data bottlenecks.

The network elements are:

- *server racks* of 20–40 server blades (hosts);
- *top of rack* (TOR, access) switches, one per rack, with 100G–400G Ethernet to the blades;
- *tier-2* (aggregation) switches, each connecting to about 16 TORs below;
- *tier-1* (core) switches, each connecting to about 16 tier-2 switches below;
- *border routers*, for connections outside the data center.

A two-layer LEAF/SPINE interconnection network connects every leaf switch to the spine switches. In Facebook's F16 data center topology, each top-of-rack switch connects to 16 fabric switches, and each fabric switch connects to 16 spine switches. Such rich interconnection among switches and racks gives *multipath* routing: increased throughput between racks (multiple routing paths are possible, for example two disjoint paths between racks 1 and 11) and increased reliability via redundancy. A LOAD BALANCER performs application-layer routing: it receives external client requests, directs the workload within the data center, and returns the results to the external client, hiding the data center's internals from the client.

Protocol innovations in data centers include, at the link layer, RoCE (remote DMA over Converged Ethernet); at the transport layer, ECN (explicit congestion notification) used in congestion control (DCTCP, DCQCN), and experiments with hop-by-hop (backpressure) congestion control; and for routing and management, SDN, widely used within and among an organization's data centers, and placing related services and data as close as possible (in the same or a nearby rack) to minimize tier-2 and tier-1 communication. Google's Orion SDN control plane manages its internal data center (Jupiter) and wide-area (B4) networks, with routing and traffic engineering as applications on the Orion core: there are no routing protocols, and congestion control is also (partially) managed by SDN rather than by protocol. Are protocols dying?

6.7 Retrospective: a day in the life of a web page request

With the journey down the protocol stack complete (application, transport, network, link), consider a student who attaches a laptop to the campus network (68.80.2.0/24, inside the Comcast network 68.80.0.0/13) and requests the page www.google.com, served by 64.233.169.105 in Google's network 64.233.160.0/19.

1. *DHCP.* The connecting laptop needs its own IP address, the address of its first-hop router, and the address of a DNS server, so it uses DHCP. The DHCP request is encapsulated in UDP, in IP, and in an 802.3 Ethernet frame, which is broadcast (destination FFFFFFFFFFFF) on the LAN and received at the router running the DHCP server; Ethernet is demultiplexed to IP, IP to

UDP, and UDP to DHCP. The DHCP server formulates a DHCP ACK containing the client's IP address, the IP address of its first-hop router, and the name and IP address of the DNS server. After encapsulation at the server, the frame is forwarded through the LAN (with switch learning) and demultiplexed at the client, which now has this information.

2. *ARP.* Before sending the HTTP request, the client needs the IP address of www.google.com: DNS. The DNS query is created and encapsulated in UDP, IP, and Ethernet. To send the frame to the router, the client needs the MAC address of the router's interface: ARP. The ARP query is broadcast and received by the router, which replies with its interface's MAC address; the client can now send the frame containing the DNS query.

3. *DNS.* The IP datagram containing the DNS query is forwarded via the LAN switch from the client to the first-hop router, then from the campus network into the Comcast network, and routed (with tables created by RIP, OSPF, IS-IS, and/or BGP) to the DNS server, where it is demultiplexed to DNS. The DNS server replies to the client with the IP address of www.google.com.

4. *TCP.* To send the HTTP request, the client first opens a TCP socket to the web server. The TCP SYN segment (step 1 of the three-way handshake) is inter-domain routed to the web server, which responds with a SYNACK (step 2), and the TCP connection is established.

5. *HTTP.* The HTTP request is sent into the TCP socket, and the IP datagram containing it is routed to www.google.com. The web server responds with an HTTP reply containing the web page, the IP datagram carrying it is routed back to the client, and the page is finally displayed.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 6 slides (9th edition).

Lecture 7: Wireless and Mobile Networks

7.1 Introduction

There are more mobile-broadband-connected (cellular) devices than fixed-broadband-connected devices (5 to 1 in 2019), and wireless connectivity is even greater when WiFi users are counted (80% of broadband homes use WiFi); 60% of Internet traffic from major web sites is destined to mobile devices. Wireless serves six application areas: wide-area mobile Internet access, local-area mobile Internet access, fixed wireless Internet access, satellite networks for Internet access and sensing, cable replacement, and the Internet of Things (IoT). They weigh differently the needs of mobility, user data rate, energy efficiency, device density, latency, and area traffic capacity.

The elements of a wireless network are:

- *wireless hosts* (laptop, smartphone, IoT device), which run applications and may be stationary or mobile: wireless does not always mean mobility;
- the base station, typically connected to the wired network, a relay responsible for sending packets between the wired network and the wireless hosts in its "area" (4G/5G cell towers, 802.11 access points);
- the *wireless link*, typically connecting devices to a base station and also used as a backbone link, with a multiple access protocol coordinating link access (random access, FDMA, TDMA, CDMA, polling) and various transmission rates, distances, and frequency bands;
- the *wireless device radio*: a device has different radios for different networks (an iPhone 16 has about 11 radios and many antennas: 5 cellular radios, WiFi, Bluetooth, UWB, satellite, NFC, GPS).

Not all wireless networks are connected into a larger network: in ad hoc networks there is no infrastructure. Link technologies trade rate against range: indoors (10–30 m), Bluetooth runs at about 2 Mbps and 802.11b, g, n, ac, ax at 11 Mbps, 54 Mbps, 600 Mbps, 3.5 Gbps, and 14 Gbps, with the faster WiFi standards reaching outdoor ranges (50–200 m); 5G reaches 10 Gbps up to midrange outdoor distances (200 m–4 km), 4G LTE and 5G eMBB extend to long range (4–15 km), 802.11af and ah cover midrange distances, and low-earth-orbit satellites (LEOS) cover hundreds of kilometers from space. A WiFi network has a wireless edge (WLANs) attached to an institutional wired core; a 5G network has an edge of user equipment (UE) and base stations (gNodeB) attached to a mobile core network.

7.2 Radio: the physical layer

Radio principles. A current at one location (the transmitter) creates an electromagnetic (EM) field in space, which induces a current at a remote location (the receiver). An EM wave has a wavelength λ, a frequency equal to the speed of light divided by the wavelength, c/λ, a direction of propagation, and a time-varying amplitude. The PHASE of a periodic signal is the fraction of the period covered up to time t, often measured in degrees (0 to 360); information can be coded into a signal by changing its phase while keeping its frequency.

Definition: Signal POWER (energy per unit time received or transmitted by an antenna) is measured linearly in watts (or milliwatts) or logarithmically in decibels:

$$P_{\text{dBm}} = 10 \log_{10} \frac{P_{\text{mW}}}{1\,\text{mW}}.$$

For example, 250 mW (a typical maximum mobile transmit power) is $10 \log_{10} 250 \approx 24$ dBm. The POWER SPECTRAL DENSITY gives the power present in a signal as a function of frequency, around the carrier (center) frequency. A signal's BANDWIDTH is the width, in Hz, of the range of frequencies it uses.

For example, a radio signal with 22 MHz of bandwidth spread evenly between 2.427 and 2.449 GHz (center 2.438 GHz) corresponds to channel 6 of a WiFi network. Radio signal "bandwidth" differs from the link "bandwidth" (maximum transmission rate) of network engineers.

Signals and noise. A received signal is corrupted by *interference*, from other transmitters and EM radiators in the same frequency band (hundreds of consumer devices share the unlicensed 2.4 GHz Industrial, Scientific, and Medical band: WiFi, Bluetooth, Zigbee, satellite TV, microwave ovens, garage-door openers, baby monitors, cordless phones, wireless microphones and speakers, drones and radio-controlled toys, amateur radio), and by thermal and electronic *noise* in the receiver (natural thermal variations, imperfections in electronics).

Definition: The SIGNAL-TO-NOISE RATIO is the ratio of signal power to noise power, often in dB,

$$\text{SNR (dB)} = 10 \log_{10} \frac{\text{received signal power}}{\text{noise power}},$$

a fundamental measure of channel quality: an SNR of 0 dB means equal signal and noise, and high (low) SNR makes it easy (hard) to extract the signal. Typical lower limits for current radios are -10 to -6 dB for cellular and 20 dB for WiFi. The SHANNON CAPACITY of a channel, the maximum rate at which data can be transmitted given bandwidth and SNR constraints (Shannon, 1948), is

$$C = B \log_2\!\left(1 + \frac{\text{received signal power}}{\text{noise power}}\right)$$

bits per second, for bandwidth B in Hz and powers measured linearly.

C scales linearly with B (for constant SNR), but at high SNR only logarithmically with SNR: there is little value in increasing SNR beyond a certain point.

Radio channels. A wireless radio signal attenuates (loses power) as it propagates. PATH LOSS (also called fading) is the reduction in power density of an EM wave as it propagates from transmitter to receiver; for frequency f and distance d, generalized path loss is $\sim (fd)^n$, with $n = 2$ in free space, $n \in (2.7, 3.5)$ in urban areas, and $n \in (3, 6)$ in buildings. Path loss causes HIDDEN TERMINALS: if A and B can hear each other and B and C can hear each other, but A and C cannot (their signals fall below the detection threshold at each other, or an object blocks them), then A and C are unaware of their interference at B, which complicates sharing the channel among A, B, C. In MULTIPATH PROPAGATION, the signal reflects off objects, the ground, and the built environment, and copies arrive at the destination at slightly different times. The COHERENCE TIME T_c is the amount of time a bit is present in the channel to be received; it influences the maximum possible transmission rate, since coherence times cannot overlap, and it is inversely proportional to the frequency and to the receiver's velocity.

Antennas: MIMO. Antennas both transmit and receive radio waves. A single antenna is old school; multiple antennas are common now (Gigabit LTE with 4×4 MIMO, WiFi 6 (802.11ax) with 2×2 MIMO). MIMO (multiple input, multiple output) uses several antennas at the transmitter and receiver.

- *Single-user MIMO, spatial diversity*: multiple antennas transmit to and receive at one user at the same time, exploiting multipath to increase the number of paths for the transmitted signal (with two antennas at each end, 12 signal paths). Signals along different paths differ (some arrive at higher quality), and *antenna gain* increases SNR by combining them. Sending redundant streams of the same information in parallel along different spatial paths improves reliability (all paths are unlikely to be degraded at once) and range (multiple antennas gather more signal). If x is transmitted and $H_{i,j}(\cdot)$ is the transformation of a signal from transmit antenna i to receive antenna j (estimated by the receiver, perhaps with the sender's help), receive antenna j gets $y_j = H_{1,j}(x) + H_{2,j}(x)$. The receiver should not just add y_1 and y_2; it might take the one with the better SNR, or re-align the phases and add coherently.
- *Spatial multiplexing*: send two *different* streams x_1 and x_2 in parallel along different spatial paths, so that
$$y_1 = H_{1,1}(x_1) + H_{2,1}(x_2), \qquad y_2 = H_{1,2}(x_1) + H_{2,2}(x_2) :$$
two equations in two unknowns, although the $H_{i,j}$ are time-varying stochastic quantities.
- *Multi-user MIMO*: multiple transmit antennas send to different users at the same time (spatial multiplexing), using directional antennas to reach geographically separated users simultaneously (BEAMFORMING).

Radio spectrum. Radio spectrum is a national asset: the national government determines how it is used locally. *Licensed* spectrum has a dedicated use, typically by one owner such as a cellular carrier, and is often allocated by spectrum auction. *Shared* spectrum is dynamically shared among users; an incumbent may get preferential access while others back off. *Unlicensed* spectrum is free for anyone to use within rules (such as transmission power levels): 2.4 and 5 GHz WiFi, and 3.5 GHz "private 5G" (CBRS). WiFi uses the 2.4 GHz band (11–14 channels depending on the country; 802.11 b/g/n/ax), the 5 GHz range (more than 150 configurable channels of different rates; 802.11 a/h/n/ac/ax), and the most recently added 6 GHz range (more than 250 channels; 802.11 ax/be). 5G uses three bands (there are no single, well-accepted ranges for "low", "mid", and "high"):

Band	Frequencies	Distance	Rate
low	< 1 GHz	tens of miles	50–250 Mbps
mid	1–6 GHz (popular: 1.8, 3.3–3.8, 6 GHz)	about 5 miles	100–900 Mbps
high (mmwave)	25–66 GHz (popular: 26, 40, 50, 66 GHz)	less than a mile	up to 3 Gbps

High-band transmission is line of sight, with poor penetration of trees and buildings.

Coding and modulation. The physical layer applies, in order, *coding* (adding error detection and correction bits), *digital modulation* (choosing the technique, for example from the family of quadrature amplitude modulation, that maximizes the number of reliably transmitted bits per second given the observed channel impairments), *pulse shaping* (matching the transmission bandwidth to the channel bandwidth), and *RF modulation* (transmitting the signal as an EM wave on the assigned CARRIER, a waveform at carrier frequency f_c modulated with the information-bearing signal). EDC bits are used at many layers (physical, link, network, transport, application). Adding them at the wireless physical layer makes sense: errors are detected and recovered locally rather than end to end, with techniques adapted to the local radio error characteristics.

Example: *Shuffling against burst errors.* The physical layer duplicates the bits $d_1, \ldots, d_8$ received from the link layer, giving $d_1 d_1' d_2 d_2' \cdots d_8 d_8'$, and shuffles them into

$$d_1\, d_5'\, d_2\, d_6'\, d_3\, d_7'\, d_4\, d_8'\, d_5\, d_1'\, d_6\, d_2'\, d_7\, d_3'\, d_8\, d_4'.$$

Without shuffling, a burst of noise that makes three consecutive bits unintelligible can destroy both copies of a bit (say $d_3' d_4 d_4'$), and the receiver cannot determine d_4. After shuffling, the two copies of each bit are at least seven positions apart, so a three-bit burst still lets the receiver determine all the original bits.

Definition: MODULATION encodes bits into an analog RF carrier signal by changing ("modulating") its amplitude, frequency, or phase. In AMPLITUDE SHIFT KEYING (ASK), the signal's amplitude encodes the bits. In BINARY PHASE SHIFT KEYING (BPSK), the phase encodes them: the 0 and 1 signals are 180° out of phase at the same carrier frequency. QPSK (quadrature phase shift keying) groups two bits

into one SYMBOL and encodes the four symbols with four phases: the same frequency as BPSK, but twice the data rate.

A CONSTELLATION DIAGRAM characterizes a modulation: each point depicts a symbol's amplitude (distance from the origin) and phase (degrees counterclockwise from east). QPSK has four symbols with identical amplitude and phases 45°, 135°, 225°, 315°. QUADRATURE AMPLITUDE MODULATION generalizes QPSK: m-QAM uses m different amplitude/phase values to encode $\log_2 m$ bits (4-QAM: b_0b_1; 16-QAM: $b_0b_1b_2b_3$; 64-QAM, used in WiFi: six bits). The receiver measures the amplitude and phase of the received signal and classifies the symbol as the bit pattern whose constellation point is closest; noise can push a received signal closer to another point, as when a transmitted 16-QAM symbol 1111 is received closer to 1011 and misclassified. A given modulation scheme has a nominal data rate (assuming no errors), and as SNR changes (changing interference, sender transmit power), the link moves along the scheme's SNR versus bit-error-rate tradeoff. ADAPTIVE MODULATION changes the modulation scheme and power on the fly, getting the highest data rate possible at an acceptable BER and energy expenditure as SNR changes; for example, the maximum data rate is 11.5 Mbps with 16-QAM and 26.9 Mbps with 64-QAM.

7.3 Sharing a wireless access channel

The classes of multiple access protocols carry over to wireless: channel partitioning (TDMA, FDMA, and OFDMA, used in 4G/5G, WiFi 6, Bluetooth, and satellite), random access (ALOHA, slotted ALOHA, CSMA, CSMA/CD in Ethernet, and CSMA/CA, used in WiFi, satellite, and the control planes of 4G/5G and Bluetooth), and taking turns (polling, used in Bluetooth and satellite), along with many wired technologies.

FDM and OFDM. Rather than one fat "wideband" channel, FDM divides the spectrum into multiple thin "narrowband" subchannels separated by guard bands to avoid interference. Each subchannel has a lower bit rate than the wideband channel, which makes multipath easier to accommodate and lower-rate implementations easier, and transmissions can happen on different subchannels in parallel. In OFDM (orthogonal FDM), the subcarriers are close together and slightly overlapping, with no guard bands; they are orthogonal, carefully designed and modulated so that crosstalk among subcarriers cancels at the center of each subcarrier frequency. (In wireless jargon, a "carrier" is a channel and a "subcarrier" a subchannel.) The number of subcarriers in a 4G/5G channel depends on the channel's bandwidth:

Channel bandwidth (MHz)	1.25	2.5	5	10	15	20
Number of subcarriers	76	150	300	600	900	1200

Definition: In 4G OFDMA, subcarriers are 15 kHz wide and one SYMBOL is transmitted per 66.6 μs on a given subcarrier; a symbol carries 2 to 8 bits, depending on the modulation chosen (which depends on channel quality). Symbols are grouped into RESOURCE BLOCKS (RBs): 84 symbols (7

symbols per 0.5 ms slot on each of 12 subcarriers, 180 kHz) in 4G. An RB is the smallest unit of data that can be sent to (assigned to) a user, and the base station decides which RBs, over all frequencies, go to which devices at 1 ms intervals.

From 4G LTE to 5G New Radio (NR): both use OFDMA and resource blocks as the smallest unit of transmission scheduled to a device. Subcarrier bandwidths are 15 kHz in 4G and 15, 30, 60, or 120 kHz in 5G; the maximum carrier bandwidth is 20 MHz in 4G and 100 MHz in 5G. An LTE physical RB spans 12 subcarriers with 7 symbols per subcarrier over 0.5 ms; a 5G NR physical RB spans 12 subcarriers (configurable bandwidth per subcarrier) with 14 symbols per subcarrier over 1 ms.

CSMA/CA. Hidden terminals (from path loss or from objects) mean that collisions cannot be reliably detected, and collisions still happen with carrier sensing: A senses the channel idle and transmits, and B senses it idle too, either because A's transmission has not reached B yet or because B cannot hear A due to fading. The 802.11 MAC protocol therefore avoids collisions instead of detecting them.

Algorithm 7.1: IEEE 802.11 CSMA/CA

```
// sender
if channel sensed idle for DIFS then
    transmit the entire frame                                   // no collision detection
else
    start a random backoff timer, which counts down while the channel is idle
    transmit when the timer expires
if no ACK received then increase the random backoff interval and repeat the backoff step
// receiver
if frame received OK then return an ACK after SIFS              // needed due to hidden terminals
```

After a collision (no ACK, so loss or collision is detected), each sender waits DIFS and then a randomized backoff, whose range depends on the PHY, and the backoff interval doubles after each collision; the sender whose timer expires first transmits successfully.

Definition: With RTS/CTS, a sender "reserves" the channel for data frames using small reservation packets. The sender first transmits a small REQUEST-TO-SEND (RTS) packet to the base station using CSMA (RTSs may still collide with each other, but they are short), containing the duration of the following data transmission. The base station broadcasts a CLEAR-TO-SEND (CTS) packet in response, also containing the duration. The RTS and/or CTS are heard by all nodes, and all nodes except the transmitter defer their transmissions while it transmits its data frame.

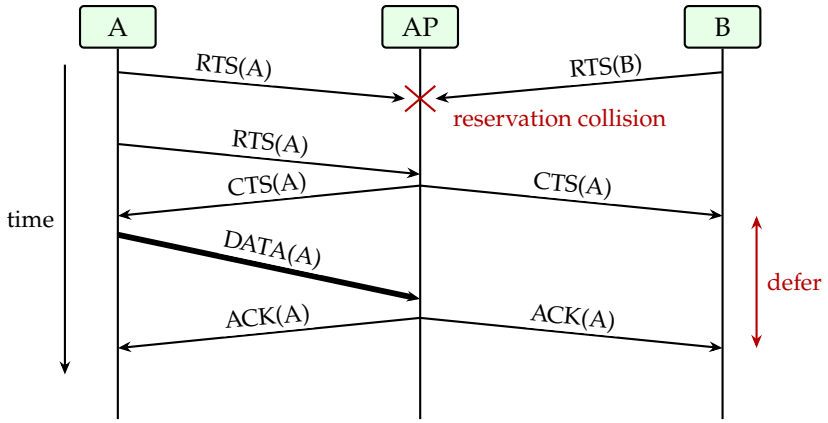


Figure 73: RTS/CTS exchange: the RTS frames of A and B collide, A retries, and the AP's CTS(A) makes B defer while A transmits its data frame.

Backwards compatibility. The WiFi generations:

Standard	Gen.	Year	Max rate	Freq. (GHz)	Bandwidth (MHz)	PHY
802.11b		1999	11 Mbps	2.4	22	spread spectrum
802.11g		2003	54 Mbps	2.4	20	OFDM
802.11n	WiFi 4	2009	600 Mbps	2.4, 5	20, 40	OFDM
802.11ac	WiFi 5	2013	6.9 Gbps	5	20–160	OFDM, MIMO
802.11ax	WiFi 6	2020	9.5 Gbps	2.4, 5	20–160	OFDM, OFDMA, MIMO
802.11be	WiFi 7	2024	30+ Gbps	2.4, 5, 6	20–320	OFDM, OFDMA, MIMO

(Rates are theoretical maxima; the 802.11ac, ax bandwidths are 20, 40, 80, 160 MHz, and 802.11be adds 320 MHz.) WiFi 6/7 access points support both OFDM (FDMA) and OFDMA (FDMA plus TDMA), and must serve user devices whose radios only do FDMA. The solution extends RTS/CTS with a MULTIUSER RTS (MU-RTS), which a transmitter must use before OFDMA transmissions. The MU-RTS is sent on the full FDMA channel with duration information, as usual, so it is heard by all 802.11 devices (OFDM- and OFDMA-capable), and it also carries OFDMA assignment information telling the receivers which OFDMA resource units to listen to. The receivers respond with CTS on their assigned OFDMA subchannels, and the transmitter sends resource blocks to the OFDMA receivers according to the assignment. For example, an AP sending to two legacy (OFDM-only) devices a, b and five OFDMA-capable devices $1, \ldots, 5$ uses ordinary RTS for a and b and MU-RTS for devices 1–5.

7.4 WiFi wireless LANs

The 802.11 link layer has a data plane and a control plane. The access point implements both the 802.11 MAC (toward the wireless devices) and the 802.3 MAC (toward the wired network). Downlink (AP-to-device) traffic consists of data frames containing IP datagrams and of control and management

frames (for RTS/CTS, reliable data transfer, energy management, channel measurement, discovery and attachment), all queued for transmission and ordered by MAC scheduling.

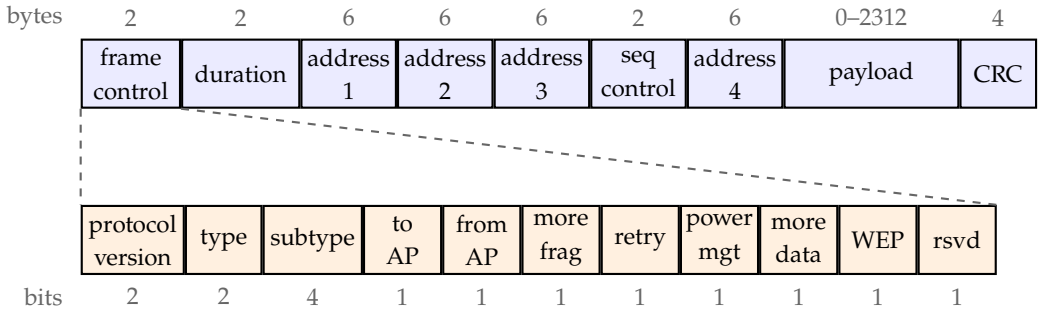


Figure 74: The 802.11 frame (field sizes in bytes) and its frame-control field (subfield sizes in bits).

Frame format (Figure 74). Address 1 is the MAC address of the wireless host or AP that is to receive the frame; address 2 is the MAC address of the wireless host or AP transmitting it; address 3 is the MAC address of the router interface to which the AP is attached; address 4 is used only in ad hoc mode. The duration field gives the reserved transmission time (RTS/CTS), and the sequence control field the frame sequence number (for reliable data transfer). The payload is an IP datagram or a link-layer control or management message. In the frame control field, the type is 0 (management), 1 (control), or 2 (data), and the subtype gives the type of management, control, or data frame.

Example: When host H1 sends a datagram to router R1 through an AP, the 802.11 frame from H1 to the AP has address 1 = AP MAC address, address 2 = H1 MAC address, and address 3 = R1 MAC address. The AP forwards it to R1 in an 802.3 Ethernet frame with destination R1 MAC address and source H1 MAC address.

Example: *A day in the life over WiFi.* (1) An HTTP/3 request is placed in a UDP segment, in an IP datagram, in an 802.11 frame, which enters the MAC frame-scheduling queue and waits. (2) The device sends an RTS control frame (using CSMA/CA) and the AP responds with CTS. (3) The device sends the data frame to the AP, while still buffering it. (4) The AP responds with an ACK, the device removes the transmitted frame, and the AP can now forward the frame to the router.

7.5 The 5G radio access network

Definition: A 5G network has a RADIO ACCESS NETWORK (RAN) and a CORE NETWORK. The 5G RAN is the edge network connecting devices to a base station: it provides link-layer service as the first hop between devices and the larger network, has limited geographic scope, and is under the control of a single service provider (somewhat analogous to a WiFi LAN). Its components are many devices (USER EQUIPMENT, UE), the radio channel (NEW RADIO, NR), and one base station (NEXT GENERATION NODE B, gNodeB or gNB). The core sits between the RAN and the other endpoints (other cores, the larger Internet).

Resource blocks and channels. A RESOURCE ELEMENT (RE) holds one transmitted OFDMA symbol, in one time minislot on one subcarrier frequency, and a resource block is a bundle of neighboring (time, frequency) REs: in 4G, 84 REs (12 neighboring subcarriers by 7 consecutive minislots), while 5G allows multiple bundlings. The RB is the smallest unit of transmission scheduling, and all symbols in an RB have the same source and destination; a 10 MHz channel has 50 RBs, each spanning a 180 kHz subchannel. In 5G a band has multiple channels, each with multiple subchannels. A PHYSICAL CHANNEL is a set of RBs carrying a similar type of data (radio channel information, user and control data, RB allocation information). There are three downlink physical channels, transmitted from the base station to devices (Physical Downlink Shared Channel, PDSCH; Physical Downlink Control Channel, PDCCH; Physical Broadcast Channel, PBCH), and three uplink physical channels (Physical Uplink Shared Channel, PUSCH; Physical Random-Access Channel, PRACH; Physical Uplink Control Channel, PUCCH). Many LOGICAL CHANNELS, per device or shared among devices, are mapped onto these six physical channels, with one or more logical channels mapped to the same physical channel.

RAN protocol stack. At the device and the base station, the RAN link layer consists, from the top, of SDAP, PDCP, RLC, and MAC above the physical layer, with RRC in the control plane.

- RRC (radio resource control) configures coarse-grained, policy-related aspects of the pipeline (scheduling prioritization, security). It implements the RAN's control plane and does not process user-plane packets.
- PDCP (packet data convergence protocol) compresses and decompresses IP headers and does ciphering and integrity protection.
- RLC (radio link control) does link-layer frame segmentation and reassembly and reliable data transfer (ARQ).
- MAC (media access control) buffers, multiplexes, and demultiplexes frames and makes real-time frame scheduling decisions in a 1 ms scheduling control loop.
- PHY (physical layer) does modulation and coding, followed by D/A conversion and the RF front end.

Example: *A day in the life over 5G.* (1) An HTTP/3 request in a UDP segment, in an IP datagram, is placed in a 5G frame, which enters and waits in the 5G MAC frame-scheduling queue. (2) The device's control plane sends a scheduling request (SR) on the PUCCH to the base station. (3) The base station's control plane sends a DCI message on the PDCCH granting upstream RBs. (4) The device sends the frame in the allocated RBs.

Split and software-defined RANs. Historically the RAN had no split: all of its functionality was implemented in a monolithic base station. In O-RAN, the pipeline is split into three units, a *central unit* (CU: RRC and PDCP), a *distributed unit* (DU: RLC, MAC, and part of PHY), and a *radio unit* (RU:

the rest of PHY, D/A conversion, and the RF front end), and where the units are implemented is an implementation choice. The traditional base station tightly couples its control and data planes. SD-RAN implements the RAN with an SDN approach, refactoring the RRC into a control-plane proxy (a 3GPP-compliant interface between the RAN and the mobile core control plane) and a Real-Time Intelligent Controller (RIC), with a new programmatic API for software control over the pipeline that implements the RAN user plane. Near-real-time network control applications (xApps: load balancing, handover, QoS, . . .) run on the RIC and consume a common RAN Network Information Base (R-NIB), holding for example longer-time-scale CQI values and other per-session state. RAN interfaces and high-level functions are standardized, but the implementation of some functions (RB assignment to channels, frame scheduling) is up to the network operator: these implementation choices are the operator's technical "secret sauce".

7.6 Common challenges: network discovery, frame scheduling, energy

Joining the edge network. An arriving device must first establish an initial connection with a node at the wireless edge of the network (a WiFi AP, a 4G/5G base station); this connection is then used for authentication, DHCP, and gaining full network access ("bootstrapping" connectivity). This association uses two approaches, either or both of which are used in practice: in beaconing, the mobile listens on wireless channels for beacons advertising wireless network nodes; in probing, the mobile actively transmits probe messages and waits for nodes to reply.

- *802.11 beaconing*: the AP broadcasts a beacon frame, the arriving device sends an association request, and the AP sends an association response. *Probing*: host H1 broadcasts a probe request frame, the APs send probe response frames, H1 sends an association request frame to the selected AP, and that AP sends an association response. No identity or credentials have been exchanged yet ("Can we talk?"); the device next authenticates to the network.
- *5G*: base stations of each network broadcast their SSB and SIBs; the device sends an RRC connection setup request to the chosen base station, which replies with an RRC connection setup response.

MAC scheduling. All packets to and from all devices in an edge network pass through the base station. In 4G/5G and in WiFi 6/7, OFDMA resource blocks are scheduled: a *downlink MAC scheduler* decides which frames to send from the base station to which devices, in which order, in which RBs; an *uplink MAC scheduler* decides which devices send which frames, in which order, in which RBs. Design considerations: is the scheduler *channel aware* (does it consider the quality of the channel between base station and device)? Is it *QoS aware* (does it consider the quality of service needed by, or guaranteed to, a device, such as voice versus web browsing)? Does it give *priority* among traffic types (user, control, real-time) and *fairness* among UEs (which has no standard definition)? Standards do not dictate the answers: scheduling algorithms and parameters are up to network implementers and operators.

In 4G, a 10 MHz channel has 50 subchannels, each RB spanning 180 kHz; a resource block of 84 symbols lets one user transmit 84 symbols on one subchannel in a 0.5 ms interval. The base station decides which frames or packets are assigned to which RBs for transmission, with a 1 ms scheduling interval, 50 subchannels per 10 MHz channel, and hundreds of active users. The scheduler takes two inputs. The Channel Quality Indicator (CQI) is measured by the user from reference signals embedded in RBs and reported back to the base station; a 4-bit CQI value maps to the modulation scheme to use (which QAM) and the expected throughput. The QoS Class Indicator (QCI) is the QoS the network wants to provide to a device (delay, guaranteed bit rate), and the scheduler allocates radio spectrum so that all UEs meet their QoS requirements:

Priority	QCI	Max delay	Max loss	Application
2	1	100 ms	10^{-2}	voice
3	4	50 ms	10^{-3}	real-time gaming
6	7	300 ms	10^{-6}	streaming video
8	8	600 ms	10^{-6}	web browsing, TCP

Abstractly, the scheduler selects segments to transmit from a set of subscriber queues, using the CQIs reported by devices and the QCIs assigned to subscribers, and allocates resource blocks to them. Scheduling policies:

- *Priority*: arriving traffic is classified (any header fields can be used, for example control versus user data) and queued by class, and the scheduler sends a packet from the highest-priority queue that has buffered packets; scheduling within a class can use another discipline.
- *Round robin* (per UE): each device has its own queue, and the scheduler cyclically scans the device queues, mapping a frame from each device (if available) to an RB in turn. It is channel unaware and QoS unaware, and fair by turns but not necessarily throughput-fair, since devices may have different channel quality.
- *Maximum throughput* (MT): with $d_k^i(t)$ the expected data rate for UE i using the kth RB at time t (for example, based on CQI), at time t select $\max_{i,k} d_k^i(t)$ and assign UE i's frame to RB k. It maximizes overall immediate throughput; it is channel aware, QoS unaware, and throughput unfair.
- *Blind equal throughput* (BET, per device): the scheduler maintains an exponentially weighted moving average of the data rate received by each UE i,

$$R^i(t) = (1-\beta)\, r^i(t) + \beta\, R^i(t-1),$$

  where $r^i(t)$ is the rate achieved in scheduling interval t, and at t it selects $\min_i R^i(t-1)$ and assigns UE i's frame blindly (without knowing channel conditions) to any RB: the RB goes to the device with the smallest data rate in the last round. It is somewhat channel aware, QoS unaware, and blindly throughput fair.

- *Proportional fairness* (PF) combines BET and MT: at t, select $\max_{i,k}\{d_k^i(t)/R^i(t-1)\}$ and assign UE i's frame to RB k. It balances a high data rate with a measure of fairness: channel aware, QoS unaware, fairness aware.

Energy. Battery energy is a critical resource in wireless devices (phones, IoT). LTE radios use 1000–3500 mW when on (transmitting, or receiving or waiting to receive) and less than 15 mW when off (about 1–2% of the on-state use). A significant amount of device energy is spent on communication across applications, and 5G implementations currently spend more energy than 4G. Cellular, WiFi, and Bluetooth radios are therefore turned off when not in use. Going to sleep is easier: it is device driven (sleep if there is nothing to send and nothing being received), may or may not be coordinated with the base station, AP, or Bluetooth controller, and may be a deep sleep (longer, more "turned off") or a light sleep. Waking up is harder: the base station, AP, or controller must signal the device to wake up when there are packets to send to it, network context (4G/5G) may need to be re-created, and data frames containing IP datagrams from remote hosts must be buffered at the base station until the device is awake. Sleeping longer saves more energy but adds latency (with effects on higher-layer protocols such as TCP); sleeping less saves less energy and adds signaling overhead. How long to stay awake depends on the time since the last activity: data transmission is typically bursty, and the burst should be over before going to sleep.

Example: *Coordinated wake/sleep in 5G* (discontinuous reception, DRX). At t_0 a transmission is received and the device starts an inactivity timer. At t_1 the timer expires and the device enters short DRX cycles: it wakes up periodically and, with nothing to send or receive, returns to short DRX sleep. At t_2, with still nothing to send or receive, it enters long DRX cycles. At t_3 an incoming transmission for the device arrives and is buffered at the base station until the end of the current DRX cycle. At t_4 the device wakes from long DRX sleep, accepts a channel scheduling assignment, and ends its sleep, and at t_5 it receives the transmission.

7.7 The wireless core network

Definition: The 5G core is a single core serving multiple RANs. It consists of links, routers, and servers providing services to devices and base stations: an "all IP" core, but with services very different from traditional Internet applications, and a clear logical separation between control plane and user plane (CUPS, control- and user-plane separation). The 5G core is mostly about control-plane services.

Its main functional components are:

- UPF (user plane function): each device is assigned to a UPF, which relays traffic between the UE and the Internet; it is responsible for policy enforcement, lawful intercept, traffic usage measurement, and QoS policing, and for tunneling (encapsulating and decapsulating) traffic to and from the base station over the N3 interface.

- AMF (access and mobility management function): connection and reachability management, mobility management, access authorization, and location services.
- SMF (session management function): manages each UE session, including IP address allocation, selection of the associated UPF, QoS control, and the control aspects of user-plane routing.
- AUSF (authentication server function) authenticates devices; UDM (unified data management) manages user identity, including the generation of authentication credentials; UDR (unified data repository) manages static subscriber-related information; UDSF (unstructured data storage function) stores unstructured data, much like a key/value store.
- Further functions include the PCF (policy control), NEF (network exposure, toward third-party applications), NRF (network repository), AF (application function), and SEPP (security edge proxy, toward the control planes of other cellular providers).

The device talks to the AMF over interface N1 and the base station over N2; the base station sends user traffic to the UPF over N3. The control plane emphasizes functions and services, with well-defined interfaces between services (N4 between SMF and UPF, N8, N10, N11, N12, N13, . . .). The data plane is a five-layer Internet data plane with extensive use of tunneling.

Example: *User-plane tunnels.* A sender on the Internet with IP address w sends a datagram to user device z. It reaches the UPF for device z (address x), which encapsulates it (GTP-U over UDP over IP) in a datagram with source x and destination y, the address of the device's base station, and sends it across the backhaul network. The base station decapsulates the original datagram (source w, destination z) and delivers it over the radio link.

Identity. On the Internet you are a name and password (different ones for different applications and services) or a name and public-key certificate (servers often have public keys, users not so much yet). In 4G/5G you are a customer of a wireless service provider, with a home cellular network (the one you pay), and you have a SIM (Subscriber Identity Module) card or vSIM, which identifies you and your home network to all cellular networks globally and holds cryptographic key information also known to your home network.

Joining a 5G network. There are two major phases, in which the AMF plays the key role. In *registration*, the device identifies and authenticates itself to the network, and vice versa: after RRC connection setup with the base station, the device sends a registration request to the AMF, identity request/response and authentication and security exchanges follow (involving the AUSF in the home 5G core), the AMF updates the device's connection data and retrieves its subscription from the UDM, and the AMF sends a registration accept. In *PDU session establishment* (PDU, protocol data unit, is another name for packet), core network functions allocate an IP address to the device and create a data-plane path between the device and its UPF: the device sends a PDU session establishment request; the AMF and SMF exchange a session creation request and response; the SMF establishes the session with the UPF and returns session information (such as the device's

IPv4 address and tunnel) to the AMF; the AMF sends a session request with the SMF information to the base station, which responds; the device, base station, and RAN data plane are configured, and the first uplink datagram flows in the data plane.

7.8 Mobility

From the network's perspective, mobility ranges from none, through devices that move between networks but power down while moving and devices that move within the same WLAN or RAN of one provider, to devices that move among WLANs or RANs of one provider network, and devices that move among multiple provider networks while maintaining ongoing connections; the last two cases are the interesting ones.

Mobility in WiFi. 802.11 is a link-layer standard and takes a link-layer approach to mobility. An EXTENDED SERVICE SET (ESS) is a set of basic service sets (BSSs) in the same layer-3 subnet, each with the same SSID, and devices move between BSSs of the same ESS, with fast re-authentication and APs suggesting new APs to a device. Vendors offer proprietary solutions for enhanced mobility.

Definition: In 5G HANDOVER, a mobile device changes its point of attachment to the network, and the data flow to the device changes from the *source* base station to the *target* base station. Handover is performed because of a stronger signal from the target base station, or because the target has fewer devices and less traffic.

1. The current (source) base station selects the target and sends it a Handover Request message.
2. The target pre-allocates radio time slots and responds with a Handover Request ACK containing information for the mobile.
3. The source informs the mobile of the new base station; the mobile can now send via the new base station, and the handover looks complete to the mobile.
4. The source stops sending datagrams to the mobile and instead forwards them to the target, which forwards them to the mobile over the radio channel.
5. The target informs the AMF and SMF that it is the mobile's new base station, and the SMF instructs the UPF to change the tunnel endpoint to the target.
6. The target ACKs back to the source: the handover is complete, and the source can release its resources.
7. The mobile's datagrams now flow through the new tunnel from the target base station to the UPF.

In the signaling timeline, the source makes the handover decision from the device's measurements (handover request, accept, handover request ACK); the handover in the RAN follows (RRC

reconfiguration, transfer status, the device detaching from the source and attaching to the target, which buffers device data forwarded from the source, RRC reconfiguration complete, handover success); and finally the handover in the core (path switch request from the target to the AMF, PDU session update toward the SMF, session modification between SMF and UPF, a data end marker, path switch ACK, and release of the device context at the source).

7.9 Bluetooth, satellite, and IoT wireless networks

Bluetooth. Bluetooth (BT) is an example of a wireless ad hoc network: devices have no infrastructure (access point, base station) to connect to, so BT devices must find other BT devices and organize themselves into a network from scratch. This requires connecting devices at the link layer with channel allocation, address assignment, service discovery, application-to-application communication over the Bluetooth network, and security: an entirely new architecture and protocol stack, different from the Internet's.

Definition: A Bluetooth network is a PICONET, generally less than 10 m in diameter, with no more than 8 devices: the device that initially forms the network takes the CENTRAL role, and up to 7 more devices take the PERIPHERAL role. Communication is only between the central node and peripheral nodes, never directly between peripherals. A BT device has a unique 48-bit address, and BT operates in the unlicensed 2.4 GHz ISM band.

Classic BT uses 79 channels of 1 MHz in the 2.4–2.5 GHz band, with a maximum network rate of 2.1 Mbit/s; BLE (Bluetooth Low Energy) uses 40 channels of 2 MHz (3 advertising and 37 data channels). The BT channel is TDM with 625 μs slots, and access is by polling: the central device advertises and grants channel access to the peripherals in its network. Transmissions use FREQUENCY HOPPING, a form of spread spectrum: senders hop among the 79 channels, transmitting on a different frequency after each slot, following a hopping pattern known to all devices in the piconet, and piconets with different patterns can coexist in the same space. Hopping minimizes the effects of interference: there are potentially many interfering transmitters in the ISM band, but if an interfering device uses channel x, then x occurs only occasionally in the hopping sequence, and a BT frame lost on x is retransmitted on a different frequency in the next slot.

The Bluetooth stack is completely different from the Internet stack, reflecting BT's special purpose: there are no network or transport layers (no routing in classic BT; BLE adds managed flooding), and the layers are less conceptually well defined than in the Internet or 5G. A host (the operating system: applications and end-to-end services over the Logical Link Control and Adaptation layer, L2CAP) talks through a host-controller interface (HCI) to the controller (firmware: link and physical layers, with separate controllers for classic BT and BLE); this host/controller structure is deeply embedded in the architecture. L2CAP creates and manages end-to-end L2CAP channels between two devices, for application and user data or higher-level services, providing reliability, flow control, and segmentation. Its channel types are connection-oriented asynchronous (ACL), connectionless asynchronous (ACL), and synchronous (eSCO, for example for a periodic audio application). In

a piconet, the central device might run end-to-end L2CAP connections for MP3 music with one peripheral and for a remote keyboard with another, plus service discovery, all across the shared frequency-hopping channel.

Satellites. Satellites are used for sensing (environment, weather, land use, human activity, at a resolution of about a meter per pixel), broadcast (direct-to-home TV and radio, content to many cable head ends, GPS: one transmission reaches many users), and Internet access (connectivity for unserved regions as an alternative to wired Internet, low latency over long distances, access in airplanes).

- *GEO* (geosynchronous) satellites are stationary with respect to the ground, 35,786 km above the earth, with about 800 ms RTT and wide coverage per satellite.
- *LEO* (low earth orbit) satellites move with respect to the ground at 27,000 km/h, staying within line of sight of a ground station for 5–15 minutes, at 550–1200 km, with about 30 ms RTT in practice and smaller coverage per satellite; a constellation (network) of LEO satellites, possibly with inter-satellite links (ISL), covers a wide area.

Many LEO satellites are being launched:

Constellation	Deployed (2025)	Planned	ISL planned?
Starlink	7,239	12,000–42,000	yes
Eutelsat OneWeb	648	716–6,372	no
Kuiper (Amazon)	27	3,236	?
Telesat	1	298–1,671	yes

A LEO network has satellites in motion with inter-satellite links, a satellite channel that broadcasts (multicasts) down and is unicast up within each satellite's "footprint", ground stations connected to the Internet, and mobile user terminals (rural and remote communications, airborne, maritime, vehicles). It can be *links in the sky*, a single satellite hop between base stations (the "bent pipe" architecture), or a *network in the sky*, with multiple satellite hops and routing among satellites. Starlink, first launched in 2019, had about 7,239 operational LEO satellites in April 2025 (12K planned, 30K requested), in three orbital shells at 525, 530, and 535 km, with satellites of 500–2700 lbs. It had 4M Internet subscribers in September 2024 (residential US: $120 per month), provides service in Ukraine (where terrestrial networks were damaged), and in 2024 was testing direct-to-smartphone service using cellular spectrum from SpaceX's US mobile partner T-Mobile.

Internet of Things. IoT has many use cases: monitoring (pressure, vibration, tilt, temperature, humidity, openings, battery charge, leaks, cracks, fill levels), asset geolocation and tracking (warehouse security, transport conditions for food safety), city monitoring (sound, air, water, weather, lighting, traffic, crowds, infrastructure, bike sharing), smart metering and remote diagnosis,

agriculture (rainfall, soil condition, animal tracking and health), and building security and predictive infrastructure monitoring. Connected devices number about 12 billion in 2014 and a forecast 45.7 billion in 2028, with short-range and wide-area IoT devices accounting for most of the growth (the number of PCs, laptops, tablets, and phones stays roughly flat). IoT communication is wireless, over distances from meters to kilometers, at data rates from hundreds of bps when active (metering) to about 1 Mbps (video surveillance), and energy is always a consideration: devices may run up to about 10 years without external power, or harvest energy locally. The technical tradeoff is among wide-area coverage, high data rate, and low energy consumption: short-range technologies give up wide-area coverage (WiFi favoring data rate, BLE and Zigbee low energy), traditional cellular (4G/5G) gives up low energy consumption, and low-power wide-area technologies (LoRaWAN, LTE-M) give up high data rate.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 7 slides (9th edition).

Lecture 8: Security in Computer Networks

8.1 What is network security?

Definition: Network security has four goals. Confidentiality: only the sender and the intended receiver should "understand" message contents (the sender encrypts the message, and the receiver decrypts it). Authentication: the sender and receiver want to confirm each other's identity. Message integrity: the sender and receiver want to ensure that a message is not altered (in transit or afterwards) without detection. Access and availability: services must be accessible and available to users.

In the standard cast, Bob and Alice want to communicate "securely" over a channel carrying data and control messages, and Trudy, an intruder, may intercept, delete, or add messages. Real-life Bobs and Alices include a web browser and server for electronic transactions (online purchases), an online banking client and server, DNS servers, and BGP routers exchanging routing table updates. A "bad guy" can do a lot: *eavesdrop* (intercept messages), actively *insert* messages into a connection, *impersonate* (fake, or spoof, the source address or any other field of a packet), *hijack* an ongoing connection ("take over" by removing the sender or receiver and inserting himself in its place), and deny service (prevent a service from being used by others, for example by overloading resources).

8.2 Principles of cryptography

Let m be a plaintext message. Alice's encryption algorithm with key K_A produces the ciphertext $K_A(m)$, and Bob's decryption algorithm with key K_B recovers $m = K_B(K_A(m))$. Trudy may attack an encryption scheme in three settings:

- *ciphertext-only attack*: Trudy has ciphertext to analyze, and can search all keys (brute force) or use statistical analysis;
- *known-plaintext attack*: Trudy has plaintext corresponding to ciphertext (for example, in a monoalphabetic cipher she determines the pairings for a, l, i, c, e, b, o);
- *chosen-plaintext attack*: Trudy can get the ciphertext for a chosen plaintext.

Definition: In symmetric key cryptography, Bob and Alice share the same (symmetric) key K_S, used both to encrypt, $K_S(m)$, and to decrypt.

For example, the key may be knowledge of the substitution pattern in a monoalphabetic substitution cipher, which substitutes one letter for another. With

plaintext letters:	`abcdefghijklmnopqrstuvwxyz`
ciphertext letters:	`mnbvcxzasdfghjklpoiuytrewq`

the plaintext `bob. i love you. alice` encrypts to `nkn. s gktc wky. mgsbc`; the encryption key is the mapping from the set of 26 letters to itself. A more sophisticated approach uses n substitution ciphers $M_1, \ldots, M_n$ and a cycling pattern, for example, with $n = 4$,

$$M_1, M_3, M_4, M_3, M_2; \quad M_1, M_3, M_4, M_3, M_2; \quad \ldots$$

Each new plaintext symbol uses the next substitution in the cycle, so "dog" encrypts d with M_1, o with M_3, and g with M_4. The key (the n ciphers and the cyclic pattern) need not be just an n-bit pattern. But how do Bob and Alice agree on the key value?

Definition: DES (Data Encryption Standard, a US encryption standard, NIST 1993) uses a 56-bit symmetric key on 64-bit plaintext blocks, as a block cipher with cipher block chaining. A 56-bit-key-encrypted phrase was decrypted by brute force in less than a day (the DES Challenge), but there is no known good analytic attack; 3DES encrypts three times with three different keys. AES (Advanced Encryption Standard), the symmetric-key NIST standard that replaced DES in November 2001, processes data in 128-bit blocks with 128-, 192-, or 256-bit keys. A brute-force decryption that takes 1 second on DES takes 149 trillion years on AES.

Public key cryptography. Symmetric key cryptography requires the sender and receiver to know a shared secret key; how do they agree on it in the first place, particularly if they have never "met"? Public key cryptography (Diffie-Hellman 1976, RSA 1978) is a radically different approach: the sender and receiver do not share a secret key; a PUBLIC encryption key K_B^+ is known to all, and a PRIVATE decryption key K_B^- is known only to the receiver, so that $m = K_B^-(K_B^+(m))$. Similar ideas emerged at roughly the same time, independently, in the US and the UK (classified there), revolutionizing a cryptography that for 2000 years had been only symmetric. A public key algorithm needs keys $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that $K_B^-(K_B^+(m)) = m$, and it must be impossible to compute the private key K_B^- from the public key K_B^+.

RSA. RSA (Rivest, Shamir, Adleman) relies on modular arithmetic, $x \bmod n$ being the remainder of x divided by n:

$$[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n,$$
$$[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n,$$
$$[(a \bmod n) \cdot (b \bmod n)] \bmod n = (a \cdot b) \bmod n,$$

and hence $(a \bmod n)^d \bmod n = a^d \bmod n$. For example, with $x = 14$, $n = 10$, $d = 2$: $(x \bmod n)^d \bmod n = 4^2 \bmod 10 = 6$, and $x^d = 14^2 = 196$, so $x^d \bmod 10 = 6$. A message is just a bit pattern, which is uniquely represented by an integer (the message 10010001 is the number 145), so encrypting a message is equivalent to encrypting a number.

Definition: *RSA key pair.* (1) Choose two large primes p, q (for example, 1024 bits each). (2) Compute $n = pq$ and $z = (p-1)(q-1)$. (3) Choose $e < n$ with no common factors with z (e and z

are relatively prime). (4) Choose d such that $ed - 1$ is exactly divisible by z, that is, $ed \bmod z = 1$. (5) The public key is $K_B^+ = (n, e)$ and the private key is $K_B^- = (n, d)$. To encrypt a message $m < n$, compute $c = m^e \bmod n$; to decrypt the received bit pattern c, compute $m = c^d \bmod n$.

Example: Bob chooses $p = 5$, $q = 7$, so $n = 35$ and $z = 24$; $e = 5$ is relatively prime to z, and $d = 29$ works since $ed - 1 = 144 = 6 \cdot 24$. To encrypt the 8-bit message 00001100, that is $m = 12$: $m^e = 12^5 = 248832$ and $c = 248832 \bmod 35 = 17$. To decrypt: $c^d = 17^{29} = 48196857210675091509141182522307169 7$ and $c^d \bmod 35 = 12 = m$.

RSA works because, for any x and y, $x^y \bmod n = x^{(y \bmod z)} \bmod n$ with $n = pq$ and $z = (p-1)(q-1)$. Hence

$$c^d \bmod n = (m^e \bmod n)^d \bmod n = m^{ed} \bmod n = m^{(ed \bmod z)} \bmod n = m^1 \bmod n = m.$$

RSA has another important property: applying the private key first and the public key second gives the same result as the usual order, $K_B^-(K_B^+(m)) = m = K_B^+(K_B^-(m))$, since $(m^e \bmod n)^d \bmod n = m^{ed} \bmod n = m^{de} \bmod n = (m^d \bmod n)^e \bmod n$. RSA is secure because determining d from Bob's public key (n, e) essentially requires finding the factors of n without knowing p and q, and factoring a big number is hard.

Session keys. Exponentiation in RSA is computationally intensive: DES is at least 100 times faster than RSA. In practice, public key cryptography establishes a secure connection and a second key, a symmetric SESSION KEY K_S, for encrypting data: Bob and Alice use RSA to exchange K_S, and once both have it, they use symmetric key cryptography. Public key cryptography also has many non-confidentiality uses: authentication, digital signatures, message integrity, and symmetric-key agreement.

8.3 Authentication, digital signatures, message integrity, shared key agreement

Authentication. The goal is for Bob to have Alice "prove" her identity to him. A sequence of protocols shows what is needed.

- *ap1.0*: Alice says "I am Alice". In a network, Bob cannot "see" Alice, so Trudy simply declares herself to be Alice.
- *ap2.0*: Alice says "I am Alice" in an IP packet containing her source IP address. Trudy can create a packet "spoofing" Alice's address.
- *ap3.0*: Alice says "I am Alice" and sends her secret password to "prove" it. This fails to a PLAYBACK ATTACK: Trudy records Alice's packet and later plays it back to Bob. Sending an *encrypted* password does not help, since the playback attack still works.
- *ap4.0*: to prove Alice is "live", Bob sends Alice a NONCE R, a number used only once in a lifetime, and Alice must return R encrypted with their shared secret key, $K_{A\text{-}B}(R)$. Bob knows Alice is

live, and only Alice knows the key to encrypt the nonce, so it must be Alice. But ap4.0 requires a shared symmetric key.

- *ap5.0*: use a nonce and public key cryptography. Bob sends R; Alice returns $K_A^-(R)$; Bob asks for Alice's public key and receives K_A^+; Bob computes $K_A^+(K_A^-(R)) = R$ and knows that only Alice could have the private key that encrypted R this way.

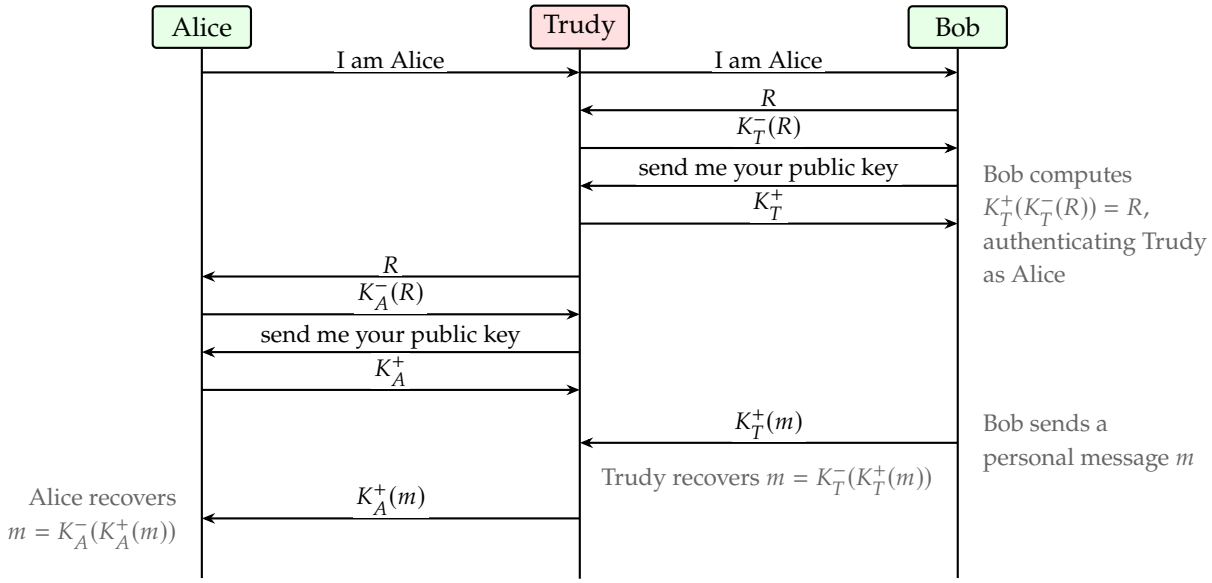


Figure 75: A man-in-the-middle attack on ap5.0: Trudy poses as Alice to Bob and as Bob to Alice.

ap5.0 still has a flaw, a MAN-IN-THE-MIDDLE ATTACK (Figure 75): Trudy poses as Alice (to Bob) and as Bob (to Alice). She answers Bob's nonce with $K_T^-(R)$ and his request for Alice's public key with her own K_T^+, so Bob computes $K_T^+(K_T^-(R)) = R$, authenticating Trudy as Alice. When Bob sends a personal message encrypted as $K_T^+(m)$, Trudy recovers $m = K_T^-(K_T^+(m))$ and sends m to Alice encrypted with Alice's public key; Alice recovers m, and when she and Bob meet a week later in person and discuss m, neither knows that Trudy knows m. The mistake is that Bob accepts a public key without any proof that it belongs to Alice.

Digital signatures. A DIGITAL SIGNATURE is a cryptographic technique analogous to a handwritten signature: the sender (Bob) digitally signs a document, establishing that he is the document's owner or creator. It is verifiable and nonforgeable: the recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed the document. In a simple digital signature for a message m, Bob signs m by encrypting it with his private key, creating the "signed" message $K_B^-(m)$, and sends $m, K_B^-(m)$. Alice verifies that m was signed by Bob by applying Bob's public key to $K_B^-(m)$ and checking that $K_B^+(K_B^-(m)) = m$; if so, whoever signed m must have used Bob's private key. Alice thus verifies that Bob signed m, that no one else signed m, and that Bob signed m and not some m'. This gives NON-REPUDIATION: Alice can take m and the signature $K_B^-(m)$ to court and prove that Bob signed m.

Definition: Public-key-encrypting long messages is computationally expensive, so the goal is a fixed-length, easy-to-compute digital "fingerprint". Applying a HASH FUNCTION H to m gives a fixed-size MESSAGE DIGEST $H(m)$. A cryptographic hash function is many-to-one, produces a fixed-size digest, and, given a digest x, makes it computationally infeasible to find m such that $x = H(m)$.

Example: The Internet checksum has some hash-function properties (a fixed-length, 16-bit digest; many-to-one), but given a message with a given hash value, it is easy to find another message with the same value. The messages `IOU100.99BOB` and `IOU900.19BOB`, in ASCII `49 4F 55 31 30 30 2E 39 39 42 4F 42` and `49 4F 55 39 30 30 2E 31 39 42 4F 42`, are different but have the identical checksum `B2 C1 D2 AC` (adding the bytes in four columns: swapping the digits 1 and 9 moves the same values between columns).

A DIGITAL SIGNATURE in practice is a signed message digest: Bob sends the large message m together with $K_B^-(H(m))$, his encrypted message digest. Alice applies H to the received m and applies Bob's public key to the signature, and checks that the two values of $H(m)$ are equal, verifying both the signature and the integrity of the digitally signed message. Hash function algorithms in use include MD5 (RFC 1321), which computes a 128-bit message digest in a 4-step process, and for which it appears difficult, for an arbitrary 128-bit string x, to construct a message m whose MD5 hash equals x; and SHA-1, a US standard (NIST, FIPS PUB 180-1) with a 160-bit message digest.

Certified public keys. To fix ap5.0, a public key must be bound to its owner. Without that binding, Trudy can play a pizza prank on Bob: she creates an e-mail order ("Dear Pizza Store, Please deliver to me four pepperoni pizzas. Thank you, Bob"), signs it with her private key, and sends it to the Pizza Store together with her public key, claiming it is Bob's. The Pizza Store verifies the signature and delivers four pepperoni pizzas to Bob, who does not even like pepperoni.

Definition: A CERTIFICATION AUTHORITY (CA) binds a public key to a particular entity E (a person, website, router). E registers its public key with the CA, providing "proof of identity", and the CA creates a CERTIFICATE binding identity E to E's public key: a certificate containing E's public key, digitally signed by the CA (with K_{CA}^-), saying "this is E's public key". When Alice wants Bob's public key, she gets Bob's certificate (from Bob or elsewhere) and applies the CA's public key K_{CA}^+ to it, obtaining K_B^+.

Shared key agreement. For Bob and Alice to use symmetric key techniques, they must agree on the shared key, even if they have never met before; public key cryptography solves this too.

Definition: In DIFFIE-HELLMAN (DH) key agreement, an algorithm different from RSA, Bob chooses public values: a large prime p and another large number $g < p$. Alice and Bob independently choose private values S_A and S_B, and their DH public keys are

$$K_A^+ = g^{S_A} \bmod p, \qquad K_B^+ = g^{S_B} \bmod p.$$

Alice computes the shared secret $(K_B^+)^{S_A} \bmod p$ using Bob's public key and S_A; Bob computes

$(K_A^+)^{S_B} \bmod p$ using Alice's public key and S_B.

The two locally computed values are equal, since both are $g^{S_A S_B} \bmod p$: $(g^{S_B} \bmod p)^{S_A} \bmod p = g^{S_B S_A} \bmod p = (g^{S_A} \bmod p)^{S_B} \bmod p$. So Alice and Bob compute a common shared secret without ever explicitly exchanging it.

8.4 Securing e-mail

Alice wants to send a confidential e-mail m to Bob. She generates a random symmetric private key K_S, encrypts the message with K_S (for efficiency), also encrypts K_S with Bob's public key, and sends both $K_S(m)$ and $K_B^+(K_S)$ to Bob. Bob uses his private key to decrypt and recover K_S, and uses K_S to decrypt $K_S(m)$ and recover m.

For message integrity and authentication, Alice digitally signs a hash of her message with her private key and sends both the message (in the clear) and the digital signature $K_A^-(H(m))$; Bob applies K_A^+ to the signature and compares the result with the hash he computes from m. For confidentiality, integrity, and authentication together, Alice uses three keys: her private key (to sign $H(m)$), a new symmetric key K_S (to encrypt m together with its signature), and Bob's public key (to encrypt K_S). Bob's complementary actions are to recover K_S with his private key, decrypt the message and signature with K_S, and verify the signature with Alice's public key.

8.5 Securing TCP connections: TLS

Definition: Transport Layer Security (TLS) is a widely deployed security protocol above the transport layer, supported by almost all browsers and web servers (https, port 443). It provides confidentiality (via symmetric encryption), integrity (via cryptographic hashing), and authentication (via public key cryptography), all techniques seen above. Its history runs from early research on secure network programming and secure sockets to the Secure Sockets Layer (SSL), deprecated in 2015, and TLS 1.3 (RFC 8446, 2018).

TLS needs a *handshake*, in which Alice and Bob use their certificates and private keys to authenticate each other and exchange or create a shared secret; *key derivation*, in which they use the shared secret to derive a set of keys; *data transfer*, of stream data as a series of records (not just one-time transactions); and *connection closure*, with special messages to close the connection securely. A toy protocol, t-tls, shows what is needed.

- *Handshake.* Bob (the client) establishes a TCP connection with Alice (TCP SYN, SYNACK, ACK) and sends a t-tls hello; Alice replies with her public key certificate, from which Bob verifies that Alice is really Alice. Bob then sends Alice a master secret key (MS), encrypted with Alice's public key as EMS = K_A^+(MS), from which all other keys for the TLS session are generated. A potential issue is that it takes 3 RTTs before the client can start receiving data, including the TCP handshake.

- *Keys.* Using the same key for more than one cryptographic function is considered bad, so there are different keys for message authentication codes (MACs) and encryption: K_c, the encryption key for data sent from client to server; M_c, the MAC key for data sent from client to server; and K_s and M_s for data from server to client. The keys are derived by a KEY DERIVATION FUNCTION (KDF), which takes the master secret and possibly additional random data and creates new keys.
- *Records.* TCP provides a byte-stream abstraction. Data cannot simply be encrypted in-stream as written into the TCP socket: where would the MAC go? If at the end, there would be no message integrity until all data had been received and the connection closed. So the stream is broken into a series of RECORDS: each client-to-server record carries a MAC created with M_c, the receiver can act on each record as it arrives, and the record (length, data, MAC) is encrypted with the symmetric key K_c and passed to TCP.
- *Attacks on the data stream.* A man in the middle can intercept TCP segments and reorder them (manipulating the sequence numbers in the unencrypted TCP header) or replay them. The solutions are TLS sequence numbers (data and TLS sequence number are incorporated into the MAC) and nonces.
- *Connection close.* In a TRUNCATION ATTACK, the attacker forges a TCP connection-close segment, so one or both sides think there is less data than there actually is. The solution is record types, with one type for closure (type 0 for data, type 1 for close), and the MAC is computed over the data, type, and sequence number.

TLS provides an API that any application can use. From HTTP's point of view, HTTP/1.0 runs over TCP, HTTP/2 runs over TLS over TCP, and HTTP/3 runs a slimmed HTTP/2 over QUIC, which incorporates TLS, over UDP.

A CIPHER SUITE is a set of algorithms used for key generation, encryption, MAC, and digital signature. TLS 1.3 (2018) has a more limited choice of cipher suites than TLS 1.2 (2008): only 5 choices rather than 37. It requires Diffie-Hellman for key exchange (rather than DH or RSA), and uses a combined encryption and authentication algorithm ("authenticated encryption") for data rather than serial encryption and authentication; 4 of the suites are based on AES, and HMAC uses the SHA cryptographic hash function (SHA-256 or SHA-384).

TLS 1.3 handshake. The handshake takes 1 RTT. (1) The client TLS hello message guesses the key agreement protocol and parameters (DH) and indicates the cipher suites it supports. (2) The server TLS hello message chooses the key agreement protocol and parameters and the cipher suite, and includes a server-signed certificate. (3) The client checks the server certificate, generates the key, and can now make an application request (for example, an HTTPS GET). A 0-*RTT* handshake resumes an earlier connection between client and server: the initial hello message already contains application data, encrypted using the "resumption master secret" from the earlier connection, and the server's hello can carry application data in reply. It is vulnerable to replay attacks, which may

be acceptable for HTTP GETs or client requests that do not modify server state.

8.6 Network-layer security: IPsec

Definition: IPsec provides datagram-level encryption, authentication, and integrity, for both user traffic and control traffic (for example BGP and DNS messages). It has two modes. In transport mode, only the datagram payload is encrypted and authenticated. In tunnel mode, the entire datagram is encrypted and authenticated, and the encrypted datagram is encapsulated in a new datagram with a new IP header and tunneled to the destination.

There are two IPsec protocols: the Authentication Header (AH) protocol (RFC 4302) provides source authentication and data integrity but not confidentiality; the Encapsulation Security Protocol (ESP) (RFC 4303) provides source authentication, data integrity, and confidentiality, and is more widely used than AH.

Definition: Before data is sent, a security association (SA) is established from the sending to the receiving entity; SAs are directional. The sending and receiving entities maintain state information about the SA, just as TCP endpoints maintain state: IP is connectionless, but IPsec is connection-oriented. For an SA from router R1 (200.168.1.100) to R2 (193.68.2.23), R1 stores a 32-bit identifier, the Security Parameter Index (SPI); the origin SA interface (200.168.1.100); the destination SA interface (193.68.2.23); the type of encryption used and the encryption key; and the type of integrity check used and the authentication key.

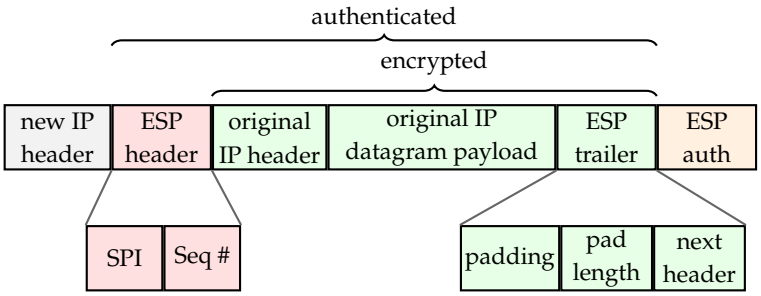


Figure 76: An IPsec datagram in tunnel mode with ESP, and the contents of the ESP header and trailer.

In the IPsec datagram of Figure 76, the ESP trailer holds padding for block ciphers; the ESP header holds the SPI, so the receiving entity knows what to do, and a sequence number, to thwart replay attacks; and the MAC in the ESP auth field is created with the shared secret key. In tunnel mode, R1 appends the ESP trailer to the original datagram (which includes the original header fields), encrypts the result using the algorithm and key specified by the SA, appends the ESP header to the front of this encrypted quantity, creates an authentication MAC over the whole using the algorithm and key specified in the SA, appends the MAC to form the payload, and creates a new IP header, with new header fields, addressed to the tunnel endpoint.

Sequence numbers. For a new SA, the sender initializes the sequence number to 0; each time a datagram is sent on the SA, the sender increments the counter and places its value in the sequence number field. The goal is to prevent an attacker from sniffing and replaying a packet, since receipt of duplicate, authenticated IP packets may disrupt service: the destination checks for duplicates, and instead of keeping track of all received packets, it uses a window.

IPsec databases. For a given datagram, the sender needs to know whether to use IPsec and which SA to use, which may depend on the source and destination IP addresses and the protocol number. This policy is stored in the Security Policy Database (SPD), which says "what" to do. An endpoint holds SA state in the Security Association Database (SAD), which says "how" to do it: when sending an IPsec datagram, R1 accesses its SAD to determine how to process the datagram, and when an IPsec datagram arrives at R2, R2 examines its SPI, indexes its SAD with the SPI, and processes the datagram accordingly.

Suppose Trudy sits somewhere between R1 and R2 and does not know the keys. With ESP in tunnel mode she cannot see the original contents of a datagram, including the source and destination IP addresses, transport protocol, and application port (all encrypted); she cannot flip bits without detection (the MAC); she cannot masquerade as R1 using R1's IP address (the MAC requires the shared key); and she cannot replay a datagram (sequence numbers).

IKE. An SA can be established manually in the IPsec endpoints, for example

SPI	12345	Protocol	ESP
Source IP	200.168.1.100	Encryption algorithm	3DES-cbc
Dest IP	193.68.2.23	HMAC algorithm	MD5
Encryption key	`0x7aeaca...`	HMAC key	`0xc0291f...`

but manual keying is impractical for a VPN with hundreds of endpoints, so IPsec uses IKE (Internet Key Exchange). Authentication (proving who you are) uses either a *pre-shared secret* (PSK), where both sides start with a secret and run IKE to authenticate each other and to generate IPsec SAs (one in each direction) including encryption and authentication keys, or a *PKI* (public/private keys and certificates), where both sides start with a public/private key pair and a certificate and run IKE to authenticate each other and obtain IPsec SAs (one in each direction), similarly to the TLS handshake. IKE has two phases. Phase 1 establishes a bidirectional IKE SA (different from an IPsec SA), also known as an ISAKMP security association, and has two modes: *aggressive mode* uses fewer messages, while *main mode* provides identity protection and is more flexible. In phase 2, ISAKMP is used to securely negotiate the IPsec pair of SAs. In summary, IKE exchanges the algorithms, secret keys, and SPI numbers; either the AH or the ESP protocol (or both) is used, AH providing integrity and source authentication and ESP additionally providing encryption; and IPsec peers can be two end systems, two routers or firewalls, or a router or firewall and an end system.

8.7 Security in wireless and mobile networks

802.11. An arriving mobile client must associate with an access point (AP), establishing communication over the wireless link; perform mutual authentication with the network; and encrypt over-the-air frames, both during authentication and afterwards (user data frames). The steps are (1) *discovery of security capabilities*: the AP advertises its presence with a beacon frame or the device sends a probe request frame, the AP sends the forms of authentication and encryption it provides, and the device requests the forms it wants; during this initial exchange the device is not authenticated and encryption keys are not known; and (2) *mutual authentication and shared symmetric key derivation*: the authentication server (AS) and mobile may already share a common secret (for example, a password) and use it, with nonces (to prevent replay attacks) and cryptographic hashing (for message integrity), to authenticate each other and derive a symmetric key (such as an AES key) for the session's frames.

Definition: WiFi security has evolved from WEP (Wired Equivalent Privacy) through WPA (WiFi Protected Access) and WPA2 (both 2004) to WPA3 (2018). *Personal* mode, common in homes and small offices, uses the same password for all devices, with a protocol between the user device and the AP: WPA2/3-PSK (pre-shared key). *Enterprise* mode, common in enterprises, is password- and/or certificate-based with per-user credentials, and runs protocols among the user device, the AP, and an authentication server (such as RADIUS), mostly between the device and the server with the AP acting as a pass-through: EAP (Extensible Authentication Protocol: EAP-TLS, EAP-TTLS, and many more).

WPA2-PSK. The same password is configured on the AP for all devices and known ahead of time ("pre-shared") by the devices. Authentication has two phases.

1. *802.11 open "authentication"* (packets 1–5 of a sample trace): the AP broadcasts beacon frames (1); the client sends a probe request (2) and the AP a probe response (3); the client sends an 802.11 authentication frame requesting "Open System", that is, no password authentication (4), and the AP responds with an authentication frame with status "Successful" (5). The client is now associated with the AP, but no real authentication has happened yet.
2. *The 4-way EAPOL handshake* (EAPOL, Extensible Authentication Protocol over LAN): with an unsecured 802.11 session established and the shared secret known to both, the client and AP use the shared secret and nonces to generate a new one-time session key for encrypting this session. The shared secret encrypts the EAPOL messages and provides their message integrity codes (MICs); nonces guarantee liveness and protect against replay attacks; and the new, one-time AES session key protects against brute-force decryption attacks on the shared secret. The AP sends an 802.1X EAPOL message announcing WPA2-PSK (RSN) with nonce1 (6); the client uses the shared secret and nonce1 to generate a reply with a MIC and also sends nonce2 (7); the AP checks the MIC (verifying the client's liveness), generates a new AES session key using the shared secret, and sends it to the client with a MIC (8); the client verifies the MIC,

sends an ACK (9), and installs the AES session key. Both sides now know the AES key, and the shared secret was never actually exchanged.

802.1X enterprise authentication. The SUPPLICANT is the client device needing authentication before network access; the AUTHENTICATOR (the AP) relays messages between the client (over EAPoL) and the AUTHENTICATION SERVER (over RADIUS) to authenticate the client and authorize its use of the WLAN; the authentication server holds the clients' authentication and access-rights information. EAP runs primarily between the client and the authentication server, with the AP mostly a pass-through, and credentials are stored at the clients and the authentication server. There are about 40 different EAP protocols, differing in their credentials (passwords, certificates, SIM cards, tokens); commonly used are EAP-MD5, EAP-TLS, EAP-PEAP, EAP-TTLS, EAP-FAST, and Cisco LEAP.

EAP-TTLS (Tunneled Transport Layer Security, RFC 5281) has two phases, followed by the 4-way EAPOL handshake to encrypt the local wireless channel (not for authentication, which is already done), as in WPA2-PSK.

- *Phase 1*: the client authenticates the AS (for example, via the server's public key) and a TLS tunnel is established between the client and the AS. After EAPOL start and an identity request/response, in which the client provides its name (1–3), the AP tells the RADIUS server that a client is seeking authentication (4), and the RADIUS server initiates EAP-TTLS with the client (5). The client sends nonce1 and its acceptable encryption schemes to the AS (6); the AS responds with its certificate (for authentication), nonce2, and the chosen encryption method (7); the client authenticates the server and sends client key information, encrypted with the server's public key (8); and the AS finishes the handshake (9). The AS is now authenticated by the client, and the session between client and AS is encrypted.
- *Phase 2*: the AS authenticates the client (for example, via the client's password) within the encrypted tunnel, and the tunnel is then torn down. The AS sends an identity request (10) and the client responds with its user name (11); the AS issues a challenge, for a password or another credential depending on the EAP-TTLS authentication method (12), and the client responds with the credential (13); the AS informs the AP that the client is authenticated, sending the information the AP and client need for channel encryption, including the client key (14), and the AP passes the EAP success message to the client (15). Next, the client and AP agree on channel encryption with the EAPoL 4-way handshake, and the client can then run DHCP.

A local authentication server may use proxies: a local RADIUS server can proxy authentication to a local directory server (such as Azure Active Directory). In EDUROAM, a user visiting a remote WiFi network is authenticated by proxying from the local eduroam RADIUS server through the national (and world-region) eduroam RADIUS servers to the RADIUS server of the user's home "realm", and the local server accepts the remote authentication decision.

4G/5G. An arriving mobile must associate with a base station, establishing communication over the wireless link, and authenticate itself to the network and authenticate the network. There are notable differences from WiFi: the UE's SIM card provides a global identity and contains shared keys, and the services available in a visited network depend on a (paid) service subscription in the home network. 5G authentication involves the AMF/SEAF (access and mobility management and security anchor function) in the visited network and the AUSF (authentication server) and UDM (unified data management) in the home network, with a shared secret on the UE's SIM known to the home network.

1. The UE sends a registration request with its identity (IMSI) information to the visited AMF, which locates the AUSF in the device's home network and establishes a secure TLS tunnel to it (all messages are authenticated and encrypted), sending an authenticate request with the IMSI information and the visited network's name.
2. The home AUSF contacts the home UDM, which creates an authentication token (using the UE's key) and a nonce, and signs them (this will prove to the UE that the home UDM generated the token). The UDM computes and saves the expected response (XRES) that the UE will compute.
3. The home AUSF sends the authentication token and nonce to the AMF, which forwards them to the UE in an EAP request.
4. The UE receives the token and nonce, determines from the shared secret on its SIM that the token is indeed from its home AUSF, computes its expected response XRES_C, and sends it to the home AUSF (in an EAP response, through the AMF).
5. The AUSF compares XRES_C with the XRES computed earlier. On success, the home UDM is updated, and the home AUSF generates an ANCHOR KEY and sends it to the visited AMF (the device computes the anchor key on its own); the AMF and device use the anchor key to secure their message exchange in the visited network.

In 5G, the home AUSF makes the authentication comparison (the MME in the visited network made it in 4G), and the home AUSF generates the anchor key used in the visited network and informs the UE, the AMF, and the gNB in the visited network; the crypto keys on the UE's SIM are known only to the UE and the home AUSF.

8.8 Operational security: firewalls and IDS

Definition: A FIREWALL isolates an organization's internal network from the larger Internet, allowing some packets to pass and blocking others. It sits between the administered network (the trusted "good guys") and the public Internet (the untrusted "bad guys").

Firewalls prevent denial-of-service attacks (such as SYN flooding, in which an attacker establishes many bogus TCP connections so that no resources are left for "real" connections), prevent illegal

modification of or access to internal data (for example, an attacker replacing the CIA's homepage), and allow only authorized access to the inside network (a set of authenticated users or hosts). There are three types: stateless packet filters, stateful packet filters, and application gateways.

Stateless packet filtering. An internal network is connected to the Internet via a router firewall that filters packet by packet, deciding whether to forward or drop a packet (should an arriving packet be allowed in, a departing packet let out?) based on the source and destination IP addresses, the TCP/UDP source and destination port numbers, the ICMP message type, and the TCP SYN and ACK bits. For example, blocking incoming and outgoing datagrams with IP protocol field 17 and those with source or destination port 23 blocks all incoming and outgoing UDP flows and telnet connections; blocking inbound TCP segments with ACK = 0 prevents external clients from making TCP connections with internal clients, but allows internal clients to connect to the outside. More examples:

Policy	Firewall setting
no outside web access	drop all outgoing packets to any IP address, port 80
no incoming TCP connections, except those for the institution's public web server only	drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80
prevent web radios from eating up the available bandwidth	drop all incoming UDP packets, except DNS and router broadcasts
prevent your network from being used for a smurf DoS attack	drop all ICMP packets going to a "broadcast" address (such as 130.207.255.255)
prevent your network from being tracerouted	drop all outgoing ICMP TTL-expired traffic

Definition: An ACCESS CONTROL LIST (ACL) is a table of rules, applied top to bottom to incoming packets: (action, condition) pairs, much like OpenFlow forwarding.

Action	Source address	Dest address	Protocol	Source port	Dest port	Flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	–
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	–
deny	all	all	all	all	all	all

This ACL lets internal hosts browse the web and query outside DNS servers, and nothing else.

Stateful packet filtering. A stateless packet filter is a heavy-handed tool: it admits packets that "make no sense", such as a packet with destination port 80 and the ACK bit set although no TCP

connection was established (the second rule above admits it). A STATEFUL PACKET FILTER tracks the status of every TCP connection: it tracks connection setup (SYN) and teardown (FIN) to determine whether incoming and outgoing packets "make sense", and times out inactive connections at the firewall, no longer admitting their packets. The ACL is augmented to indicate when the connection-state table must be checked before a packet is admitted (in the example, for the second and fourth rules, the inbound TCP and UDP replies).

Application gateways. An APPLICATION GATEWAY filters packets on application data as well as on IP/TCP/UDP fields. For example, to allow selected internal users to telnet outside: (1) all telnet users are required to telnet through the gateway; (2) for authorized users, the gateway sets up a telnet connection to the destination host and relays data between the two connections (host to gateway, gateway to remote host); (3) the router filter blocks all telnet connections not originating from the gateway.

Firewalls and gateways have limitations. With IP spoofing, a router cannot know whether data "really" comes from the claimed source. If multiple applications need special treatment, each needs its own application gateway, and client software must know how to contact the gateway (for example, the IP address of a proxy must be set in a web browser). Filters often use an all-or-nothing policy for UDP. There is a tradeoff between the degree of communication with the outside world and the level of security, and many highly protected sites still suffer from attacks.

Definition: Packet filtering operates on TCP/IP headers only, with no correlation check among sessions. An INTRUSION DETECTION SYSTEM (IDS) performs DEEP PACKET INSPECTION, looking at packet contents (for example, checking character strings in packets against a database of known virus and attack strings), and examines the correlation among multiple packets to detect port scanning, network mapping, and DoS attacks.

Multiple IDSs can perform different types of checking at different locations: for example, one IDS sensor where the firewall connects to the DEMILITARIZED ZONE (the segment holding the public web, FTP, and DNS servers) and others inside the internal network.

Source. Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, Chapter 8 slides (9th edition).